

# Modèles et Algorithmes pour les Graphes (MAG)

Équipe pédagogique :

Rumen Andonov <[randonov@irisa.fr](mailto:randonov@irisa.fr)> : CM et TD (G1 et G2 Info, G2 Miage)

César Viho <[cesar.viho@irisa.fr](mailto:cesar.viho@irisa.fr)> : TD (G3 Info, G1 Miage)

Olivier Dameron <[Olivier.Dameron@irisa.fr](mailto:Olivier.Dameron@irisa.fr)> : TD (G2 Info)

Université de Rennes et INRIA Rennes Bretagne-Atlantique



1. Généralités sur les graphes : exemples de graphes comme modèles de situations concrètes, et questions associées pour découverte des notions (chemins, PCC, fermeture transitive, descendance, clique, CFC, arbres, etc.).
2. Représentation des graphes, structures de données associées : plusieurs représentations (matrice d'adjacence, liste des prédécesseurs, liste des successeurs), équivalence et passage de l'une à l'autre.
3. Notions de complexité des algorithmes (ordre de grandeur des fonctions).
4. Parcours en profondeur *Depth-first search (DFS)* et en largeur *Breadth-first search (BFS)*. *Directed Acyclic Graph (DAG)*.
5. L'arbre couvrant de poids minimal (ACM) (*Minimum spanning tree*) : algorithme de Prim et algorithme de Kruskal.
6. Le problème du flot maximum dans les réseaux de transport.
7. Les problèmes du plus court/long chemin (PCC/PLC) :
  - algorithme de Dijkstra. *Binary heap* (tas binaire);
  - algorithme de Bellman-Ford. Découverte des circuits négatifs;
  - PCC/PLC dans un DAG (ordre topologique).
  - Programmation dynamique. Algorithmes pour ordonnancer les tâches.
8. Chemins, flots et programmes linéaires dans les graphes.

Ce cours est fondé essentiellement sur les références suivantes.

- *Algorithms*, S. Dasgupta, C. H. Papadimitriou, et U. V. Vazirani, McGraw-Hill 2006 (*the undergraduate Algorithms course at Berkeley and U.C. San Diego*)
- *Graphes et algorithmes*, Michel Gondran et Michel Minoux, Eyrolles, 1995
- *Introduction à algorithmique*, T. Cormen, C. Leiserson, R. Rivest, DUNOD, 1994.

Généralités, notions de base, exemples d'applications

## Définitions formelles et notations : graphe, graphe orienté et non orienté

Un *graphe* est un doublet  $G = (V, E)$ , où  $V = \{v_1, v_2, \dots, v_n\}$  est l'ensemble des **sommets/noeuds** et  $E$  est un ensemble de couples  $(u, v) \in V \times V$ . Si tous les couples  $(u, v) \in E$  sont symétriques, le graphe est dit *non orienté*. Sinon, le graphe est *orienté*. Un couple symétrique / non ordonné est dit une *arête*. Un couple non symétrique / ordonné est dit un *arc*.

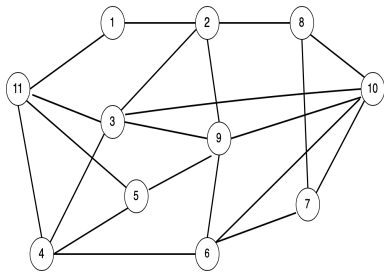


Figure 1 – Un graphe non orienté

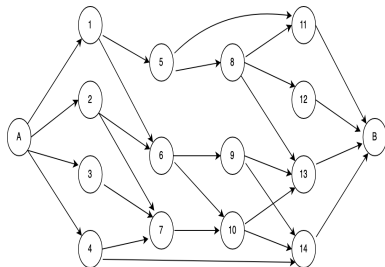


Figure 2 – Un graphe orienté.

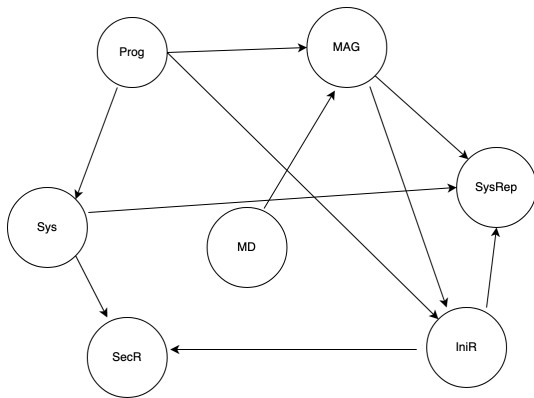
### Obtention d'un diplôme Master d'informatique (Exam. décembre 2008)

Pour obtenir un Master d'informatique, il est nécessaire d'avoir passé un certain nombre de modules. Chaque module demande un certain nombre de prérequis. Un master simplifié pourrait se composer (prérequis entre parenthèses) de :

- Système (Programmation) ;
- MAG (Programmation, Math Discrètes) ;
- Sécurité réseau (Système, Initiation réseau) ;
- Initiation réseau (Programmation, MAG) ;
- Systèmes répartis (MAG, Système, Initiation réseau).

1. Modéliser les prérequis à l'aide d'un graphe.
2. Quel est le nombre minimum de semestres pour obtenir ce master ? On suppose bien sur que vous passez tous les examens avec succès, que chaque module dure un semestre et est enseigné chaque semestre. Le nombre de modules suivis par un étudiant pendant un semestre n'est pas limité et il n'y a pas de problème d'emploi du temps. Indiquer l'algorithme utilisé et justifiez votre choix.
3. Un étudiant décide d'étudier un module dès qu'il a obtenu les modules prérequis. Quel est le nombre maximum de modules qu'il devra suivre simultanément en appliquant cette stratégie ? Quel algorithme permet de le calculer ? Justifiez votre choix.

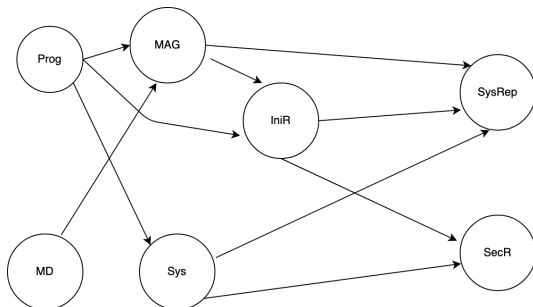
## 1. Modéliser les prérequis à l'aide d'un graphe.



Modélisation à l'aide d'un graphe : les sommets correspondent aux modules, les arcs  
- aux prérequis.

## Obtention d'un diplôme Master d'informatique : suite 2

1. Quel est le nombre minimum de semestres pour obtenir ce master ?
2. Quel est le nombre maximum de modules qu'il devra suivre simultanément ?

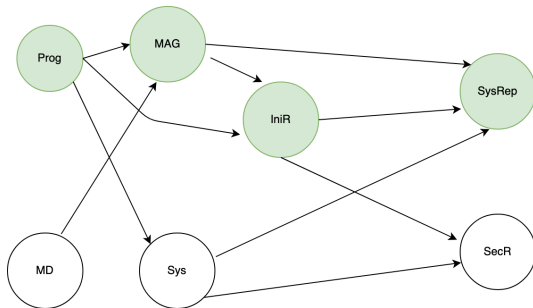


Les modules sont ordonnancés de gauche à droite sur l'axe horizontal.



## Obtention d'un diplôme Master d'informatique : suite 3

1. Quel est le nombre minimum de semestres pour obtenir ce master ?
2. Quel est le nombre maximum de modules qu'il devra suivre simultanément



- Au moins 4 semestres sont nécessaires pour obtenir ce master. Cela peut se calculer par un algorithme du plus long chemin dans un graphe sans circuit.
- Un étudiant peut suivre maximum deux modules par semestre. Nous allons étudier l'algorithme adéquat qui permet de calculer ce nombre.

## Exemple de graphes : suite

### Construction d'un pavillon

La construction d'un pavillon demande la réalisation d'un certain nombre de tâches. La liste des tâches à effectuer, leur durée et les contraintes d'antériorités à respecter sont données dans le table ci-dessus. Le travail commençant à la date 0, on cherche un planning des opérations qui permet de minimiser la durée totale.

| Code tâche | libellé                 | durée<br>(semaines) | antériorité |
|------------|-------------------------|---------------------|-------------|
| A          | Travaux de maçonnerie   | 7                   | -           |
| B          | Charpente de la toiture | 3                   | A           |
| C          | Toiture                 | 1                   | B           |
| D          | Installation électrique | 8                   | A           |
| E          | Façade                  | 2                   | D,C         |
| F          | Fenêtres                | 1                   | D,C         |
| G          | Aménagement du jardin   | 1                   | D,C         |
| H          | Travaux de plafonnage   | 3                   | F           |
| J          | Mise en peinture        | 2                   | H           |
| K          | Emménagement            | 1                   | E,G,J       |

Figure 3 – Liste des tâches, durée et contraintes.

Devoir maison : Illustrer la construction de ce pavillon par un graphe adéquat.

## Définitions formelles et notations : chemins, circuit, chaîne, cycle

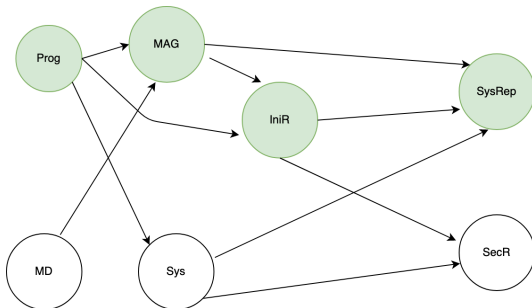
- Un **chemin**  $P$  dans un graphe orienté  $G$  est une suite d'arcs dont les extrémités droites/gauches coïncident de la façon suivante :  
 $P = ((u_0, u_1), (u_1, u_2), (u_2, u_3), \dots, (u_{k-1}, u_k))$ . La longueur du  $P$  est le nombre d'arcs (ici  $k$ ).
- Si  $u_0 = u_k$ , le chemin est appelé **circuit**.
- Un *chemin élémentaire* est défini par une suite de sommets sans répétition (sauf pour le premier et le dernier sommet).
- Si le graphe est non orienté, on utilise **chaîne** et **cycle** à la place de chemin et circuit.
- Un graphe sans cycle/circuit est appelé **acyclique** / *sans circuit*.
- Un **graphe non orienté** tel que chaque couple de sommets est connecté par une chaîne est dit **connexe**.
- Un **graphe orienté** tel que chaque couple de sommets  $(u, v)$  est connecté par un chemin dans les deux sens (c.-à-d. de  $u$  à  $v$  et de  $v$  à  $u$ ) est dit **fortement connexe**. On dit aussi que les sommets  $u$  et  $v$  sont mutuellement accessibles.
- Un graphe, non orienté, connexe et acyclique est dit **arbre**.

Soit le graphe orienté  $G = (V, E)$ .

- Pour un arc  $(x, y) \in E$ ,  $x$  est l'origine et  $y$  est l'extrémité de l'arc.  
On dit aussi que  $y$  est successeur de  $x$ , ou que  $x$  est prédécesseur de  $y$ .
- $\Gamma^+(x) = \{y \in V \mid (x, y) \in E\}$  est l'ensemble des successeurs de  $x$ .
- $\Gamma^{-1}(x) = \{z \in V \mid (z, x) \in E\}$  est l'ensemble des prédécesseurs de  $x$ .
- $d^+(x)$  est le degré sortant du sommet  $x$ , c'est-à-dire le nombre d'arcs sortant de  $x$ .
- $d^-(x)$  est degré entrant du sommet  $x$ , c'est-à-dire le nombre d'arcs dirigés vers le sommet  $x$ .
- $d(x) = d^+(x) + d^-(x)$  est le degré de  $x$ .
- Un chemin  $P$  allant de  $x$  à  $y$ , dont on ne précise pas les sommets intermédiaires, sera noté :  $P = x \rightsquigarrow y$ .  $y$  est alors un descendant de  $x$  et  $x$  un ascendant de  $y$ .

Dans le cas d'un graphe non-orienté le degré d'un sommet est le nombre de liens (arêtes ou arcs) reliant ce sommet, avec les boucles comptées deux fois.

## Vocabulaire : Illustration pour le problème "Obtention d'un diplôme"



$\Gamma^+(PROG) = \{MAG, InR, Sys\}$  : l'ensemble des successeurs de *PROG*,  
 $d^+(PROG) = 3$ .

$\Gamma^-(PROG) = \emptyset$ ,  $d^-(PROG) = 0$  : *PROG* est un sommet *source*.

$\Gamma^-(SysRep) = \{InR, MAG, Sys\}$  : l'ensemble des prédécesseurs de *SysRep*.  
 $d^-(SysRep) = 3$

$\Gamma^+(SysRep) = \emptyset$ ,  $d^+(SysRep) = 0$  : *SysRep* est un sommet *target/puits*

$d^+(MAG) = 2$ ,  $d^-(MAG) = 2$  : *MAG* est un sommet *intermédiaire*.

Soit le graphe  $G = (V, E)$  où  $|V| = n$  et  $|E| = m$ . On dit parfois que  $G$  est d'ordre  $n$ .

- Deux sommets sont **indépendants** s'ils ne sont **pas connectés**, c'est-à-dire pas *adjacents*.
- Il existe 2 cas extrêmes pour l'ensemble de ses arêtes : soit **le graphe n'a aucune arête** : on parle alors de **stable**. Soit toutes les arêtes possibles pouvant relier les sommets 2 à 2 sont présentes : le graphe est dit alors *complet*.
- On dit aussi qu'un ensemble de sommets est *indépendant (ou stable)* s'il n'y a pas deux de ses sommets adjacents.
- Une **clique** est un **sous-graphe complet**.
- Un stable est un sous-graphe sans arête.

Pour un graphe général, il est souvent intéressant de rechercher de tels sous-graphes.

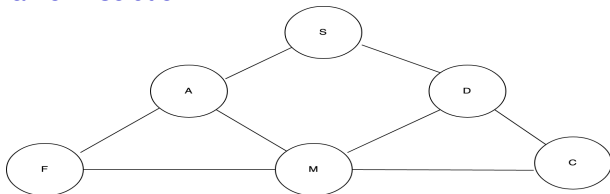
## Exemple de graphes : (clique, stable)

### Planning d'examen

Les cinq étudiants : Dupont, Dupond, Durand, Duval et Duduche doivent passer certaines épreuves parmi les suivants : Français, Anglais, Dessin, Couture, Mécanique et Solfège. L'examen se déroulant par écrit, on désire que tous les étudiants qui doivent subir une même épreuve le fassent simultanément. Chaque étudiant ne peut se présenter qu'à une épreuve au plus chaque jour. Ci-dessous la liste des épreuves que doit passer chaque étudiant : Dupont : Français, Anglais, Mécanique ; Dupond : Dessin, Couture ; Durand : Anglais, Solfège ; Duval : Dessin, Couture, Mécanique ; Duduche : Dessin, Solfège.

1. Quel est le nombre maximum d'épreuves que l'on peut organiser le même jour ?
2. Quel est le nombre minimum de jours nécessaires à l'organisation de toutes les épreuves ?

## Planning d'examen : solution

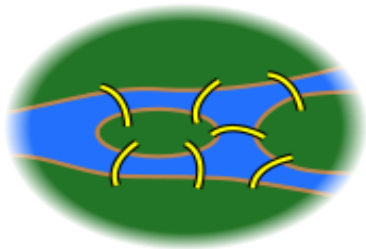
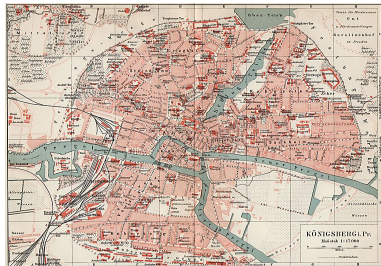


On construit le graphe  $G = (V, E)$  tel que  $V = \{F, A, D, C, M, S\}$  (un sommet par épreuve) et une arête relie deux sommets si et seulement si un même candidat doit subir les épreuves correspondants.

1. Le nombre maximum d'épreuves que l'on peut organiser le même jour est donné alors par le cardinal du stable maximum  $\{F, S, C\}$ , donc 3.
2. Pour trouver le nombre minimum de jours nécessaires à l'organisation de toutes les épreuves on peut appliquer l'algorithme suivant : tant qu'il y a un graphe non-vidé, on trouve le stable max et on enlève du graphe ; sur le graphe ainsi réduit on applique la même stratégie. La réponse est trois jours dans le cas considéré ; le 1<sup>er</sup> jour on organise  $\{F, S, C\}$ , le 2<sup>ème</sup> jour on organise  $\{A, D\}$ , le dernier jour on organise  $\{M\}$ ,



## Exemple des graphes : les 7 ponts de Königsberg



La ville de Königsberg sur le Pregel et ses 7 ponts. Déterminer s'il existe ou non une promenade dans les rues de Königsberg permettant, à partir d'un point de départ au choix, de passer une et une seule fois par chaque pont, et de revenir à son point de départ. Ce problème (résolu par Leonhard Euler en 1736) est considéré comme l'origine de la théorie des graphes.

- On appelle **chaîne eulérienne** (resp. cycle eulérien) une chaîne (resp. un cycle) qui emprunte une fois et une seule chaque arête du graphe.
- **Théorème d'Euler** : Un graphe connexe a une chaîne eulérienne si et seulement si tous ses sommets ont un degré pair ou tous ses sommets ont un degré pair sauf deux sommets.  
On peut démontrer que :
  - si le graphe n'a pas de sommet impair, alors il a un cycle eulérien.
  - un graphe ayant plus de deux sommets impairs ne possède pas de chaîne eulérienne (donc non pour le graphe de la ville de Königsberg).
  - si le graphe a deux sommets impairs, ce sont les extrémités de la chaîne eulérienne.

## Les 7 ponts de Königsberg : Modélisation et solution

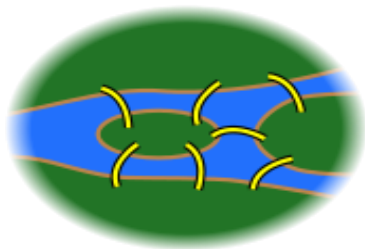


Figure 4 – Les 7 ponts de Königsberg

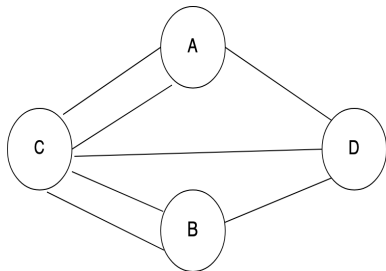


Figure 5 – Représentation des ponts de Königsberg sous forme d'un graphe non-orienté.

L'analyse des degrés des sommets du graphe de la figure (5) montre que

$$d(A) = 3, d(B) = 3, d(C) = 5, d(D) = 3.$$

Ce graphe a plus de deux sommets impairs et ne possède donc pas de cycle eulérien d'après le théorème d'Euler.

### **Modéliser l'accessibilité au centre-ville de l'Utopia (examen 2011/12)**

Un ingénieur du service technique de la ville Utopia propose, pour la zone centrale, le plan sens unique représenté à la figure (6). Avant d'adopter le plan, il convient d'examiner s'il autorise la circulation des automobiles, c'est à dire s'il permet d'aller de n'importe quel point à n'importe quel autre.

1. Enumérer les algorithmes vus en cours pouvant résoudre ce problème. Donner la complexité de ces algorithmes pour un graphe quelconque  $G = (V, E)$ .
2. Appliquer l'algorithme de votre choix sur le graphe de la figure. Le calcul à chaque itération sera clairement indiqué.
3. Vous constaterez facilement que le plan n'est pas acceptable. Quelles sont les zones où les sommets sont mutuellement accessibles ? Quelle est la dénomination de ces zones dans la terminologie spécifique pour l'algorithme en considération ?
4. Trouver le nombre minimum de rues tel que l'inversion du sens de circulation dans chacune de ces rues rend ce plan acceptable. Combien de solutions avez vous trouvées ? Quelles sont ces solutions ?

## Modéliser l'accessibilité au centre-ville de l'Utopia

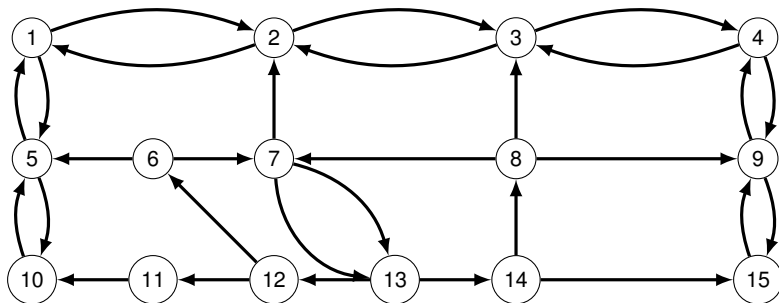


Figure 6 – Le plan de sens unique du centre-ville.

## Modéliser l'accessibilité au centre-ville de l'Utopia : solution

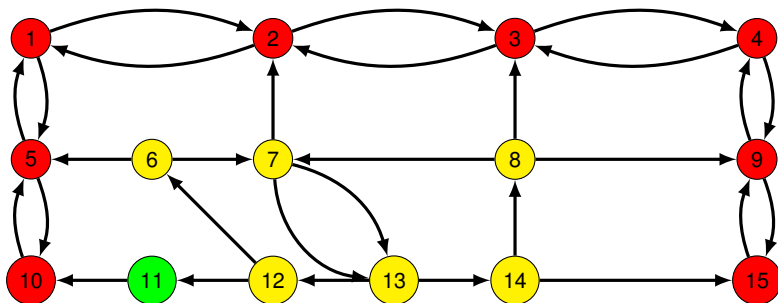


Figure 7 – Les sommets jaunes et verts ne sont pas joignables à partir des sommets rouges. Les trois zones coloriées en rouge, jaune et vert sont des *Composantes Fortement Connexes* (CFC)

## Modéliser l'accessibilité au centre-ville de l'Utopia

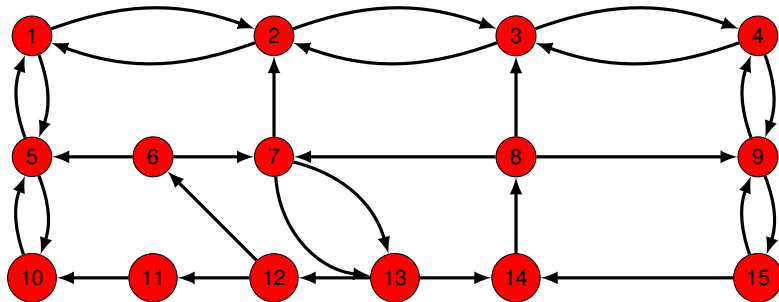


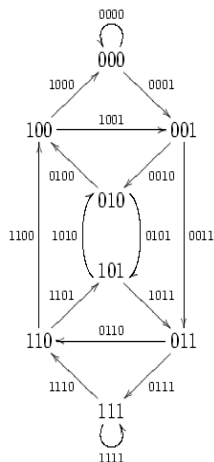
Figure 8 – Le plan devient acceptable si on inverse au moins un arc (par exemple si l'arc (14,15) devient (15,14)).

## Graphe de de Bruijn : à voir en TD

- Un graphe de de Bruijn est un graphe orienté qui permet de représenter les chevauchements de longueur  $n - 1$  entre tous les mots de longueur  $n$  sur un alphabet donné.

Le graphe de de Bruijn  $B(k, n)$  d'ordre  $n$  sur un alphabet  $A$  à  $k$  lettres est construit comme suit. Les sommets de  $B(k, n)$  sont étiquetés par les  $k^n$  mots de longueur  $n$  sur  $A$ . Si  $u$  et  $v$  sont deux sommets, il y a un arc de  $u$  à  $v$  s'il existe deux lettres  $a$  et  $b$ , et un mot  $x$ , tels que  $u = ax$  et  $v = xb$ . La présence d'un arc signifie donc un chevauchement maximal entre deux mots de même longueur.

- Le graphe  $B(2, 3)$  ci-contre est construit sur un alphabet binaire  $A = \{0, 1\}$  pour des mots de longueur  $n = 3$ .





## Réseaux : une application importante pour les graphes

- Soit  $G = (X, U, C)$  un graphe connexe orienté (réseau). A  $\forall$  arc  $u$  on affecte une valeur (*capacité*)  $c_u$  qui est une borne supérieure du flot sur l'arc.
- Soit deux sommets particuliers  $s \in X$  (source) et  $t \in X$  (puits). Considérons le graphe  $G^0 = (X, U^0)$  où  $U^0 = U \cup \{t, s\}$ . L'arc  $(t, s)$  est appelé l'*arc de retour* du flot (numéroté 0). Notons  $n = |X|$  et  $m = |U|$ .

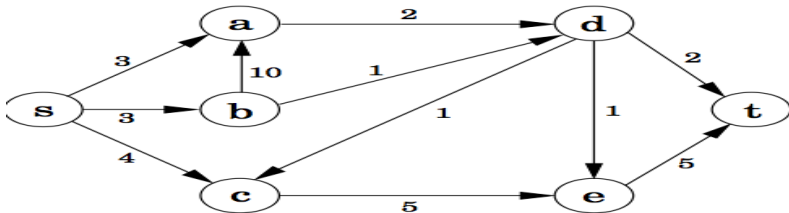
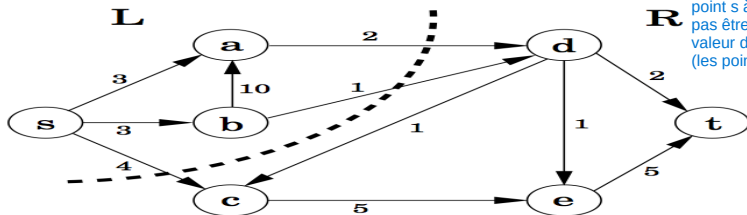


Figure 9 – Un réseau  $G = (X, U, C)$ . Chaque arc  $u \in U$  est muni d'une capacité  $c_u$ .

On cherche dans  $G$  un **flot** de  $S$  à  $T$  compatible avec les capacités des arcs et qui soit de valeur maximale.

## Définition d'une coupe et de sa capacité

- Soit le réseau  $G = (X, U, C)$ . Une coupe séparant  $s$  et  $t$  (notée  $s - t$ ) est une partition  $(L, R)$  de  $X$ , telle que  $s \in L$  et  $t \in R = \bar{L} = X \setminus L$ .
- La capacité  $C(L, R)$  de la coupe  $s - t$  est définie par  $C(L, R) = \sum_{e \in \omega^+(L)} c_e$  où  $\omega^+(L)$  représente le sous-ensemble d'arcs qui ont leurs extrémités initiales dans  $L$  et leurs extrémités terminales dans  $\bar{L} = R$ .
- **Lemme** : La valeur maximale d'un flot de  $s$  à  $t$  compatible avec  $c_u$  n'excède jamais la capacité d'une coupe séparant  $s$  et  $t$ .



la valeur max d'un  
flot ( la distance du  
point  $s$  à  $t$  ) ne doit  
pas être  $>$  à la  
valeur de la coupe  
(les pointiers)

Figure 10 – Une coupe  $s - t$  définie par la partition  $L = \{s, a, b\}$  and  $R = \{c, d, e, t\}$ . Chaque flot  $s \rightsquigarrow t$  doit passer de  $L$  à  $T$  et traverser ainsi la coupe  $s \rightsquigarrow t$ . Donc, la valeur d'aucun flot ne peut dépasser la valeur de la capacité  $C(L, R)$  qui vaut 7. Ainsi, chaque flot  $s \rightsquigarrow t$  de valeur 7, devrait être maximal (**certificat d'optimalité**).

## Exemple de graphes : Le problème de mariage (couplage)

Soit  $G = (V, E)$  un graphe. Rappel :

- Un couplage  $M$  est un ensemble d'arêtes deux à deux non adjacentes.
- Une couverture par sommets  $C$  est un ensemble de sommets tel que chaque arête est incidente avec au moins un sommet de  $C$ .

Soit un graphe biparti  $G(U \cup V, E)$ . L'ensemble des arêtes représente les couples compatibles. Il s'agit de trouver le *couplage maximum* (un couplage contenant le plus grand nombre possible d'arêtes) en satisfaisant les contraintes des couples compatibles.

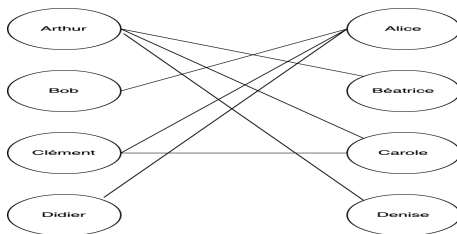
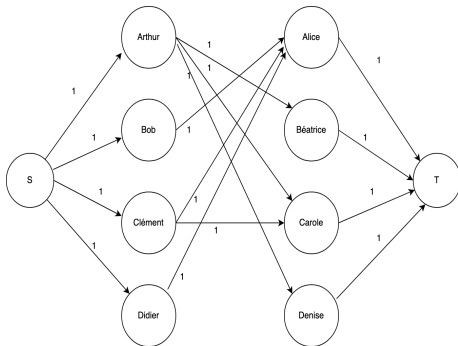
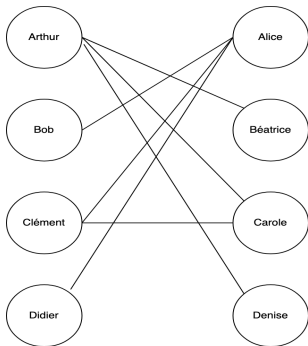


Figure 11 – Il s'agit de marier ces personnes en satisfaisant les contraintes des couples compatibles. On peut voir ici (par énumération) qu'on ne peut marier que trois couples.

- Montrer que ce problème se réduit au problème du flot maximum.

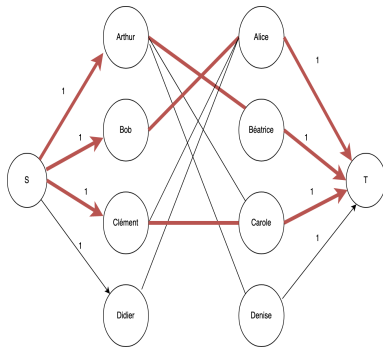
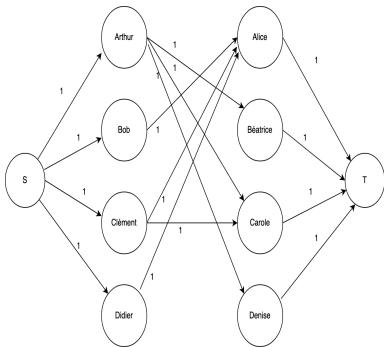
## Le problème du flot maximum : un grand classique du domaine

- Soit  $R = (X, U, C)$  un graphe connexe orienté (réseau). A  $\forall$  arc  $u$  on affecte une valeur (*capacité*)  $c_u$  qui est une borne supérieure du flux sur l'arc.
- Soit deux sommets particuliers  $s \in X$  (source) et  $t \in X$  (puits). Considérons le graphe  $G^0 = (X, U^0)$  où  $U^0 = U \cup \{t, s\}$ .
- la loi de conservation du flot doit être satisfaite pour chaque sommet.
- Ci-dessous le problème de mariage modélisé comme un problème du flot maximum. Les capacités sont fixées à 1 partout.



## Le problème du flot maximum : solution

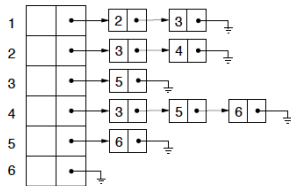
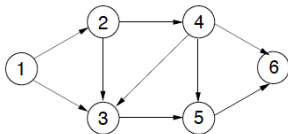
- Dans le graphe de la figure ci-dessous à gauche on cherche le nombre maximum de chemins *sommet-disjoints* (donc ils ne se croisent pas) de  $s$  à  $t$ .
- La solution est donnée sur la figure à droite. On trouve trois chemins *sommet-disjoints* de  $s$  à  $t$ . Cette valeur correspond à la valeur du couplage maximum.



## Représentations des graphes : listes de successeurs

Soit le graphe  $G = (V, E)$  où  $|V| = n$  et  $|E| = m$ .  $G$  peut être représenté en mémoire soit par des listes d'adjacence, soit par une matrice d'adjacence.

- Un graphe et sa représentation par des listes de successeurs.



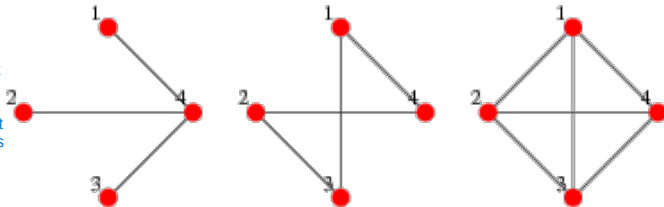
## Représentations des graphes : matrice d'adjacence

Soit le graphe  $G = (V, E)$  où  $|V| = n$  et  $|E| = m$ .  $G$  peut être représenté en mémoire soit par des listes d'adjacence, soit par une matrice d'adjacence.

- La matrice d'adjacence d'un graphe  $G = (V, E)$  où  $|V| = n$ , est une matrice carrée de taille  $n$  et définie par :

$$A(u, v) = \begin{cases} 1 & \text{si } (u, v) \in E \\ 0 & \text{sinon} \end{cases}$$

sources sont  
les lignes  
et  
les puits sont  
des colonnes



ex :  $L1 \rightarrow C2 = 0$   
car 1 n'est pas  
relrier à 2  
mais  $L1 \rightarrow C4 = 1$   
car 1 est relrier  
à 4

$$\begin{pmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}$$

$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

## Représentations des graphes : matrice d'incidence sommet/arc

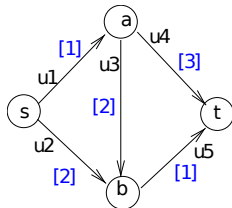
Soit le graphe  $G = (V, E)$  où  $|V| = n$  et  $|E| = m$ .

- La matrice d'incidence sommet/arc d'un graphe orienté est une matrice

$A = [a_{i,j}]$   $i = 1, 2, \dots, |V|$  et  $j = 1, 2, \dots, |E|$  telle que :

$$a_{i,j} = \begin{cases} +1 & \text{si l'arc } u_j \text{ est sortant pour le sommet } i \\ -1 & \text{si l'arc } u_j \text{ est entrant pour le sommet } i \\ 0 & \text{sinon} \end{cases}$$

lignes = sommets  
colonnes = arcs



|     | $u1$ | $u2$ | $u3$ | $u4$ | $u5$ |
|-----|------|------|------|------|------|
| $s$ | +1   | +1   | 0    | 0    | 0    |
| $t$ | 0    | 0    | 0    | -1   | -1   |
| $a$ | -1   | 0    | +1   | +1   | 0    |
| $b$ | 0    | -1   | -1   | 0    | +1   |

Remarque : les valeurs en bleu sont les poids (longueurs) des arcs.



# Complexité des algorithmes : Ordre de grandeur des fonctions

**But :** Etre capable de comparer les algorithmes et pouvoir affirmer :

*Sur toute machine, et quel que soit le langage de programmation, l'algorithme A1 est meilleur que l'algorithme A2 pour les données de grande taille.*

## Critères d'évaluation des algorithmes

- **Validité** : s'effectue à la base de la notion "invariant de boucle". Cet aspect sera vu en détail dans la 2<sup>e</sup> partie du cours.
- **Complexité temporelle** : à définir les notions de "taille de l'entrée" et de "temps d'exécution".
- **Complexité spatiale** : à définir la notion de "place mémoire".
- **Simplicité et clarté** : à tenir compte du *temps humain* à implanter le code.
- **Précision** : pour un algorithme numérique.
- **Stabilité** : pour un algorithme de tri par exemple.

**Conclusion** : Savoir faire des compromis et prendre en compte le contexte dans lequel le programme sera développé et utilisé.

---

**Algorithm 1** Insertion-sort( $A = [a_1, a_2, \dots, a_n]$ )

---

**Require:** An array  $A = [a_1, a_2, \dots, a_n]$  of  $n$  numbers.

**Ensure:** An array  $A = [a'_1, a'_2, \dots, a'_n]$  s.t.  $a'_1 \leq a'_2 \leq \dots a'_n$  and  $[a'_1, a'_2, \dots, a'_n]$  is a permutation of  $[a_1, a_2, \dots, a_n]$ .

```
1: for  $j = 2$  to  $n$  do
2:    $clef \leftarrow A[j]$ 
3:    $i \leftarrow j - 1$ 
4:   while  $i > 0$  and  $A[i] > clef$  do
5:      $A[i + 1] \leftarrow A[i]$ 
6:      $i \leftarrow i - 1$ 
7:   end while
8:    $A[i + 1] \leftarrow clef$ 
9: end for
```

---

## Analyse de complexité de Tri-Insertion

On cherche une mesure du temps d'exécution  $T(n)$  qui doit être la plus *indépendante* possible de l'implémentation et de la machine.

- Associons une constante  $c_i$  à chaque exécution de la  $i$ -ème ligne.
- Soit  $t_j$  le # de fois que le test de la ligne 4 est exécuté pour cette valeur de  $j$ .

| Tri-Insertion(A(n))                                | coût     | # fois                   |
|----------------------------------------------------|----------|--------------------------|
| 1: <b>for</b> $j = 2$ to $n$ <b>do</b>             | 1: $c_1$ | $n$                      |
| 2: $clef \leftarrow A[j]$                          | 2: $c_2$ | $n - 1$                  |
| 3: $i \leftarrow j - 1$                            | 3: $c_3$ | $n - 1$                  |
| 4: <b>while</b> $i > 0$ and $A_i > clef$ <b>do</b> | 4: $c_4$ | $\sum_{j=2}^n t_j$       |
| 5: $A[i + 1] \leftarrow A[i]$                      | 5: $c_5$ | $\sum_{j=2}^n (t_j - 1)$ |
| 6: $i \leftarrow i - 1$                            | 6: $c_6$ | $\sum_{j=2}^n (t_j - 1)$ |
| 7: <b>end while</b>                                | 7: 0     |                          |
| 8: $A[i + 1] \leftarrow clef$                      | 8: $c_8$ | $n - 1$                  |
| 9: <b>end for</b>                                  | 9: 0     |                          |

$$T(n) = c_1 n + (c_2 + c_3 + c_8)(n - 1) + c_4 \sum_{j=2}^n t_j + (c_5 + c_6) \sum_{j=2}^n (t_j - 1) \quad (1)$$

## Influence de la nature de données sur le temps d'exécution

On différencie 3 cas possibles

- le meilleur des cas (**best case**).

C'est celui où le tableau est déjà trié. Dans ce cas  $t_j = 1$  pour  $j = 2, 3, \dots, n$  et

$$T(n) = c_1 n + (c_2 + c_3 + c_4 + c_8)(n - 1) = an + b \quad \text{(fonction linéaire)} \quad (2)$$

- le pire des cas (**worst case**) ; C'est celui où le tableau est trié dans l'ordre décroissant. Dans ce cas  $t_j = j$  pour  $j = 2, 3, \dots, n$  et on obtient :

$$\begin{aligned} T(n) &= c_1 n + (c_2 + c_3 + c_8)(n - 1) + c_4 \sum_{j=2}^n t_j + (c_5 + c_6) \sum_{j=2}^n (t_j - 1) \\ &= (c_1 + c_2 + c_3 + c_8)n - (c_2 + c_3 + c_8) + c_4 \left( \frac{n(n+1)}{2} - 1 \right) \end{aligned} \quad (3)$$

$$\begin{aligned} &+ (c_5 + c_6) \frac{n(n-1)}{2} \\ &= an^2 + bn + c \quad \text{(fonction quadratique)} \end{aligned} \quad (4)$$

- le cas moyen (**average case**). Dans ce cas en moyenne  $t_j = j/2$  et on obtient aussi une **fonction quadratique**.

On a fait les hypothèses simplificatrices suivantes :

- on a ignoré le coût réel de chaque ligne en considérant un coût abstrait  $c_i$  ;
- même ce coût abstrait était après simplifié pour donner le temps d'exécution comme une fonction linéaire/quadratique ;
- dernière simplification : on ne considère que le 1<sup>er</sup> terme des polynômes. On s'intéresse donc au comportement asymptotique des algorithmes (pour  $n$  suffisamment grand).

**Remarques :** Pour chaque algorithme on peut mettre en évidence une ou plusieurs *opérations* qui sont *fondamentales* au sens où le temps d'exécution est proportionnel au nombre de ces opérations. Il est alors possible de comparer les algorithmes selon cette mesure simplifiée.

Exemples pour opérations fondamentales : comparaisons, additions, multiplications, lectures ; accès à la mémoire secondaire, etc.

## Ordre de grandeur des fonctions : définitions principales

- **Notion O** Soient deux fonctions  $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ . On dit que  $f(n) = O(g(n))$  ssi  $\exists c > 0, \exists n_0$  tq  $\forall n > n_0, f(n) \leq cg(n)$ .  
Prononcer :  $f$  est en grand O de  $g$  ( $f$  égale grand O de  $g$ ;  $f$  est d'ordre au plus de  $g$ )
- **Notion  $\Omega$**  Soient  $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ . On dit que  $f(n) = \Omega(g(n))$  ssi  $\exists c > 0, \exists n_0$  tq  $\forall n > n_0, cg(n) \leq f(n)$ .  
Prononcer :  $f$  est en grand  $\Omega$  de  $g$  ( $f$  égale grand  $\Omega$  de  $g$ ;  $f$  est d'ordre au moins de  $g$ )
- **Notion  $\Theta$**  Soient  $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ . On dit que  $f(n) = \Theta(g(n))$  ssi  $\exists c_1 > 0, \exists c_2 > 0, \exists n_0$  tq  $\forall n > n_0, c_1 g(n) \leq f(n) \leq c_2 g(n)$ .  
Prononcer :  $f$  est en grand  $\Theta$  de  $g$  ( $f$  égale grand  $\Theta$  de  $g$ ;  $f$  est de même ordre que  $g$ )
- **Notion o** Soient  $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ . On dit que  $f(n) = o(g(n))$  ssi  $\forall c > 0, \exists n_0$  tq  $\forall n > n_0, 0 \leq f(n) < cg(n)$ . On dit que  $f$  est négligeable par rapport à  $g$ .
- **Notion  $\omega$**  Soient  $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ . On dit que  $f(n) = \omega(g(n))$  ssi  $\forall c > 0, \exists n_0$  tq  $\forall n > n_0, cg(n) < f(n)$ . On dit que  $g$  est une borne inférieure stricte de  $f$ .

# Propriétés principales liées à l'ordre de grandeur des fonctions

## ■ Transitivité

- $f(n) = \Theta(g(n))$  et  $g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$
- $f(n) = O(g(n))$  et  $g(n) = O(h(n)) \implies f(n) = O(h(n))$
- $f(n) = \Omega(g(n))$  et  $g(n) = \Omega(h(n)) \implies f(n) = \Omega(h(n))$
- $f(n) = o(g(n))$  et  $g(n) = o(h(n)) \implies f(n) = o(h(n))$
- $f(n) = \omega(g(n))$  et  $g(n) = \omega(h(n)) \implies f(n) = \omega(h(n))$

## ■ Reflexivité

- $f(n) = O(f(n)), f(n) = \Theta(f(n)), f(n) = \Omega(f(n))$

## ■ Symétrie

- $f(n) = \Theta(g(n))$  ssi  $g(n) = \Theta(f(n))$

## ■ Symétrie transposée

- $f(n) = O(g(n))$  ssi  $g(n) = \Omega(f(n))$
- $f(n) = o(g(n))$  ssi  $g(n) = \omega(f(n))$

## ■ Analogie entre le comportement asymptotique des fonctions $f$ et $g$ et la comparaison de 2 nombres réels $a$ et $b$ .

- $f(n) = O(g(n)) \simeq a \leq b$
- $f(n) = \Omega(g(n)) \simeq a \geq b$
- $f(n) = \Theta(g(n)) \simeq a = b$  : On dit que  $f$  et  $g$  sont asymptotiquement équivalents
- $f(n) = o(g(n)) \simeq a < b$  : On dit que  $f$  est négligeable devant  $g$
- $f(n) = \omega(g(n)) \simeq a > b$



# Propriétés principales liées à l'ordre de grandeur des fonctions

## ■ Classement de quelques fonctions

$$\Theta(1) < \Theta(\log n) < \Theta(\sqrt{n}) < \Theta(n) < \Theta(n \log n) < \Theta(n^a) < \Theta(c^n) < \Theta(n!) < \Theta(n^n)$$

où  $a > 1$  et  $c > 1$ .

*Introduction à l'analyse des algorithmes*

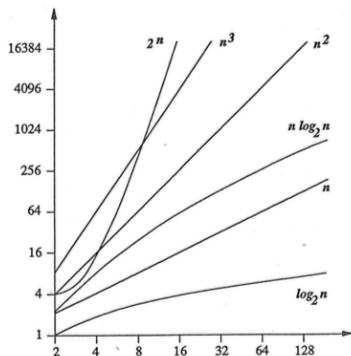


Figure 1. Rapidités de croissance comparées de certaines fonctions usuelles.

## Critères pour classer les fonctions : Soit $f, g, : \mathbb{N} \rightarrow \mathbb{R}^+$ .

### Lemme 1

- a)  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = a \neq 0 \Rightarrow f = \Theta(g) \text{ et } g = \Theta(f)$
- b)  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Rightarrow f = o(g) \Rightarrow f = O(g) \text{ et } f \neq \Theta(g)$
- c)  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \Rightarrow g = o(f) \Rightarrow g = O(f) \text{ et } g \neq \Theta(f)$

**Remarque :** Le Lemme 1 est un critère suffisant. Les réciproques de a), b), c) sont fausses. Si la limite  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  n'existe pas, il faut revenir aux définitions.

**Exemple 1 :** Soit  $f(n) = \begin{cases} n & \text{si } n \text{ est impair} \\ 2n & \text{si } n \text{ est pair} \end{cases}$  et  $g(n) = 2n, \forall n$

alors  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \nexists$ , mais  $f = \Theta(g)$  puisque  $\frac{g(n)}{2} \leq f(n) \leq g(n)$ .

**Lemme 2 :** Si  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$  n'est pas déterminée ( $\frac{0}{0}$  ou  $\frac{\infty}{\infty}$ ) on peut appliquer la règle de

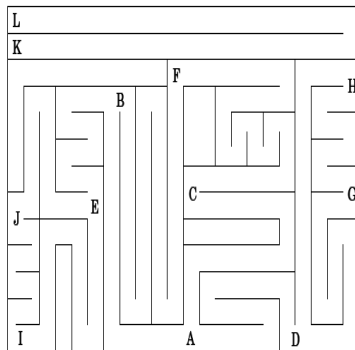
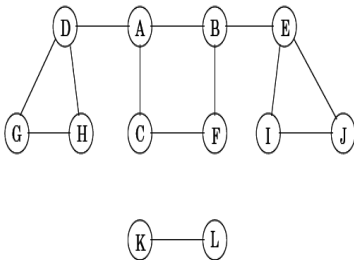
l'Hôpital :  $\lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)} \exists$  on a  $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{f'(n)}{g'(n)}$ .

**Exemple 2 :**  $n^2 = o(e^n)$  car  $\lim_{n \rightarrow \infty} \frac{n^2}{e^n} = \lim_{n \rightarrow \infty} \frac{2n}{e^n} = \lim_{n \rightarrow \infty} \frac{2}{e^n} = 0$ .

Exploration des graphes :  
Parcours en profondeur et parcours en largeur

# Exploration des graphes

- **L'exploration d'un graphe** (c.-à-d. la visite de tous les sommets joignables à partir d'un sommet de départ) est une opération fréquente et importante.
- elle ressemble à l'exploration d'un labyrinthe
- pour explorer un labyrinthe il faut :
  - une craie : pour noter les carrefours/couloirs déjà visités
  - un fil : pour pouvoir retourner sur ses pas (backtrack)



# Les structures Pile (Stack) et File (Queue)

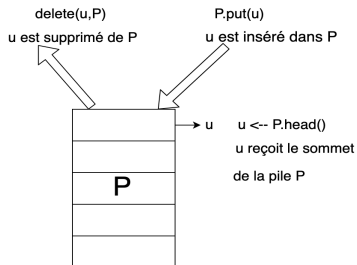


Figure 12 – Le fonctionnement LIFO (Last In First Out) : Une pile P et ses opérateurs

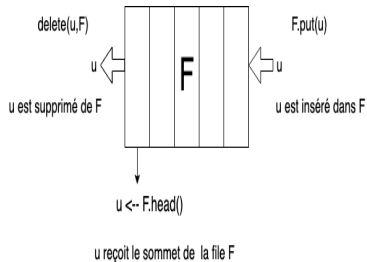


Figure 13 – Le fonctionnement FIFO (First In First Out) : Une file F et ses opérateurs

## Parcours en profondeur (Depth-first Search (DFS)) : 1<sup>e</sup> version itérative

---

**Algorithm 2** DFS( $G,s$ ) (utilise une **pile**  $P$  et suit la stratégie **LIFO** (Last In First Out)).

---

**Require:**  $G=(V,E)$ , start vertex  $s$ .

**Ensure:**  $v.visited = \text{true}$  for all vertices  $v \in V$  reachable from  $s$ .

```
1: Initialisation :  $\forall v \in V : v.visited \leftarrow \text{false}$ ;  $\text{clock} \leftarrow 1$ ;  $\text{previsit}(s)$ ;  $P \leftarrow [s]$ ;  
2:  $u \leftarrow P.\text{head}()$  (  $u$  reçoit le sommet de la pile  $P$  )  
3: while ( $u \neq \text{nil}$ ) do  
4:   if ( $\exists (u,v) \in E \mid v.visited = \text{false}$ ) then  
5:     {  $\text{previsit}(v)$ ;  $P.\text{put}(v)$  } {  $v$  est inséré dans  $P$  }  
6:   else  
7:     {  $\text{postvisit}(u)$ ;  $\text{delete}(u,P)$  } { le sommet  $u$  est supprimé de  $P$  }  
8:   end if  
9:    $u \leftarrow P.\text{head}()$  {  $u$  reçoit le sommet de la pile  $P$  }  
10: end while
```

---

where :

- $\text{previsit}(v) = \{v.visited \leftarrow \text{true}; v.in \leftarrow \text{clock}; \text{clock} \leftarrow \text{clock} + 1\}$
- $\text{postvisit}(v) = \{v.out \leftarrow \text{clock}; \text{clock} \leftarrow \text{clock} + 1\}$

## Analyse de l'algorithme (2)

L'algorithme (2) est appliqué sur le graphe  $G$  de la Fig. (15) à partir du sommet  $S$ . Les valeurs  $v.in$  ( $v.out$ ) seront indiquées en haut à gauche(droite) du sommet  $v$  (une fois calculées).

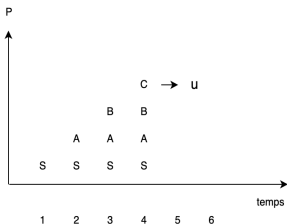


Figure 14 – L'état de la pile  $P$  après les premiers quatre coups d'horloge (clock). La pile contient les quatre sommets  $S, A, B, C$  qui sont empilés l'un après l'autre sur la pile. La tête de la pile correspond au sommet de la pile ( $C$  à l'occurrence).

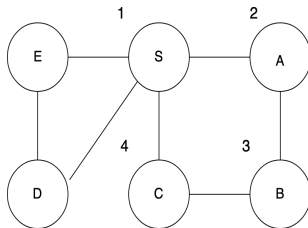
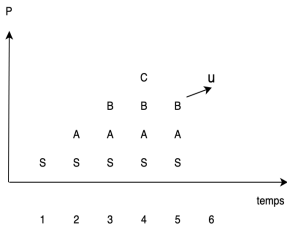


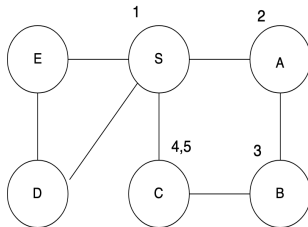
Figure 15 – Le graphe  $G$  après les premiers quatre coups d'horloge. Les valeurs  $v.in$  correspondent au moment de la visite/découverte du sommet  $v$ .

## Analyse de l'algorithme (2) : suite

L'algorithme (2) est appliqué sur le graphe  $G$  de la Fig. (17) à partir du sommet  $S$ .



**Figure 16** – L'état de la pile  $P$  après les premiers cinq coups d'horloge (clock). La pile contient les trois sommets  $S, A, B$ . Le sommet  $B$  est actuellement la tête de la pile.

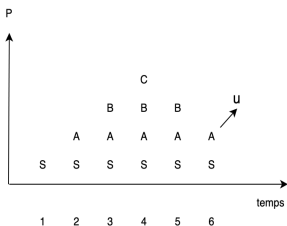


**Figure 17** – Le graphe  $G$  après les premiers cinq coups d'horloge.  $C.out = 5$  : c'est le moment quand le sommet  $C$  a été dépilé de la pile.

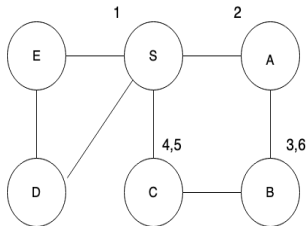


## Analyse de l'algorithme (2) : suite

L'algorithme (2) est appliqué sur le graphe  $G$  de la Fig. (19) à partir du sommet  $S$ .



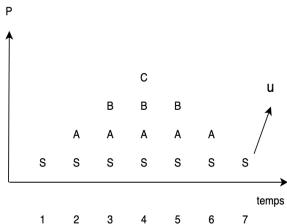
**Figure 18** – L'état de la pile  $P$  après les premiers six coups d'horloge. La pile contient les deux sommets  $S, A$ . Le sommet  $A$  est actuellement la tête de la pile.



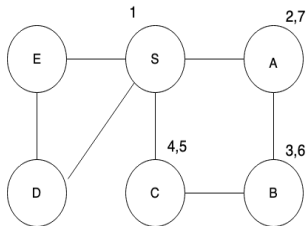
**Figure 19** – Le graphe  $G$  après les premiers six coups d'horloge. Les valeurs  $v.in$  correspondent au moment de la visite/découverte du sommet  $v$ . Les valeurs  $v.out$  indiquent le moment de l'enlèvement du sommet  $v$  de la pile.

## Analyse de l'algorithme (2) : suite

L'algorithme (2) est appliqué sur le graphe  $G$  de la Fig. (21) à partir du sommet  $S$  .



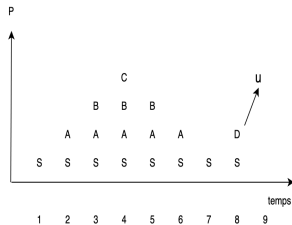
**Figure 20** – L'état de la pile  $P$  après sept coups d'horloge (clock). La pile ne contient que le sommet  $S$  qui est la tête de la pile.



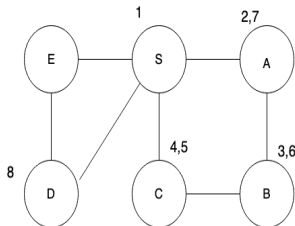
**Figure 21** – Le graphe  $G$  après les sept coups d'horloge. Les valeurs  $v.in$  correspondent au moment de la visite/découverte du sommet  $v$ . Les valeurs  $v.out$  indiquent le moment de l'enlèvement du sommet  $v$  de la pile.

## Analyse de l'algorithme (2) : suite

L'algorithme (2) est appliqué sur le graphe  $G$  de la Fig. (23) à partir du sommet  $S$ .



**Figure 22** – L'état de la pile  $P$  après les premiers huit coups d'horloge. La pile ne contient que les sommets  $S, D$ . Le sommet  $D$  est la tête de la pile.



**Figure 23** – Le graphe  $G$  après les premiers huit coups d'horloge.

## Analyse de l'algorithme (2) : suite

L'algorithme (2) est appliqué sur le graphe  $G$  de la Fig. (25) à partir du sommet  $S$  .

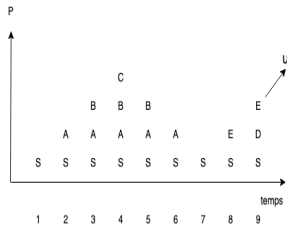


Figure 24 – L'état de la pile  $P$  après les premiers neuf coups d'horloge. La pile contient les sommets  $S, D, E$ . Le sommet  $E$  est la tête de la pile.

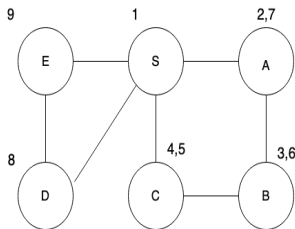


Figure 25 – Le graphe  $G$  après les premiers neuf coups d'horloge. Tous les sommets ont été visités. Les valeurs  $v.in$  correspondent au moment de la visite/découverte du sommet  $v$ .

## Analyse de l'algorithme (2) : suite

L'algorithme (2) est appliqué sur le graphe  $G$  de la Fig. (26) à partir du sommet  $S$ .

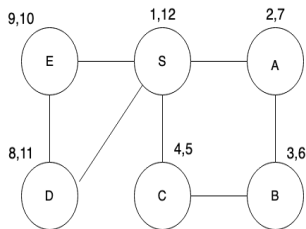


Figure 26 – Tous les sommets ont été visités. La pile  $P$  est vide.

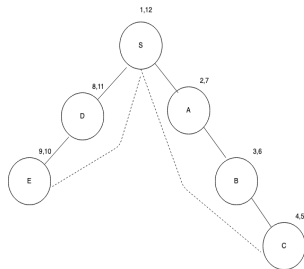


Figure 27 – L'arbre de l'appel  $DFS(G, S)$ . Le graphe  $G$  a été parcouru en profondeur d'abord. Les lignes pointillées indiquent des tests/vérifications effectués dans le code.

## Parcours en largeur d'abord (Breadth First Search (BFS))

---

**Algorithm 3** BFS( $G,s$ ) (utilise une **file**  $F$  et suit la stratégie **FIFO** (First In First Out) ).

---

**Require:**  $G=(V,E)$ , start vertex  $s$ .

**Ensure:** Breadth First Search (BFS) of the graph  $G$

```
1: Initialisation :  $\forall v \in V : v.visited \leftarrow \text{false}$  ;  $\text{clock} \leftarrow 1$  ;  $\text{previsit}(s)$  ;  $F \leftarrow [s]$  ;  
2:  $u \leftarrow P.\text{head}()$  ( $u$  reçoit le sommet de la file  $F$ )  
3: while ( $u \neq \text{nil}$ ) do  
4:   if ( $\exists (u, v) \in E \mid v.visited = \text{false}$ ) then  
5:     {  $\text{previsit}(v)$  ;  $F.\text{put}(v)$  } {  $v$  est inséré dans  $F$  }  
6:   else  
7:     {  $\text{postvisit}(u)$  ;  $\text{delete}(u, F)$  } { le sommet  $u$  est supprimé de  $F$  }  
8:   end if  
9:    $u \leftarrow F.\text{head}()$  {  $u$  reçoit le sommet de la file  $F$  }  
10: end while
```

---

where :

- $\text{previsit}(v) = \{ v.visited \leftarrow \text{true} ; v.in \leftarrow \text{clock} ; \text{clock} \leftarrow \text{clock} + 1 \}$
- $\text{postvisit}(v) = \{ v.out \leftarrow \text{clock} ; \text{clock} \leftarrow \text{clock} + 1 \}$

## Analyse de l'algorithme BFS(G,S)

L'algorithme (3) est appliqué sur le graphe  $G$  de la Fig. (29) à partir du sommet  $S$ . Les valeurs  $v.in$  ( $v.out$ ) seront indiquées en haut à gauche(droite) du sommet  $v$  (dès qu'elles soient calculées).

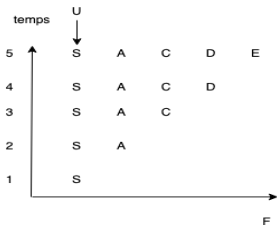


Figure 28 – L'état de la file  $F$  après les premiers cinq coups d'horloge. La file contient le sommet  $S$  et les quatre sommets qui lui sont adjacents  $A, C, D, E$ . La tête de la file n'a pas bougé et pointe sur le sommet  $S$ .

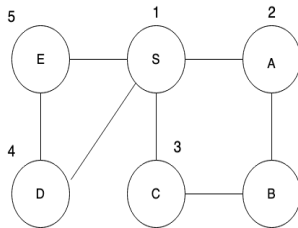


Figure 29 – Le graphe  $G$  après les premiers cinq coups d'horloge. Les valeurs  $v.in$  correspondent au moment de la visite/découverte du sommet  $v$ .

## Analyse de l'algorithme BFS(G,S) : suite

L'algorithme (3) est appliqué sur le graphe  $G$  de la Fig. (31) à partir du sommet  $S$ .

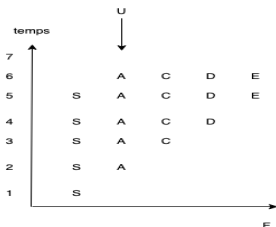


Figure 30 – L'état de la file  $F$  après les premiers six coups d'horloge. Le sommet  $S$  a été enlevé et la tête de la file pointe sur le sommet  $A$ .

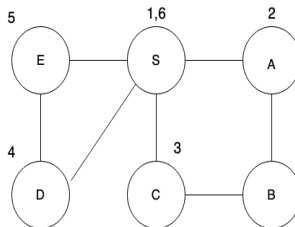
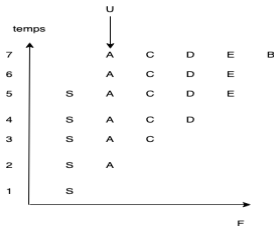


Figure 31 – Le graphe  $G$  après les premiers six coups d'horloge. Les valeurs  $v.in$  correspondent au moment de la visite/découverte du sommet  $v$ . Les valeurs  $v.out$  indiquent le moment de l'enlèvement du sommet  $v$  de la file.

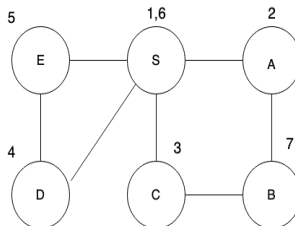


## Analyse de l'algorithme BFS(G,S) : suite

L'algorithme (3) est appliqué sur le graphe  $G$  de la Fig. (33) à partir du sommet  $S$ .



**Figure 32** – L'état de la file  $F$  après les premiers sept coups d'horloge. La tête de la file pointe sur le sommet  $A$ . Le sommet  $B$  qui lui est adjacent est ajouté(enfilé) dans la file  $F$ .



**Figure 33** – Le graphe  $G$  après les premiers sept coups d'horloge. Les valeurs  $v.in$  correspondent au moment de la visite/découverte du sommet  $v$ . Les valeurs  $v.out$  indiquent le moment de l'enlèvement du sommet  $v$  de la file.

## Analyse de l'algorithme BFS(G,S) : suite

L'algorithme (3) est appliqué sur le graphe  $G$  de la Fig. (35) à partir du sommet  $S$ .

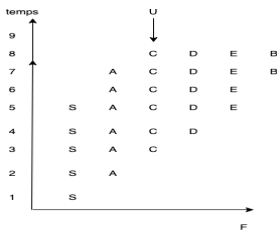


Figure 34 – L'état de la file  $F$  après les huit coups d'horloge. Après l'enlèvement de  $A$ , la tête de la file pointe sur le sommet  $C$ .

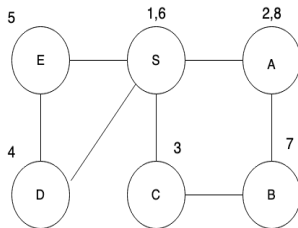


Figure 35 – Le graphe  $G$  après les huit coups d'horloge. Les valeurs  $v.in$  correspondent au moment de la visite/découverte du sommet  $v$ . Les valeurs  $v.out$  indiquent le moment de l'enlèvement du sommet  $v$  de la file.

## Analyse de l'algorithme $BFS(G,S)$ : fin

L'algorithme (2) est appliqué sur le graphe  $G$  de la Fig. (36) à partir du sommet  $S$ .

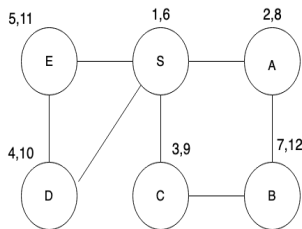


Figure 36 – Tous les sommets ont été visités. La file  $F$  est vide.

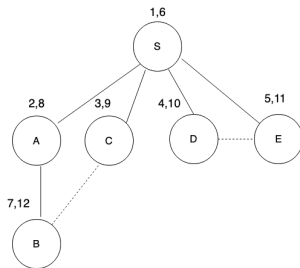


Figure 37 – L'arbre de l'appel  $BFS(G, S)$ . Le graphe  $G$  a été parcouru en largeur d'abord. Les lignes pointillées indiquent des tests/vérifications effectués dans le code.

## Parcours en profondeur d'abord : version récursive

---

**Algorithm 4** Explore( $G,v$ ).

---

**Require:**  $G=(V,E)$ , start vertex  $v \in V$ .

**Ensure:**  $u.visited = \text{true}$  for all vertices  $u$  reachable from  $v$ .

```
1:  $v.visited \leftarrow \text{true}$ 
2:  $previsit(v)$ 
3: for all each edge  $(v,u) \in E$  do
4:   if not  $u.visited$  then Explore( $G,u$ )
5: end for
6:  $postvisit(v)$ 
```

---

- Les procédures «  $previsit(v)$  » et «  $postvisit(v)$  » contiennent des opérations optionnelles à effectuer lors de la première visite du sommet  $v$ , et après l'avoir complètement explorer. Ces procédures peuvent être extrêmement utiles.

## Explore( $G,v$ ) : validation formelle

- **Proposition** : Explore( $G,v$ ) visite chaque sommet joignable à partir de  $v$ .
- *Preuve* : Supposons que le sommet  $u$  est joignable à partir de  $v$ , mais n'a pas été visité par Explore( $G,v$ ). Notons  $P$  le chemin allant de  $v$  à  $u$ . Considérons le dernier sommet  $z$  visité par Explore( $G,v$ ) sur ce chemin et soit  $w$  le sommet qui suit immédiatement  $z$  sur  $P$ .



- D'après la définition d'Explore( $G,v$ ),  $w$  devrait être aussi visité ; contradiction.

## Parcours en profondeur d'abord : l'algorithme complet

- Puisque le graphe  $G$  peut avoir plusieurs composantes distinctes, plusieurs appels d'Explore( $G, \cdot$ ) à partir de différents sommets de départ sont éventuellement nécessaires.

---

### Algorithm 5 DFS( $G$ ) : algorithme **depth-first search**.

---

**Require:**  $G=(V,E)$ .

**Ensure:**  $v.\text{visited} = \text{true}$  for all vertices  $v \in V$ .

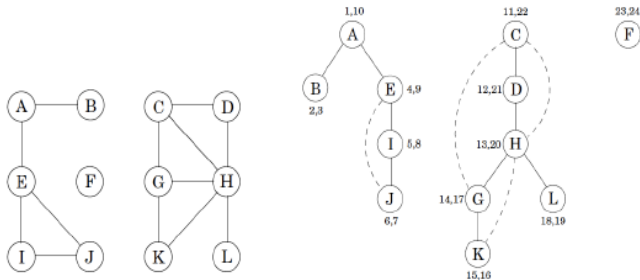
```
1: for all  $v \in V$  do  
2:    $v.\text{visited} \leftarrow \text{false}$   
3: end for  
4: for all  $v \in V$  do  
5:   if not  $v.\text{visited}$  then Explore( $G, v$ )  
6: end for
```

---

- Analyse de complexité :
  - $\forall v \in V$  Explore( $G, v$ ) est appelée une seule fois (grâce à l'étiquette  $v.\text{visited}$ ).
  - Chaque arête  $(u, v) \in E$  est examinée exactement deux fois : la première fois lors d'Explore( $G, u$ ) et une deuxième fois lors d'Explore( $G, v$ ).
  - Sous l'hypothèse que previsit et postvisit prennent un temps constant, DFS( $G$ ) nécessite un temps  $\Theta(|V| + |E|)$  (linéaire).

## Forêt couvrante associée à un parcours en profondeur d'abord

- En général,  $\text{DFS}(G)$  engendre une forêt qui contient plusieurs arbres de parcours, une pour chaque composante connexe du graphe.



- Lors du parcours, chaque sommet  $v$  est muni de deux étiquettes ( $v.in$ ,  $v.out$ ). On utilise aussi les notations ( $pre(v)$ ,  $post(v)$ )<sup>1</sup>.  $v.in$  indique le moment de la première visite du sommet  $v$ .  $v.out$  indique le moment quand la procédure DFS a définitivement quitté le sommet  $v$  (il a été donc complètement exploré).

1. Dans la terminologie de ce cours la numérotation ( $pre$ ,  $post$ ) est une autre dénomination de la numérotation ( $in$ ,  $out$ ) (c.-à-d.  $pre(u) = u.in$  et  $post(u) = u.out$ )



## Calcul la numérotation (in, out) ((pre, post))

- Les valeurs in/out peuvent être calculées dans les procédures previsit et postvisit.
- On utilise pour ce faire le compteur « clock » initialisé à 1 et défini comme suit.
  - $\text{previsit}(v) = \{v.in \leftarrow \text{clock}; \text{clock} \leftarrow \text{clock} + 1\}$
  - $\text{postvisit}(v) = \{v.out \leftarrow \text{clock}; \text{clock} \leftarrow \text{clock} + 1\}$
- **Proposition** : Pour chaque couple de sommet  $u$  et  $v$ , les intervalles  $[u.in, u.out]$  et  $[v.in, v.out]$  sont soit complètement disjoints, soit l'un est entièrement inclus dans l'autre.
- *Preuve* : L'intervalle  $[u.in, u.out]$  indique le temps durant lequel le sommet  $u$  est présent dans la pile. La stratégie LIFO explique alors la propriété ci-dessus.

# Parcours en profondeur dans les graphes orientés

- $\text{DFS}(G)$  s'applique d'une façon similaire dans les graphes orientés. En raison de l'orientation des arcs, les arbres de parcours engendrés sont plus complexes (appelés aussi arborescences).

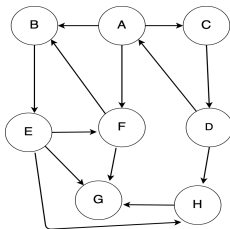


Figure 38 – Un graphe  $G$ .

A FAIRE A LA MAISON

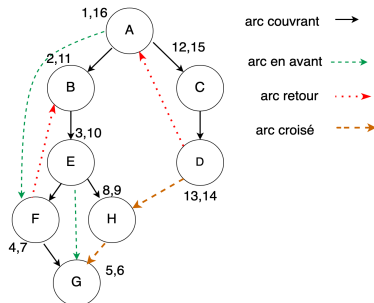
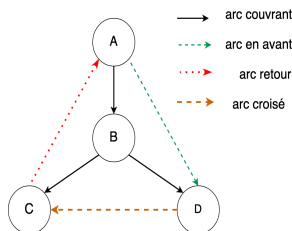


Figure 39 – L'arborescence de l'appel  $\text{DFS}(G, A)$ .

## Parcours en profondeur dans les graphes orientés : classification des arcs



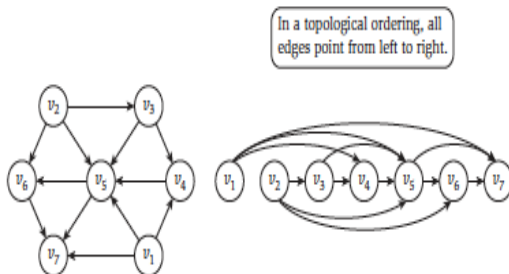
- Les arcs  $u \rightarrow v$  tels que  $\text{Explore}(G, u)$  appelle  $\text{Explore}(G, v)$  sont nommés **arcs couvrants** (« *tree edges* »). Les arcs couvrants constituent une **forêt couvrante d'arborescences de parcours en profondeur**.
- Les arcs en pointillé peuvent être de trois types différents.
  - Les **arcs en avant** (« *forward edges* ») sont les arcs  $(u, v)$  qui relient un sommet  $u$  à un descendant  $v$ .
  - Les **arcs retour** (« *back edges* ») sont les arcs  $(u, v)$  reliant un sommet  $u$  à un ancêtre  $v$ . Les boucles sont considérées comme des arcs retour.
  - Les **arcs croisés** (« *cross edges* ») sont tous les autres arcs. Ce sont des arcs pour lesquels il n'existe pas de chemin entre leurs extrémités dans la forêt. Ils peuvent relier deux sommets d'une même arborescence, tant que l'un des sommets n'est pas ancêtre de l'autre ; ils peuvent aussi relier deux sommets appartenant à des arborescences différentes.

## Relations entre le type d'arc et la numérotation [pre, post] ([in, out])

- Dans une forêt DFS, le sommet  $u$  est un ancêtre du sommet  $v$  ssi  $u$  est visité le premier et  $v$  est visité lors de l'appel  $\text{explore}(u)$  (c.-à-d.  $\text{pre}(u) < \text{pre}(v) < \text{post}(v) < \text{post}(u)$ ).
- Dans une forêt DFS, le sommet  $u$  est un descendant du sommet  $v$  ssi  $v$  est visité le premier et  $u$  est visité lors de l'appel  $\text{explore}(v)$  (c.-à-d.  $\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$ ).
- On peut alors donner la classification suivante pour l'arc  $(u,v)$  :
  - $\text{pre}(u) < \text{pre}(v) < \text{post}(v) < \text{post}(u)$  ssi l'arc  $(u,v)$  est de type arc couvrant/arc en avant.
  - $\text{pre}(v) < \text{pre}(u) < \text{post}(u) < \text{post}(v)$  ssi l'arc  $(u,v)$  est de type arc retour.
  - les intervalles  $[\text{pre}(v), \text{post}(v)]$  et  $[\text{pre}(u), \text{post}(u)]$  sont disjoints ssi l'arc  $(u,v)$  est de type arc croisé.

## Application de DFS : tri topologique (linéaire)

Le parcours en profondeur peut être utilisé pour effectuer un tri topologique (ou tri linéaire) d'un graphe orienté sans circuit. Le tri topologique d'un DAG (graphe orienté acyclique)  $G = (V, E)$  consiste à ordonner linéairement tous ses sommets de telle sorte que si  $G$  contient l'arc  $(u, v)$  alors,  $u$  apparaisse avant  $v$ .



**Figure 40 – Gauche :** Un graphe  $G = (V, E)$ . Les sommets représentent un ensemble de tâches, les arcs visualisent leurs relations d'antériorités. Il n'est pas évident de trouver l'ordre d'exécution de ces tâches ? **Droite :** le même graphe est trié topologiquement (linéarisé) par l'ordonnancement des sommets  $v \in V$ . Ainsi, un ordre d'exécution des tâches est explicité.

## Application de DFS : linéarisation d'un DAG (Directed Acyclic Graph)

Les propositions suivantes sont utilisées pour effectuer un tri topologique.

- **Proposition** : Un graphe orienté  $G$  contient un circuit, ssi la forêt engendrée par le parcours DFS( $G$ ) contient un arc retour.
- *Preuve* :
  - Evident si le parcours DFS( $G$ ) contient un arc retour.
  - Supposons maintenant que  $G$  contient un circuit  $C : v_0 \rightarrow v_1 \rightarrow v_2 \rightarrow \dots \rightarrow v_k \rightarrow v_0$ . Soit  $v_i$  le premier sommet de  $C$  qui a été visité durant le parcours DFS. Tous les autres sommets de  $C$  sont des descendants de  $v_i$  dans l'arbre de parcours et alors  $v_{i-1} \rightarrow v_i$  est un arc retour (si  $i = 0$  alors  $v_k \rightarrow v_0$ ).

- **Proposition** : La numérotation post d'un DAG obtenue lors d'un parcours DFS satisfait l'inégalité  $post(u) > post(v)$  pour chaque arc  $(u, v)$ .
- *Preuve* : Les seuls arcs  $(u, v)$  pour lesquels  $post(u) < post(v)$  dans l'arbre DFS sont les arcs retour. Or, il n'y pas de tels arcs dans un DAG.

---

**Algorithm 6** Premier algorithme pour linéariser un DAG.

---

**Require:**  $G=(V,E)$ , DAG

**Ensure:** List vertices in linear order.

- 1: Perform a DFS search and obtain the number  $post(v)$  for any vertex  $v \in V$ .
  - 2: List vertices in order of decreasing  $post(v)$  values.
-

## Linéarisation d'un DAG par l'algorithme 6

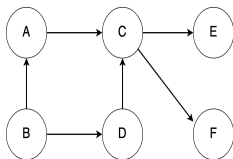


Figure 41 – Un graphe  $G = (V, E)$ .

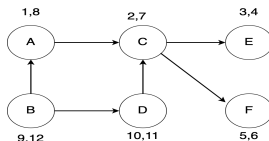


Figure 42 – La numérotation  $[pre, post]$  obtenue avec un parcours  $DFS(G, A)$ .

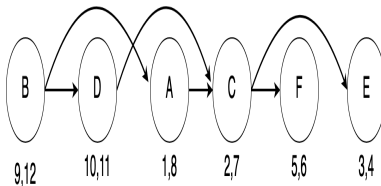


Figure 43 – Le graphe  $G$  est trié topologiquement (linéarisé) par l'ordonnancement des sommets  $v \in V$  dans l'ordre décroissant de leur numérotation  $post(v)$ .



## Composantes fortement connexes (CFC)

## Modéliser l'accessibilité au centre-ville de l'Utopia

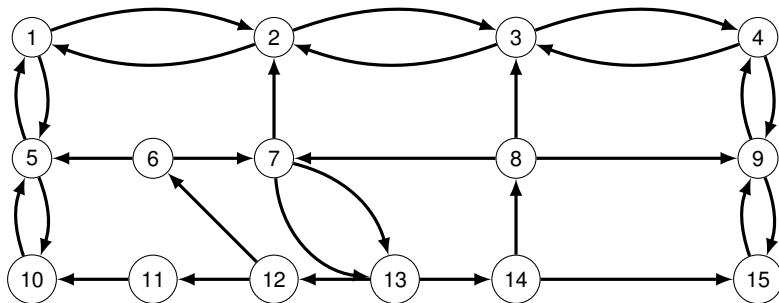
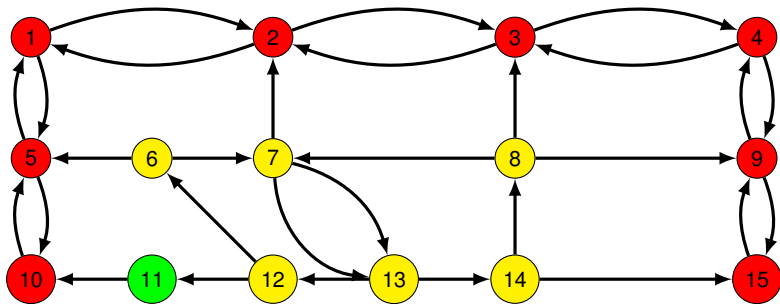


Figure 44 – Le plan de sens uniques du centre-ville.

Comment détecter si ce plan est acceptable ?

## Modéliser l'accessibilité au centre-ville de l'Utopia : solution



**Figure 45** – Les sommets jaunes et verts ne sont pas joignables à partir des sommets rouges. Les trois zones coloriées en rouge, jaune et vert sont des *Composantes Fortement Connexes (CFC)*

Le plan donc n'est pas acceptable puisque le graphe associé contient trois CFC.

## Application de DFS : Composantes fortement connexes (CFC)

- Soit le graphe orienté  $G = (V, E)$ .  $G$  est dit *fortement connexe*, si pour chaque couple de sommets  $(u, v)$ , il existe un chemin de  $u$  à  $v$ , noté  $(u \rightsquigarrow v)$ , et un chemin de  $v$  à  $u$ , noté  $(v \rightsquigarrow u)$ .
- Cette définition définit une relation binaire dans  $V$ .

$$uRv \equiv (u \in \Gamma_v^*) \wedge (v \in \Gamma_u^*) \quad (5)$$

où  $\Gamma_v^*$  indique la descendance du sommet  $v$ .

- C'est une relation d'équivalence dont les classes s'appellent composantes fortement connexes (c.f.c.).
- Elle partitionne  $V$  en ensembles disjoints (appelés C.F.C.).

## Application de DFS : Composantes fortement connexes (suite)

- La détection des c.f.c. se fait facilement grâce aux propriétés du parcours DFS.
- **Propriété 1** : La procédure  $\text{explore}(G,u)$  ne se termine que si tous les sommets accessibles à partir de  $u$  soient visités/explorés (c.-à-d. après avoir exploré toute la descendance  $\Gamma_u^*$  du sommet  $u$ ).
- Ainsi, si la procédure  $\text{explore}(G,v)$  est exécutée à partir d'un sommet  $v$  appartenant à une c.f.c. *puits* (une c.f.c. sans arcs sortants), alors elle visiterait exactement cette c.f.c. La dernière pourrait être alors enlevée du graphe  $G$ , et le processus continuerait sur le graphe résultant  $G'$ .
- Illustrer cette propriété avec les deux meta-sommets  $\{D\}$  et  $\{G,H,I,K,J,L\}$  du graphe de la figure (46).

# Application de DFS : Composantes fortement connexes (suite)

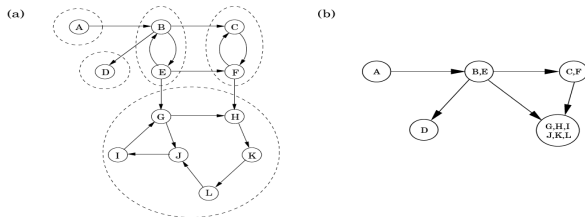


Figure 46 – a) Un graphe orienté  $G$  et ses C.F.C. b) Son graphe réduit  $\tilde{G}$

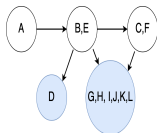


Figure 47 – Ce meta-graphe a deux sommets-puits (en gris). On peut les enlever.

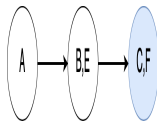


Figure 48 – On enlève l'unique sommet-puits.

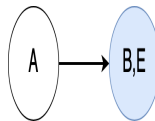


Figure 49 – On enlève l'unique sommet-puits.

La connaissance des sommets-puits permet de les enlever un par un et de découvrir ainsi toutes les C.F.C. du graphe. Pour se faire, on peut utiliser le deuxième algorithme de tri topologique.

## Application de DFS : Composantes fortement connexes (suite)

- Comment détecter une c.f.c. puits ?
- Trouver une c.f.c. puits est difficile, mais trouver une c.f.c. *source* (sans arcs entrants) est facile en raison de l'observation suivante.
- **Propriété 2** : Le sommet  $v$  avec la plus grande valeur  $post(v)$  calculée lors d'un parcours DFS se trouve dans une c.f.c. source.
- Cette propriété est déduite du fait suivant.
- **Propriété 3** : Soit deux c.f.c.  $C$  et  $C'$ , telles qu'il existe un arc d'un sommet de  $C$  vers un sommet de  $C'$ , alors :

$$\max_{u \in C} \{post(u)\} > \max_{u \in C'} \{post(u)\} \quad (6)$$

- *Preuve* : Deux cas sont à considérer.
  - DFS commence à partir d'un sommet  $u \in C$ . Alors DFS visite  $\Gamma_u^*$  (tous les sommets appartenants à  $C$  et  $C'$ ) et  $post(u) > post(v), \forall v \in C \cup C'$  (propriété 1).
  - DFS commence à partir d'un sommet  $u \in C'$ . Alors tous les sommets de  $C'$  sont visités avant de commencer la visite de  $C$ . Donc,  $\underset{u \in C}{post(u)} > \underset{u \in C'}{post(u)}$ .

## Application de DFS : Composantes fortement connexes (suite)

- Grace à la propriété 3, nous savons comment trouver une c.f.c. source dans  $G$ .
- Déterminer les sommets appartenant aux c.f.c. nécessite que le parcours commence à partir d'une c.f.c. puits. Comment trouver une telle c.f.c. ?
- Idée : Renverser le graphe (c.-à-d. changer l'orientation de tous les arcs). Le graphe ainsi obtenu sera noté  $G_R$ .
- Alors les c.f.c. ne changent pas (facile à vérifier). Mais les sommets puits se transforment en sources et vice versa. Le graphe  $\tilde{G}$  se renverse aussi (voir les graphes des figures (46) et (51)).
- La linéarisation du DAG  $\tilde{G}_R$  permet de trier topologiquement les sommets de DAG  $\tilde{G}$  dans l'ordre des sommets-uits vers les sommets-sources.



## Application de DFS : Composantes fortement connexes (suite)

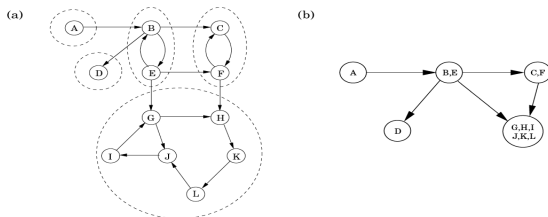


Figure 50 – a) Un graphe orienté  $G$  et ses c.f.c. b) Son graphe réduit  $\tilde{G}$

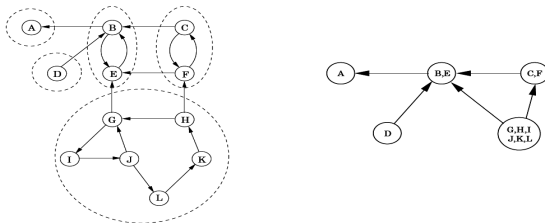


Figure 51 – a)  $G_R$  : le graphe renversé du graphe  $G$  et ses c.f.c. b)  $\tilde{G}_R$  : le graphe réduit de  $G_R$

---

**Algorithm 7** Déterminer les CFC d'un graphe orienté  $G$ .

---

**Require:**  $G=(V,E)$

**Ensure:** Detect the strongly connected components (SCC) of  $G$

- 1: Reverse all the edges in  $G$ , yielding digraph  $G_R$ .
  - 2: Run DFS on  $G_R$ , obtaining the  $post(v)$  numbers for all vertices  $v$ .
  - 3:  $k \leftarrow 1$ .
  - 4: Run EXPLORE( $G,v$ ) in  $G$  from the vertex  $v$  that has the highest  $post(v)$  value in  $G_R$ , and has not yet been assigned to any SCC. Assign all vertices mapped out by the exploration into SCC  $k$ .
  - 5: Set  $k \leftarrow k + 1$  and repeat from step 4, until all vertices have been assigned to SCCs.
- 

- Complexité : linéaire.
- Appliquer l'algorithme 7 pour trouver les C.F.C. du graphe de fig. (50)

## Application de l'algorithme de Kosaraju (algorithme 7) : suite

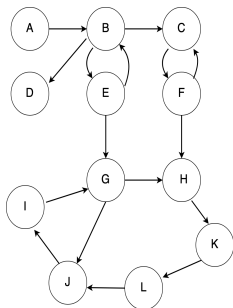


Figure 52 – Le graphe  $G = (V, E)$ .

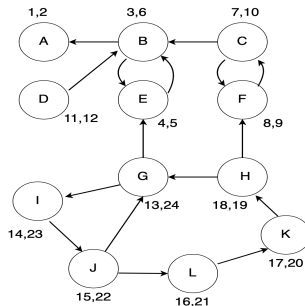


Figure 53 – Le graphe  $G_R = (V, E_R)$  avec la numérotation  $[pre, post]$  [in, out] obtenue avec un parcours  $DFS(G_R, A)$ .

## Application de l'algorithme de Kosaraju (algorithme 7) : suite et fin

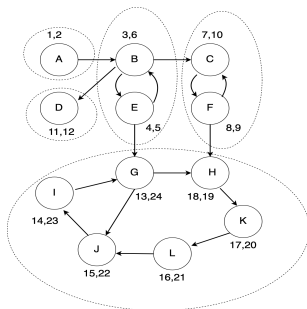


Figure 54 – Le graphe  $G = (V, E)$  avec la numérotation  $[pre, post]$  obtenue lors du parcours  $DFS(G_R, A)$ .

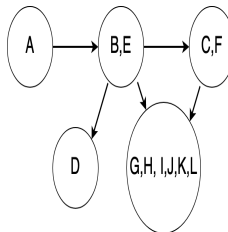


Figure 55 – Les cinq CFC découvertes par l'algorithme de Kosaraju et le meta-graphe associé (on dit aussi son graphe réduit  $\tilde{G}$ )

On applique  $EXPLORE(G, v)$  sur le graphe d'origine  $G = (V, E)$  dans l'ordre décroissant des valeurs  $post(v)$  qui ont été obtenues lors du parcours de du graphe renversé  $DFS(G_R, A)$ . Cela donne l'ordre suivant :  $EXPLORE(G, G)$ ,  $EXPLORE(G, D)$ ,  $EXPLORE(G, C)$ ,  $EXPLORE(G, B)$ ,  $EXPLORE(G, A)$ .

## Application de DFS : Composantes fortement connexes (suite)

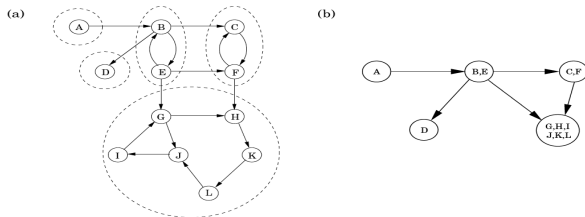


Figure 56 – a) Un graphe orienté  $G$  et ses c.f.c. b) Le graphe réduit  $\tilde{G}$

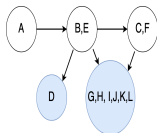


Figure 57 – Ce meta-graphe a deux sommets-puits (en gris). On peut les enlever.

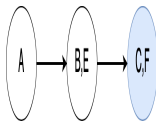


Figure 58 – On enlève l'unique sommet-puits.

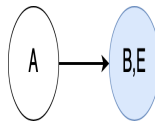


Figure 59 – On enlève l'unique sommet-puits.

La connaissance des sommets-puits permet de les enlever un par un et de découvrir ainsi toutes les c.f.c. du graphe. Pour se faire, on peut utiliser le deuxième algorithme de tri topologique (Alg 7).

- **Propriété** : Chaque DAG a au moins un sommet source (sommet sans prédécesseurs) et au moins un sommet puits (sommet sans successeurs)
- *Preuve* : Ce sont respectivement le sommet avec la plus grande, et la plus petite valeur de la numérotation *post*.

---

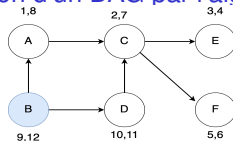
**Algorithm 8** Deuxième algorithme pour linéariser un DAG.

---

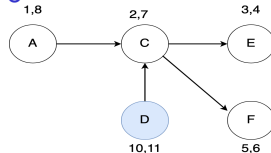
- 1: Find a source, output it, and delete it from the graph.
  - 2: Repeat until the graph is empty.
- 

- **Devoir** : Analyser et donner la complexité des algorithmes 6 et 8.

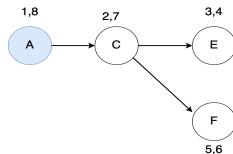
## Linéarisation d'un DAG par l'algorithme 8



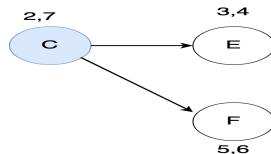
**Figure 60** – B est un sommet-source dans le graphe d'origine  $G = (V, E)$ . On l'enlève de  $G$ .



**Figure 61** – D est un sommet-source dans le graphe induit  $G_B = (V_B, E_B)$ . On l'enlève de  $G_B$ .



**Figure 62** – A est un sommet-source dans le graphe induit  $G_D = (V_D, E_D)$ . On l'enlève de  $G_B$ .



**Figure 63** – C est un sommet-source dans le graphe induit  $G_A = (V_C, E_C)$ . On l'enlève de  $G_A$ .

Les deux dernières tâches E et F peuvent être exécutées dans un ordre quelconque. L'ordre d'exécution des tâches est donné par l'ordre d'enlèvement des sommets-sources : B, D, A, C, E, F.

---

**Algorithm 9** `topological_sort_2( $G, d^-, Z$ )`

---

**Require:**  $G = (V, E)$  given by the list of successor  $\text{succ}$ ,  $d^-$  the list of input degree value,  $Z$  a LIFO/FIFO queue containing the indices of vertices  $v$  such that  $d^-[v] = 0$ , i.e.  $Z = \{v \in V \mid d^-[v] = 0\}$ .

**Ensure:** the list  $\text{top}$  is a topological sort of the graph  $G$

```
1:  $\text{top} \leftarrow []$ 
2: while  $Z \neq \emptyset$  do
3:    $u \leftarrow Z.\text{pop}()$ 
4:   for  $v \in \text{succ}[u]$  do
5:      $d^-[v] \leftarrow d^-[v] - 1$ 
6:     if  $d^-[v] = 0$  then
7:        $Z.\text{add}(v)$ 
8:     end if
9:   end for
10:   $\text{top}.\text{add}(u)$ 
11: end while
12: return  $\text{top}$ 
```

---

On peut démontrer que la complexité de cet algorithme est  $\Theta(|V| + |E|)$ .



# Le problème du flot maximum dans les réseaux : l'algorithme de Ford-Fulkerson

- Un *réseau* est un graphe connexe orienté (triplet)  $G = (V, U, C)$ , où  $V = \{v_1, v_2, \dots, v_n\}$  est l'ensemble de sommets/noeuds et  $U$  est l'ensemble d'arcs (couples  $(i, j) \in V \times V$ ). A  $\forall$  arc  $u \in U$  on affecte une valeur (*capacité*)  $c_u$  qui est une borne supérieure du flot sur l'arc.
- Soit deux sommets particuliers  $s \in V$  (source) et  $t \in V$  (puits). Considérons le graphe  $G^0 = (V, U^0)$  où  $U^0 = U \cup \{(t, s)\}$ . L'arc  $(t, s)$  est appelé l'*arc de retour* du flot (numéroté 0). Notons  $n = |V|$  et  $m = |U|$ .

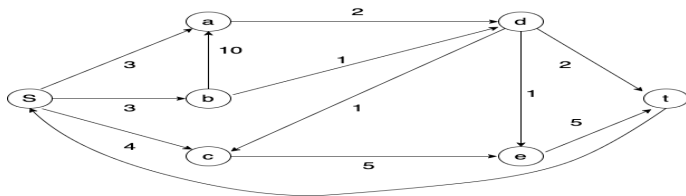


Figure 64 – Un réseau  $G = (V, U, C)$ . Chaque arc  $u \in U$  est muni d'une capacité  $c_u$ .

## Exemple de flot

- On dit que  $[\phi_1, \phi_2, \dots, \phi_m]^T$  est un *flot* de  $s$  à  $t$  ssi la loi de conservation du flot est vraie en tout  $\forall u \in V \setminus \{s, t\}$ , i.e.

$$\sum_{v \in \Gamma^+(u)} \phi_{(u,v)} = \sum_{v \in \Gamma^-(u)} \phi_{(v,u)} \quad (7)$$

- La valeur du flot est notée par  $\phi_0$ . Elle est définie par

$$\sum_{v \in \Gamma^+(s)} \phi_{(s,v)} = \sum_{u \in \Gamma^-(t)} \phi_{(u,t)} = \phi_0 \text{ (valeur du flot)} \quad (8)$$

- But :** Trouver dans  $G^0$  un *flot compatible*  $\phi' = [\phi_0, \phi_1, \phi_2, \dots, \phi_m]$  (c.-à-d.  $0 \leq \phi_u \leq c_u \forall u \in U$ ) et tel que la valeur  $\phi_0$  soit *maximale*.

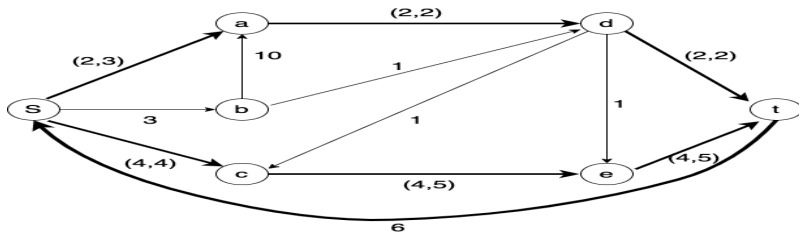


Figure 65 – Un flot de valeur 6. La première valeur dans un couple  $(\phi_u, c_u)$  sur l'arc  $u$  correspond au flot, tandis que la deuxième valeur indique sa capacité.

## Définition d'une coupe et de sa capacité

- Soit le réseau  $G = (V, U, C)$ . Une coupe séparant  $s$  et  $t$  (notée  $s - t$ ) est une partition  $(L, R)$  de  $V$ , telle que  $s \in L$  et  $t \in R = \bar{L} = V \setminus L$ .
- La capacité  $C(L, R)$  de la coupe  $s - t$  est définie par  $C(L, R) = \sum_{e \in \omega^+(L)} c_e$  où  $\omega^+(L)$  représente le sous-ensemble d'arcs qui ont leurs extrémités initiales dans  $L$  et leurs extrémités terminales dans  $\bar{L} = R$ .
- **Lemme** : La valeur maximale d'un flot de  $s$  à  $t$  compatible avec  $c_u$  n'excède jamais la capacité d'une coupe séparant  $s$  et  $t$ .

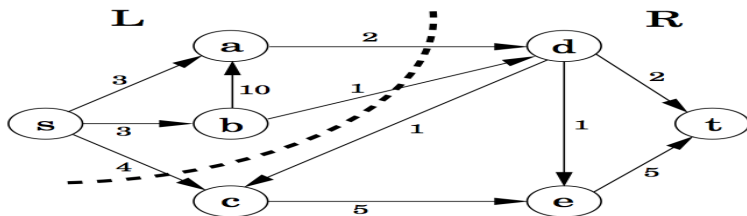


Figure 66 – Une coupe  $s - t$  définie par la partition  $L = \{s, a, b\}$  and  $R = \{c, d, e, t\}$ . Chaque flot  $s \rightsquigarrow t$  doit passer de  $L$  à  $T$  et traverser ainsi la coupe  $s - t$ . Donc, la valeur d'aucun flot ne peut dépasser la valeur de la capacité  $C(L, R)$  qui vaut 7. Ainsi, chaque flot  $s \rightsquigarrow t$  de valeur 7, devrait être maximum (**certificat d'optimalité**).

## L'égalité entre le flot et la capacité d'une coupe est un certificat d'optimalité

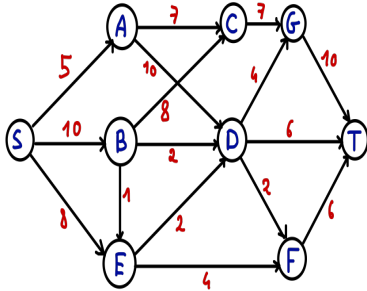


Figure 67 – Les valeurs en rouges sont les poids des arcs.

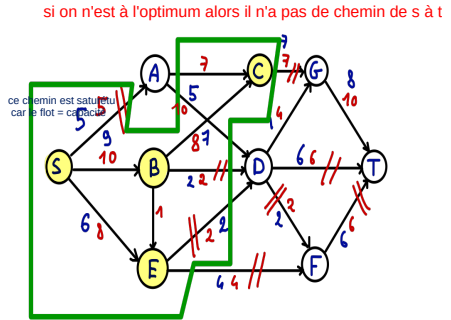


Figure 68 – Les valeurs en bleu visualisent le flot sur les arcs.

La valeur du flot sur le réseau ci-dessous vaut 20. La capacité de la coupe  $(\{S, B, E, C\}, \{A, G, D, F, T\})$  est donnée par la somme des capacités des arcs  $(S, A), (C, G), (E, D), (B, D), (E, F)$  est égale aussi à 20. Cette égalité sert comme un certificat d'optimalité du flot et de la coupe respectivement. C'est la preuve que ce flot est maximal, tandis que la coupe est minimale. On observe que tous les arcs sur cette coupe sont saturés - ce qui est indiqué par deux traits verticaux.

## Graphes d'écart (graphe résiduel)

- Soit  $\phi = [\phi_1, \phi_2, \dots, \phi_m]^T$  un flot entre  $s$  et  $t$  compatible avec  $c_u$ .
- Le graphe d'écart associé à  $\phi$  est le graphe  $\bar{G}(\phi) = [V, \bar{U}(\phi)]$  où  $\bar{U}(\phi)$  est constitué de façon suivante :  
pour  $\forall u = (i, j) \in U$  de  $G$  on associe au plus deux arcs de  $\bar{G}(\phi)$ 
  - $u^+ = (i, j)$  si  $\phi_u < c_u$  avec capacité (résiduelle)  $c_u - \phi_u > 0$
  - $u^- = (j, i)$  si  $\phi_u > 0$  avec capacité (résiduelle)  $\phi_u > 0$

Remarque : si  $\phi_u = c_u$ , on lui associe seulement  $u^-$ , si  $\phi_u = 0$ , on lui associe seulement  $u^+$ .

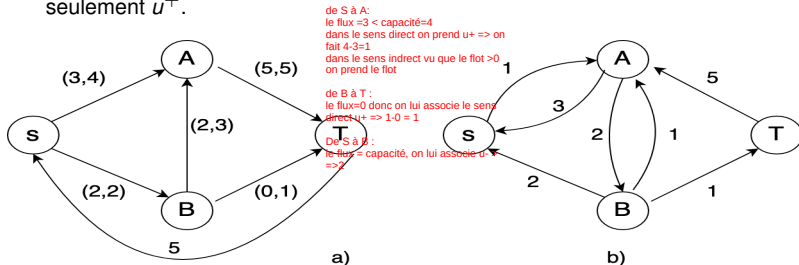
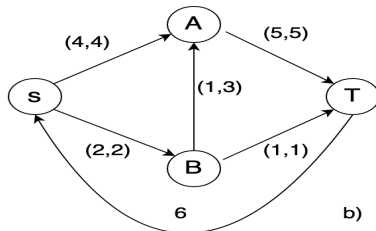
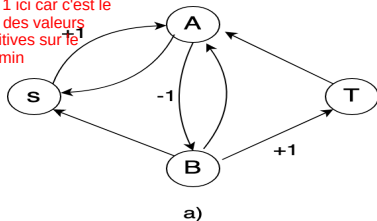


Figure 69 – a) Un réseau  $G$  avec un flot  $\phi$  de valeur 5. Le couple  $(\phi_u, c_u)$  associé à chaque arc  $u$  indique la valeur du flot  $\phi_u$  et sa capacité  $c_u$ . b) : Le graphe d'écart  $\bar{G}(\phi)$  associé au flot  $\phi$ .

## Un chemin améliorant

- Etant donné un flot  $\phi$ , un *chemin améliorant* ( $s \rightsquigarrow t$ ) est un chemin simple<sup>2</sup> de  $s$  à  $t$  dans le réseau résiduel  $\bar{G}(\phi)$ .
- Soit  $\varepsilon > 0$  le minimum des capacités résiduelles d'un chemin améliorant  $\pi$ .
- On ajoute  $\varepsilon$  au flot sur les arcs de type  $u^+$  et on l'enlève du flot des arcs de type  $u^-$  ; la loi de conservation du flot est satisfaite ainsi.
- On obtient ainsi un nouveau flot de valeur  $\phi'_0 = \phi_0 + \varepsilon > \phi_0$ .

$\varepsilon = 1$  ici car c'est le min des valeurs positives sur le chemin



**Figure 70** – a) Le chemin  $\pi = ((S, A)(A, B)(B, T))$  est un chemin améliorant ( $s \rightsquigarrow t$ ) dans le graphe résiduel  $\bar{G}(\phi)$  et  $\varepsilon = 1$  sur ce chemin. On ajoute cette valeur à la valeur du flot sur les arcs  $(S, A)$  et  $(B, T)$  ; on la soustrait de la valeur de l'arc  $(A, B)$ . b) Le nouveau flot de valeur  $\phi'_0 = 6$ . Peut-on l'améliorer ? Le réponse est donnée par le critère d'arrêt dans l'algorithme de Ford et Fulkerson.

2. On dit aussi un chemin élémentaire. C'est un chemin qui passe au plus une fois par chaque sommet.

## Algorithme de Ford et Fulkerson (1956)

La notion du chemin améliorant est à la base de l'algorithme de Ford et Fulkerson.

■  $k = 0$ . Partir d'un flot initial  $\phi^0$  compatible avec les capacités.

■ Soit  $\phi^k$  le flot courant.

Tant qu'il existe un chemin améliorant  $(s \rightsquigarrow t)$  dans  $\bar{G}(\phi^k)$  faire

soit  $\pi^k$  un chemin améliorant  $(s \rightsquigarrow t)$  dans  $\bar{G}(\phi^k)$ .

Soit  $\varepsilon^k$  le minimum des capacités résiduelles du  $\pi^k$ .

Définir le flot  $\phi^{k+1}$  par

$$\phi_u^{k+1} = \phi_u^k + \varepsilon^k \text{ si } u^+ \in \pi^k \quad (9)$$

$$\phi_u^{k+1} = \phi_u^k - \varepsilon^k \text{ si } u^- \in \pi^k \quad (10)$$

$$\phi_0^{k+1} = \phi_0^k + \varepsilon^k \quad (11)$$

$$k \leftarrow k + 1$$

■ fin tant que : le flot  $\phi^k$  est de valeur maximale.



## Algorithme de Ford et Fulkerson : illustration

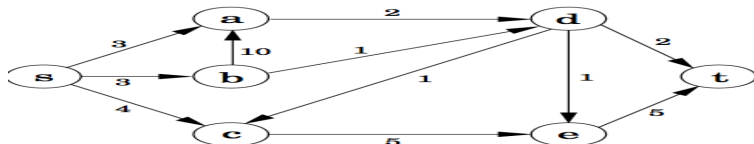


Figure 71 – Un réseau  $G = (V, U, C)$ . On cherche le flot de valeur maximale.

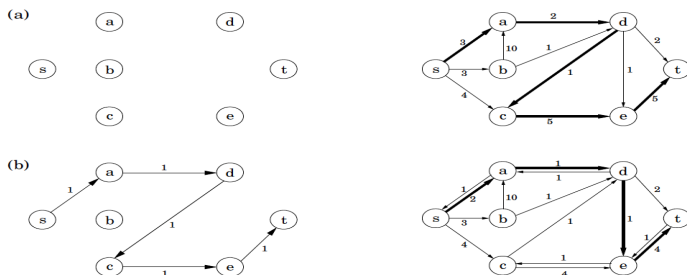


Figure 72 – Le fonctionnement de l'algorithme de Ford et Fulkerson sur le réseau de la figure 71. Le flot courant est visualisé à gauche, son graphe d'écart est représenté à droite. Les figures a) et b) visualisent les deux premières itérations avec le flot nul (la figure a) à gauche) et le flot de valeur 1 (la figure b) à gauche).

## Algorithme de Ford et Fulkerson : illustration (suite)

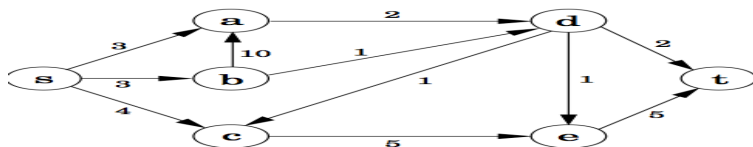


Figure 73 – Un réseau  $G = (V, U, C)$ . On cherche le flot de valeur maximale.

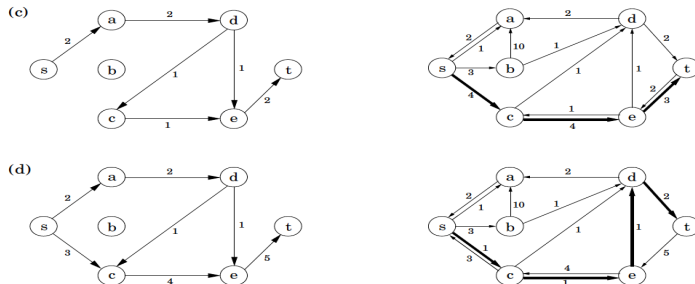
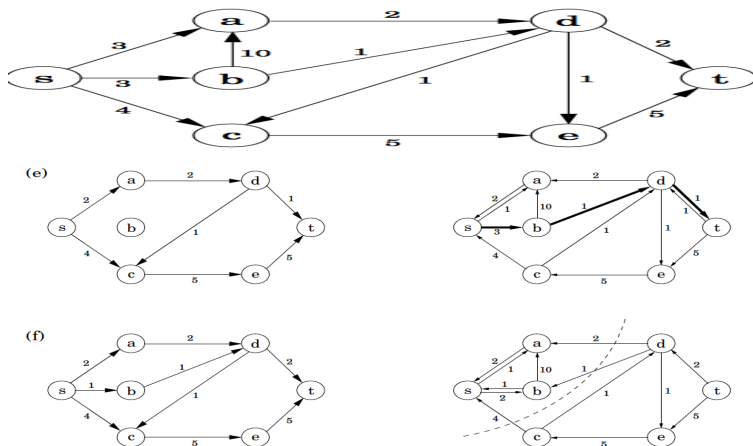


Figure 74 – Le fonctionnement de l'algorithme de Ford et Fulkerson sur le réseau de la figure 73. Le flot courant est visualisé à gauche, son graphe d'écart est représenté à droite. Les figures c) et d) visualisent les itérations 3 et 4 respectivement.

## Algorithme de Ford et Fulkerson : illustration (fin)



**Figure 75** – Les figures e) et f) visualisent les dernières itérations de l'algorithme de Ford et Fulkerson. Le flot de valeur 7 (la figure (f) à gauche) est l'optimal. L'optimalité est démontrée par le fait qu'il n'existe pas de chemin  $(s \rightsquigarrow t)$  dans le graphe d'écart  $\bar{G}(\phi^k)$  (la figure (f) à droite). Il n'y a que les sommets **a** et **b** qui sont joignables à partir du sommet **s**. Une coupe de capacité minimale, égale à la valeur du flot, est visualisée.

## La relation entre le flot maximum et la coupe minimale

**Théorème :** Soit  $\phi$  un flot compatible dans le réseau  $G$ . Les conditions suivantes sont équivalentes :

1.  $\phi$  est un flot maximum dans  $G$ .
2. Le réseau résiduel  $\tilde{G}(\phi)$  ne contient aucun chemin améliorant.
3. La valeur du flot  $\phi$  est égale à la capacité d'une coupe de capacité minimale séparant  $s$  et  $t$ .

Démonstration :

- (1)  $\Rightarrow$  (2) : Evident
- (2)  $\Rightarrow$  (3) : On définit  $L = \{v \in V : \exists \text{ un chemin de } s \text{ à } v \text{ dans } \tilde{G}(\phi)\}$ .  
Soit  $R = V \setminus L$ . La partition  $(L, R)$  est une coupe : de façon triviale  $s \in L$  et  $t \in R$ .  
Par hypothèse, pour chaque arc  $(u, v)$  tq  $u \in L$  et  $v \in R$  on a  $\phi(u, v) = c(u, v)$ .  
D'autre part, pour chaque arc  $(v, u)$  tq  $v \in R$  et  $u \in L$  on a  $\phi(v, u) = 0$ .  
Donc,  $size(\phi) = C(L, R)$ .
- (3)  $\Rightarrow$  (1) : Conséquence du lemme : La valeur maximale d'un flot de  $s$  à  $t$  compatible avec  $c_u$  n'excède jamais la capacité d'une coupe séparant  $s$  et  $t$ .

## Analyse de complexité de l'algorithme de Ford-Fulkerson

## Le cas où la valeur de chaque capacité égale à 1

- La recherche d'un chemin améliorant dans le graphe résiduel peut se faire en  $O(|V| + |U|)$  (avec un des parcours vus en cours).
- Supposons que chaque sommet  $u \in U$  est incident avec au moins un arc. Alors  $m = |U|$ ,  $m \geq \frac{|V|}{2}$  et  $O(|V| + |U|) = O(m)$
- La mise à jour du flot dans chaque itération de la boucle **tant que** (équations (9) et (10) de l'algorithme de Ford-Fulkerson) nécessite  $O(m)$  soustractions/additions.
- Considérons le cas où la valeur de chaque capacité vaut 1. Alors chaque itération de la boucle **tant que** sature exactement un arc. Donc, le nombre d'itérations de la boucle **tant que** ne dépasse pas  $\min(d^+(s), d^-(t))$ .
- En conséquence, le temps total de l'algorithme de Ford-Fulkerson (FF) dans le cas où la valeur de chaque capacité égale à 1 est en  $O(\min(d^+(s), d^-(t))m)$ .

## Le cas où la valeur de chaque capacité est une valeur entière $\geq 1$

Réécrivons l'algorithme de Ford-Fulkerson de la manière suivante :

---

**Algorithm 10** Ford-Fulkerson algorithm-bis

---

- 1: **Initialisation** : set  $\phi(e) = 0, \forall e \in E$  to be the initial flow
  - 2: **while** there is an  $s \rightsquigarrow t$  path in the residual graph  $\bar{G}(\phi)$  **do**
  - 3:   Let  $\pi$  be the current  $s \rightsquigarrow t$  path in  $\bar{G}(\phi)$
  - 4:    $\phi' = \text{augment}(\phi, \pi)$
  - 5:   Update  $\phi$  to be  $\phi'$
  - 6:   Update the residual graph  $\bar{G}(\phi)$  to be  $\bar{G}(\phi')$
  - 7: **end while**
  - 8: Return  $\phi$
- 

### Theorem

*A chaque étape intermédiaire les valeurs du flot  $\phi(e)$  et des capacités résiduelles  $\bar{G}(\phi)$  sont entières.*

## Analyse de complexité quand les capacités sont des entiers $\geq 1$

### Theorem

Notons  $\phi$  le flot dans  $G$ , et soit  $\pi$  un simple chemin  $s \rightsquigarrow t$  dans  $\bar{G}(\phi)$ . Alors  $v(\phi') = v(\phi) + \text{bottleneck}(\pi, \phi)$ ; et puisque  $\text{bottleneck}(\pi, \phi) > 0$ , on a  $v(\phi') > v(\phi)$ .

- Pour démontrer la terminaison de l'algorithme, nous allons expliciter une borne supérieure des valeurs du flot.
- Supposons que tous les arcs sortants de  $s$  et entrant dans  $t$  soient saturés; la valeur du flot dans ce cas devrait être égale à  $C = \min\left(\sum_{e \in \Gamma^+(s)} c_e, \sum_{e \in \Gamma^-(t)} c_e\right)$ .
- Nous avons ainsi  $v(\phi) \leq C$  pour chaque flot  $s \rightsquigarrow t$ .

### Theorem

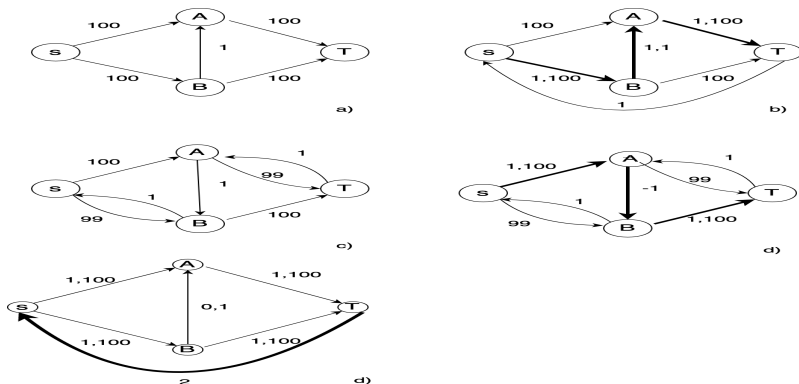
L'algorithme de Ford-Fulkerson effectue au plus  $C$  itérations de la boucle **tant que**.

### Theorem

L'algorithme de Ford-Fulkerson s'exécute en  $O(mC)$  temps.



## Un cas difficile pour l'algorithme de Ford et Fulkerson



**Figure 76 – a)** Analysons le comportement de l'algorithme de FF pour ce réseau ; **b)** Le chemin améliorant (S,B,A,T) permet d'obtenir un flot de valeur 1 ; **c)** Le graphe résiduel associé à ce flot ; **d)** Le chemin améliorant (S,A,B,T) sur ce graphe permet d'améliorer la valeur du flot et elle devient 2 ; Ainsi l'algorithme de Ford et Fulkerson peut continuer de sélectionner les chemins (S,B,A,T) et (S,A,B,T) alternativement 100 fois en améliorant chaque fois la valeur du flot d'une unité. Cette stratégie dépend de la valeur des capacités et n'est pas efficace. On peut facilement l'améliorer en choisissant le chemin le plus court de  $s$  vers  $t$  (cette stratégie est connue sous le nom de l'algorithme d'Edmonds-Karp).

La borne de l'algorithme de Ford-Fulkerson peut être améliorée si on implémente la recherche du chemin améliorant à la ligne 2 avec une recherche en largeur (c.a.d. le chemin améliorant est un plus court chemin de  $s$  vers  $t$  dans le graphe résiduel, où chaque arc possède une pondération unitaire). Cette implémentation a pour nom **algorithme de Edmonds-Karp**. On peut démontrer que cet algorithme s'exécute en  $O(|V| \times |U|^2) = O(n \times m^2)$  où  $V$  est l'ensemble de sommets et  $U$  est l'ensemble d'arcs.

# Applications

## Barrage routier par les flots

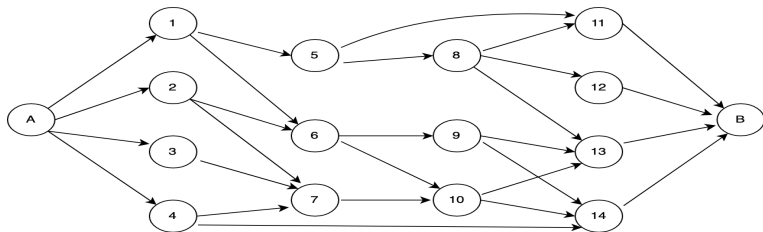


Figure 77 – Le réseau routier d'une île. Pour exprimer leur mécontentement, les agriculteurs décident de bloquer toute possibilité d'aller de A à B. Pour réduire la gêne occasionnée, on envisage bloquer seulement les routes, pas les agglomérations.

- Proposer une méthode générale pour déterminer où placer les barrages routiers.
- Déterminer le nombre minimal de barrages nécessaires et leur emplacement.
- Les forces publiques de la ville A ont décidé de surveiller les routes au départ de A. Toute possibilité de dresser un barrage sur ces routes est donc exclue. Déterminer à nouveau le nombre minimal de barrages nécessaires et leur emplacement.

## Barrage routier par les flots - suite

Le problème consiste à supprimer un nombre minimal d'arcs de façon à ce qu'il n'existe plus de chemin de A vers B. Le problème se ramène à la recherche d'une coupe de capacité minimale. On affecte une capacité 1 à chaque arc. La valeur d'une coupe fournit le nombre d'arcs de supprimer afin d'éliminer toute communication de A vers B.

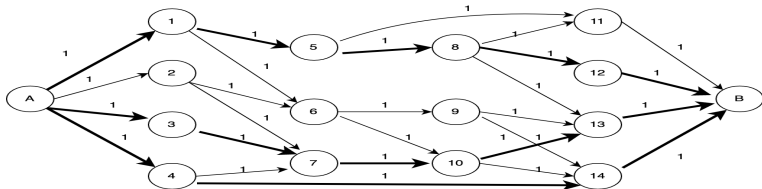


Figure 78 – On construit un flot de valeur 3.

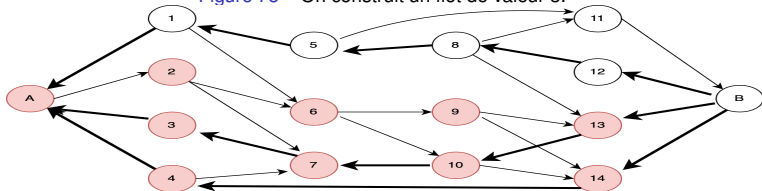


Figure 79 – Le flot trouvé est le flot maximum puisqu'il n'existe pas de chemin dans le graphe résiduel. La coupe associée (donnée par la partition des sommets en sommets rouges et blancs) est la coupe minimale. Elle suggère l'emplacement des barages – en l'occurrence sur les arcs  $(A, 1)$ ,  $(13, B)$ ,  $(14, B)$ .

## Barrage routier par les flots - suite

Pour modéliser le fait que les forces publiques de la ville A ont décidé de surveiller les routes au départ de A, on affecte une capacité  $\infty$  aux arcs sortants de A.

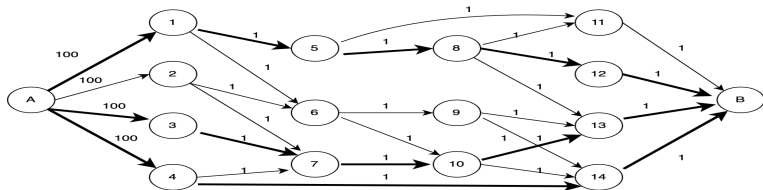


Figure 80 – On construit un flot de valeur 3.

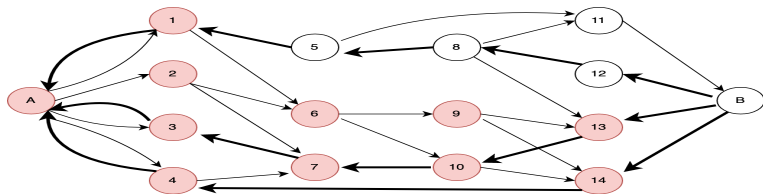


Figure 81 – Le flot trouvé est le flot maximum puisqu'il n'existe pas de chemin dans le graphe résiduel. La coupe associée (donnée par la partition des sommets en sommets rouges et blancs) est la coupe minimale. Elle suggère l'emplacement des barages – en l'occurrence sur les arcs  $(1, 5)$ ,  $(13, B)$ ,  $(14, B)$ .

## Construction d'autoroutes

Avant d'établir un projet de construction d'autoroutes on désire étudier la capacité du réseau routier, représenté par le graphe ci-dessous, reliant la ville S à la ville T. Pour cela, on a évalué le nombre maximal de véhicules que chaque route peut écouler par heure, compte tenu des ralentissements aux traversées des villes et villages, des arrêtes aux feux etc. ...Ces évaluations sont indiquées entre crochets, en centaines de véhicules par heure sur les arcs du graphe. Les temps de parcours entre villes sont tels que les automobilistes n'emprunteront que les chemins représentés par le graphe. On cherche le débit horaire total maximal de véhicules susceptible de s'écouler entre les villes S et T.

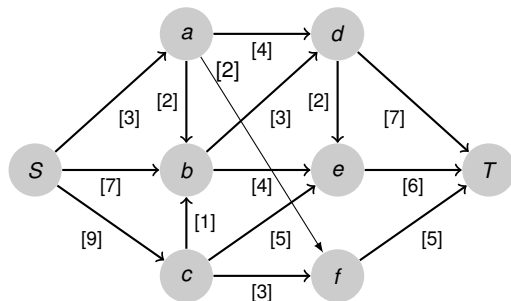


Figure 82 – Capacité d'un réseau routier.

## Arbres couvrants minimaux (ACM)



# Définitions et propriétés principales

**Définition :** Un graphe, non-orienté, connexe et acyclique est dit arbre.

On peut démontrer les propriétés suivantes :

- Propriétés :**
- Un arbre avec  $n$  sommets possède  $n - 1$  arêtes.
  - Si le graphe  $G = (V, E)$ , non-orienté et connexe, est tel que  $|E| = |V| - 1$ , alors  $G$  est un arbre.
  - Un graphe non-orienté est un arbre, si et seulement si, entre chaque couple de sommets il existe un seul chemin.

**Problème :** Donné un graphe  $G = (V, E, w_e, e \in E)$ , non-orienté, connexe. On cherche  $T \subseteq E$  qui connecte tous les sommets et qui minimise le poids total

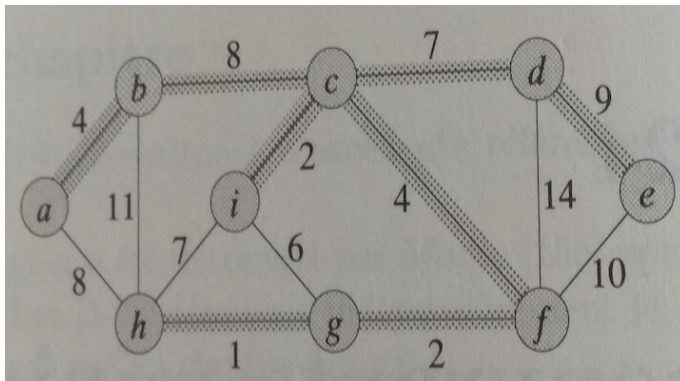
$$w(T) = \sum_{(u,v) \in T} w(u,v).$$

On voit facilement que  $T$  est acyclique, donc un arbre.  $T$  est appelé **Arbre Couvrant Minimal (ACM)**<sup>3</sup>.

---

3. " Arbre Couvrant Minimal" est un raccourci pour " Arbre Couvrant de poids Minimal". Tous les arbres couvrants comportent exactement  $|V| - 1$  arêtes.

## Arbre Couvrant Minimal : Illustration



**Figure 83** – Un graphe connexe. Les arêtes de l'arbre couvrant sont grisées. Le poids total de l'arbre montré est 37. L'arbre n'est pas unique : on peut remplacer l'arête  $(b, c)$  par l'arête  $(a, h)$  et on obtient un autre arbre de même poids.

### Des chemins forestiers

Les Eaux et Forêts ont décidé d'abattre 5 bosquets situés dans une forêt. Les distances mutuelles entre les bosquets sont données (en km) par la matrice suivante.

|   | a  | b  | c | d | e |
|---|----|----|---|---|---|
| a |    | 10 | 1 | 2 | 5 |
| b | 10 |    | 6 | 4 | 3 |
| c | 1  | 6  |   | 8 | 9 |
| d | 2  | 4  | 8 |   | 7 |
| e | 5  | 3  | 9 | 7 |   |

Pour mener à bien cette exploitation il est nécessaire de tracer un réseau de chemins de coût de construction minimum qui permette de circuler entre tous les bosquets. Les coûts de construction de ces chemins sont proportionnels à leurs longueurs.

- Enumérer les algorithmes vus en cours pouvant résoudre ce problème. Donner la complexité de ces algorithmes pour un graphe quelconque  $G = (V, E)$ .
- etc. (cf. le sujet d'examen du décembre 2011 )

# Définitions et propriétés principales

**Problème :** Donné un graphe  $G = (V, E, w_e, e \in E)$ , non-orienté, connexe. On cherche  $T \subseteq E$  qui connecte tous les sommets et qui minimise le poids total  $w(T) = \sum_{(u,v) \in T} w(u,v)$ . On voit facilement que  $T$  est acyclique, donc un arbre.  $T$  est appelé **Arbre Couvrant Minimal (ACM)**.

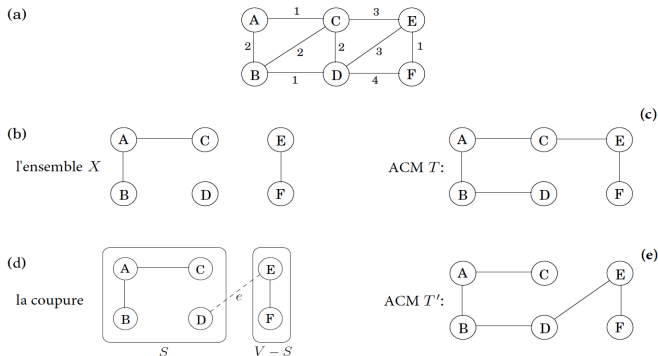
**Définitions :**

- Soit  $T$  un ACM, et soit  $X \subseteq T$ . Une arête  $(u, v)$ , telle que  $X \cup \{(u, v)\} \subseteq T$ , est appelée *arête sure* pour  $X$ .
- Une *coupure*  $(S, V \setminus S)$  du graphe  $G(V, E)$  est une partition des sommets  $V$ .
- Une arête *traverse la coupure*  $(S, V \setminus S)$  si l'une de ses extrémités appartient à  $S$  et la seconde appartient à  $V \setminus S$ .
- Un ensemble d'arêtes  $X$  traverse la coupure  $(S, V \setminus S)$  si  $\exists$  au moins une arête  $e \in X$  qui traverse cette coupure.
- Une arête est dite *minimale* pour la traversée de la coupure si son poids est minimal parmi toutes les arêtes qui traversent la coupure.

**Théorème :** Soit  $T$  un ACM, et soit  $X \subset T$ . Soit  $S \subset V$  tq  $X$  ne traverse pas la coupure  $(S, V \setminus S)$  (on dira que la coupure respecte  $X$ ), et soit  $(u, v)$  une arête minimale traversant  $(S, V \setminus S)$ . Alors  $(u, v)$  est une arête sure pour  $X$ .

## Théorème de l'arête minimale qui traverse la coupure : Illustration

Soit  $T$  un ACM, et soit  $X \subset T$ . Soit  $S \subset V$  tq  $X$  ne traverse pas la coupure  $(S, V \setminus S)$  (on dira que la coupure respecte  $X$ ), et soit  $e = (u, v)$  une arête minimale traversant  $(S, V \setminus S)$ . Alors  $e$  est une arête sûre pour  $X$ .

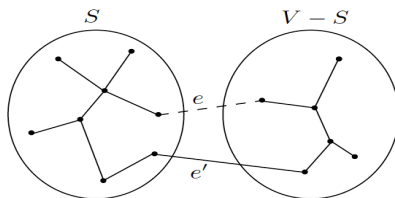


**Figure 84** – (a) Un graphe  $G = (V, E)$ ; (b) l'ensemble  $X = \{(A, B), (A, C), (E, F)\}$  a trois arêtes et fait partie de l'ACM  $T$  visualisé sur (c); (d)  $S = \{A, B, C, D\}$  est la coupure  $(S, V \setminus S)$  respecte  $X$  puisqu'aucune arête de  $X$  ne traverse la coupure. (e) l'arête minimale est  $e = (D, E)$ . D'après le théorème de l'arête minimale  $X \cup \{e\}$  fait partie d'un ACM (à l'occurrence  $T'$ ).

## Théorème de l'arête minimale qui traverse la coupure : preuve

Soit  $T$  un ACM, et soit  $X \subset T$ . Soit  $S \subset V$  tq  $X$  ne traverse pas la coupure  $(S, V \setminus S)$  (on dira que la coupure respecte  $X$ ), et soit  $e = (u, v)$  une arête minimale traversant  $(S, V \setminus S)$ . Alors  $e$  est une arête sûre pour  $X$ .

**Preuve :** Si  $e \in T$  il n'y a rien à démontrer. Supposons  $e \notin T$ . Puisque  $T$  est un arbre,  $\exists$  dans  $T$  un chemin entre les sommets  $u \in S$  et  $v \in V \setminus S$ . Soit  $e' \in T$  l'arête de ce chemin qui traverse la coupure  $(S, V \setminus S)$ . Notons  $T' = T \cup \{e\} \setminus \{e'\}$ .  $T'$  est un arbre (puisque un graphe connexe et a le même nombre d'arêtes que  $T$ ). De plus,  $w(T') = w(T) - w(e') + w(e) \leq w(T)$  puisque  $w(e) \leq w(e')$ .  $T'$  est donc un ACM.



## Algorithme générique pour la construction d'un ACM

---

**Algorithm 11** ACM\_generique ( $G(V,E),w$ )

---

**Require:** Un graphe  $G = (V, E)$  avec des poids  $w_e, e \in E$ .

**Ensure:** Un ACM défini par les arêtes de l'ensemble  $X$ .

$X \leftarrow \emptyset$

**while**  $X$  ne forme pas un arbre couvrant **do**

    trouver pour  $X$  une arête sûre  $(u, v)$

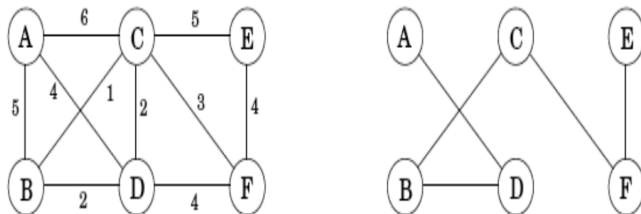
$X \leftarrow X \cup \{(u, v)\}$

**end while**

**return**  $X$

---

## Illustration pour l'algorithme **ACM\_Prim** ( $G(V,E),w,A$ )



| itér | S                  | F        |               |               |               |               |               |
|------|--------------------|----------|---------------|---------------|---------------|---------------|---------------|
|      |                    | A        | B             | C             | D             | E             | F             |
| 0    | {}                 | <b>0</b> | $\infty, nil$ | $\infty, nil$ | $\infty, nil$ | $\infty, nil$ | $\infty, nil$ |
| 1    | {A}                | —        | 5, A          | 6, A          | <b>4, A</b>   | $\infty, nil$ | $\infty, nil$ |
| 2    | {A; D}             | —        | <b>2, D</b>   | 2, D          | —             | $\infty, nil$ | 4, D          |
| 3    | {A; D; B}          | —        | —             | <b>1, B</b>   | —             | $\infty, nil$ | 4, D          |
| 4    | {A; D; B; C}       | —        | —             | —             | —             | 5, C          | <b>3, C</b>   |
| 5    | {A; D; B; C; F}    | —        | —             | —             | —             | <b>4, F</b>   | —             |
| 6    | {A; D; B; C; F; E} | —        | —             | —             | —             | —             | —             |

Figure 85 – Un graphe et son ACM de poids 14 trouvé par l'algorithme de Prim.



## Algorithme de Prim pour la construction d'un ACM (version tbl)

---

### Algorithm 12 ACM\_Prim ( $G(V,E),w,r$ )

---

**Require:** Un graphe  $G = (V, E)$  avec des poids  $w_e, e \in E$ .  $r$  est le sommet initial.

**Ensure:** Un ACM défini par les pointeurs  $prev(v), \forall v \in V$ .

```
1: Initialisation :  $\forall u \in V : clef(u) \leftarrow \infty$  and  $prev(u) \leftarrow nil$ ;  $S \leftarrow \emptyset$ ;  $clef(r) \leftarrow 0$ ;
2:  $F \leftarrow \text{maketbl}(V)$  {crée un tbl  $F$  avec  $clef(v), v \in (V)$  comme clés}
3: while  $F$  is not empty do
4:    $u \leftarrow \text{ExtractMin}(F)$  {l'élément  $u$  sera considéré hors  $F$ }
5:    $S \leftarrow S \cup \{u\}$ 
6:   for all edges  $(u, v) \in E$  do
7:     if  $v \in F$  and  $clef(v) > w(u, v)$  then
8:        $clef(v) \leftarrow w(u, v)$ 
9:        $prev(v) \leftarrow u$ 
10:    end if
11:  end for
12: end while
13: return  $prev$ 
```

---

- **Complexité** : pour chaque  $u$  de la ligne 4 on fait  $d(u)$  mises à jour. En total  $\sum_{v \in V} d(u) = O(|E|)$  mises à jour. Le coût de  $O(|V|)$  recherches de min dans un tableau est  $O(|V|^2)$ . En total  $O(|V|^2 + |E|) = O(|V|^2)$ .

## Arbres parfaits partiellement ordonnés et tas binaire (**Binary heap**)

- Le tas binaire (**Binary heap**) est une file de priorité importante fréquemment utilisée pour améliorer la complexité des algorithmes comme le tri par tas, l'alg. de Dijkstra, l'alg. de Prim ....
- **Définition** : Soit  $\pi(j)$  la clé associée avec l'élément  $j$  de la file  $F$ .  $F$  est un tas binaire si la condition suivante est satisfaite (relation entre le père et ses deux fils) :

$$\pi(j) \geq \pi(\lfloor \frac{j}{2} \rfloor) \quad \forall j \quad (12)$$

$$\iff \pi(j) \leq \pi(2j) \text{ and } \pi(j) \leq \pi(2j+1) \quad \forall j$$

- On représente le tas binaire par un **arbre parfait partiellement ordonné** :
  - $\forall$  niveau est rempli de gauche à droite ;
  - $\forall$  niveau est complètement rempli avant de commencer le niveau suivant ;
  - la condition (12) est satisfaite (l'élément contenu dans tout nœud est inférieur ou égal aux éléments contenus dans les fils de ce nœud).
- Le tas est facile à implémenter en tableau.

## Arbres parfaits partiellement ordonnés et tas binaire : implementation en tableau

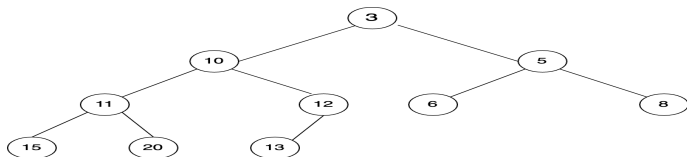


Figure 86 – L'arbre parfait qui visualise un tas binaire avec 10 éléments. Seules les clés sont indiquées. Ci-dessous le tableau  $\pi$  associé avec ce tas binaire.

| j        | 1 | 2  | 3 | 4  | 5  | 6 | 7 | 8  | 9  | 10 |
|----------|---|----|---|----|----|---|---|----|----|----|
| $\pi(j)$ | 3 | 10 | 5 | 11 | 12 | 6 | 8 | 15 | 20 | 13 |

On a les règles suivantes :

- $\pi(1)$  est la racine qui contient l'élément le plus petit ;
- $\pi(2j)$  and  $\pi(2j + 1)$  sont les deux fils, s'ils existent, de  $\pi(j)$ . En plus,  $\pi(j) \leq \pi(2j)$  and  $\pi(j) \leq \pi(2j + 1)$ . ;  
**exemple**  $j = 4 : \pi(4) = 11, \pi(8) = 15, \pi(9) = 20$  ;
- $\pi(\lfloor \frac{j}{2} \rfloor)$  est le père de  $\pi(j)$  pour  $j > 1$  ;  
**exemple**  $j = 9 : \pi(9) = 20, \pi(\lfloor \frac{9}{2} \rfloor) = \pi(4) = 11$  ;

### Propriétés d'un tas de $k$ éléments :

Chaque élément d'un tas binaire est caractérisé par le couple (nom, valeur numérique)  $(v, clef(v))$ .

- la racine contient le plus petit élément. L'opération « accès à la racine » se fait en  $\Theta(1)$ ;
- l'opération « insertion » se fait en  $\Theta(\log_2 k)$ ;
- l'opération « suppression du minimum » se fait en  $\Theta(\log_2 k)$ .

## Arbres parfaits partiellement ordonnés et tas binaire : illustration du fonctionnement

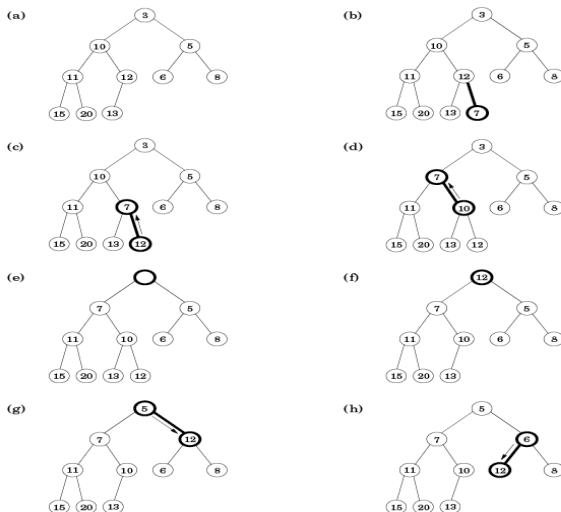
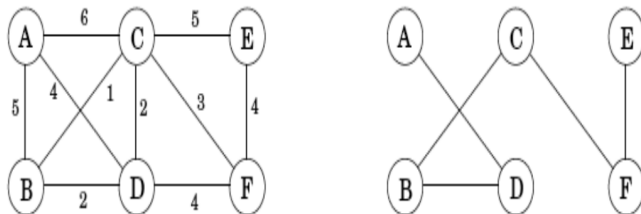


Figure 87 – (a) : un tas binaire avec 10 éléments. Seules les clés sont indiquées. (b)-(d) : illustration de l'opération « insertion ». (e)-(g) : illustration de l'opération « extraction et suppression du minimum ».

## Illustration pour l'algorithme **ACM\_Prim** ( $G(V,E),w,A$ )



| itér | S                  | F        |               |               |               |               |               |
|------|--------------------|----------|---------------|---------------|---------------|---------------|---------------|
|      |                    | A        | B             | C             | D             | E             | F             |
| 0    | {}                 | <b>0</b> | $\infty, nil$ | $\infty, nil$ | $\infty, nil$ | $\infty, nil$ | $\infty, nil$ |
| 1    | {A}                | —        | 5, A          | 6, A          | <b>4, A</b>   | $\infty, nil$ | $\infty, nil$ |
| 2    | {A; D}             | —        | <b>2, D</b>   | 2, D          | —             | $\infty, nil$ | 4, D          |
| 3    | {A; D; B}          | —        | —             | <b>1, B</b>   | —             | $\infty, nil$ | 4, D          |
| 4    | {A; D; B; C}       | —        | —             | —             | —             | 5, C          | <b>3, C</b>   |
| 5    | {A; D; B; C; F}    | —        | —             | —             | —             | <b>4, F</b>   | —             |
| 6    | {A; D; B; C; F; E} | —        | —             | —             | —             | —             | —             |

Figure 88 – Un graphe et son ACM de poids 14 trouvé par l'algorithme de Prim.

## Algorithme de Prim pour la construction d'un ACM (version tas binaire)

---

### Algorithm 13 ACM\_Prim ( $G(V,E),w,r$ )

---

**Require:** Un graphe  $G = (V, E)$  avec des poids  $w_e, e \in E$ .  $r$  est le sommet initial.

**Ensure:** Un ACM défini par les pointeurs  $prev(v), \forall v \in V$ .

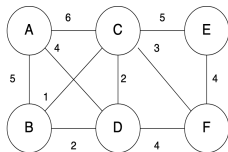
```
1: Initialisation :  $\forall u \in V : clef(u) \leftarrow \infty$  and  $prev(u) \leftarrow nil; clef(r) \leftarrow 0;$   
2:  $F \leftarrow \text{makequeue}(V)$  {crée une file de priorité F (binary heap) }  
3: while F is not empty do  
4:    $u \leftarrow \text{ExtractMin}(F)$   
5:   for all edges  $(u, v) \in E$  do  
6:     if  $clef(v) > w(u, v)$  then  
7:        $clef(v) \leftarrow w(u, v)$   
8:        $prev(v) \leftarrow u$   
9:       ChangeKey(F,v) {F is updated according to the new value of  $clef(v)$  }  
10:    end if  
11:  end for  
12: end while  
13: return  $prev$ 
```

---

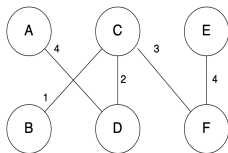
### Complexité :

- La ligne (2) s'exécute en  $O(|V| \log |V|)$ .
- pour un sommet  $u$  donné par la ligne 4 on fait  $d(u)$  mises à jour de la clef.
- En total, pour  $\forall v \in V$ , on fait  $O(|E|)$  mises à jour de la clef. Donc la boucle (3-11) s'exécute en  $|E| \log |V|$  puisque chaque mise à jour se fait en  $O(\log |V|)$ .
- On obtient en total :  $O(|V| \log |V| + |E| \log |V|) = O(|E| \log |V|)$ .

## Algorithme de Kruskal pour la construction d'un ACM **ACM\_Kruskal** **(G(V,E),w) : Illustration**



a)



b)

| Arête<br>considérée | Poids | Garder<br>l'arête | count |
|---------------------|-------|-------------------|-------|
| (B,C)               | 1     | Oui               | 1     |
| (C,D)               | 2     | Oui               | 2     |
| (B,D)               | 2     | Non               | 2     |
| (C,F)               | 3     | Oui               | 3     |
| (A,D)               | 4     | Oui               | 4     |
| (D,F)               | 4     | Non               | 4     |
| (E,F)               | 4     | Oui               | 5     |
| (A,B)               | 5     | Non               | -     |
| (C,E)               | 5     | Non               | -     |
| (A,C)               | 6     | Non               | -     |

c)

**Figure 89** – a) un graphe  $G = (V, E)$  ; b) son l'ACM de poids 14 ; c) Illustration du fonctionnement de l'alg. de Kruskal pour ce graphe. L'algorithme s'arrête quand le compte *count* atteint la valeur  $|V| - 1$ .



## Algorithme de Kruskal pour la construction d'un ACM

---

**Algorithm 14** ACM\_Kruskal ( $G(V,E),w$ )

---

**Require:** Un graphe  $G = (V, E)$  avec des poids  $w_e, e \in E$ .

**Ensure:** Un ACM défini par les arêtes de l'ensemble  $X$ .

```
for all  $u \in V$  do
    makeset( $u$ )
end for
 $X \leftarrow \emptyset$ 
trier les arêtes  $E$  par leurs poids
for all  $(u, v) \in E$ , dans l'ordre croissant des poids do
    if find( $u$ )  $\neq$  find( $v$ ) then
         $X \leftarrow X \cup \{(u, v)\}$ 
        union( $u, v$ )
    end if
end for
return  $X$ 
```

---

- Complexité : On fait  $|V|$  makeset,  $2|E|$  find et  $|V| - 1$  unions.

## Opérations et structures de données pour les ensembles disjoints

La structure de données maintient à jour une collection  $\{S_1, S_2, \dots, S_k\}$  d'ensembles dynamiques disjoints. Chaque ensemble est un arbre orienté, dont la racine est le représentant de cet ensemble. Chaque noeud de l'arbre est muni d'un pointeur, ( $\pi$ ), vers son père, ainsi que d'un rang, (**rank**), qui équivaut la taille de sous-arbre enraciné à ce noeud. Cette structure permet de connaître rapidement à quel ensemble appartient un élément donné, et de pouvoir réunir deux ensembles.

## Opérations et structures de données pour les ensembles disjoints (suite)

---

**Algorithm 15** procedure makeset ( $x$ )

---

**Ensure:** crée un nouvel ensemble dont le seul élément (et donc le représentant) est

$x$ .

$\pi(x) \leftarrow x$

$rank(x) \leftarrow 0$

---

---

**Algorithm 16** function find( $x$ )

---

**Ensure:** retourne un pointeur vers le représentant de l'ensemble (unique) contenant

$x$ .

**while**  $x \neq \pi(x)$  **do**

$x \leftarrow \pi(x)$

**end while**

**return**  $x$

---

---

**Algorithm 17** function find\_bis( $x$ )

---

**Ensure:** effectue une compression du chemin lors de l'opération find

**if**  $x \neq \pi(x)$  **then**

$\pi(x) \leftarrow \text{find}(\pi(x))$

**end if**

**return**  $\pi(x)$

---

---

**Algorithm 18** procedure union ( $x, y$ )

---

**Require:** Deux ensembles dynamiques  $S_x$  et  $S_y$  représentés par leurs racines  $x$  et  $y$ .  
 $S_x$  et  $S_y$  sont supposés disjoints avant l'opération.

**Ensure:** Réunit les ensembles dynamiques  $S_x$  et  $S_y$  dans  $S_x \cup S_y$ . On fait pointer la racine du moindre rang sur celle de rang supérieur au moment de l'union. Comme les ensembles de la collection sont obligatoirement disjoints, on supprime les ensembles  $S_x$  et  $S_y$ .

$r_x \leftarrow \text{find}(x)$

$r_y \leftarrow \text{find}(y)$

**if**  $r_x = r_y$  **then**

    return

**end if**

**if**  $\text{rank}(r_x) > \text{rank}(r_y)$  **then**

$\pi(r_y) \leftarrow r_x$

**else**

$\pi(r_x) \leftarrow r_y$

**if**  $\text{rank}(r_x) = \text{rank}(r_y)$  **then**

$\text{rank}(r_y) \leftarrow \text{rank}(r_y) + 1$

**end if**

**end if**

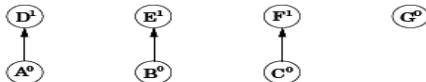
---

## Illustration pour les opérations sur les ensembles disjoints

After `makeset(A), makeset(B), ..., makeset(G)`:



After `union(A, D), union(B, E), union(C, F)`:



After `union(C, G), union(E, A)`:



After `union(B, G)`:

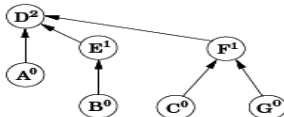


Figure 90 – Résultat des opérations **makeset** et **union** sur des ensembles disjoints

## Illustration pour les opérations sur les ensembles disjoints

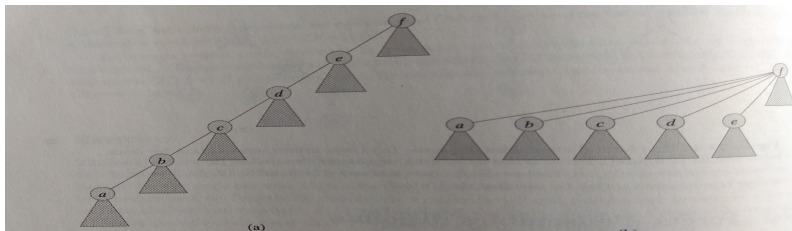


Figure 91 – Compression du chemin lors de la fonction **find-bis**.

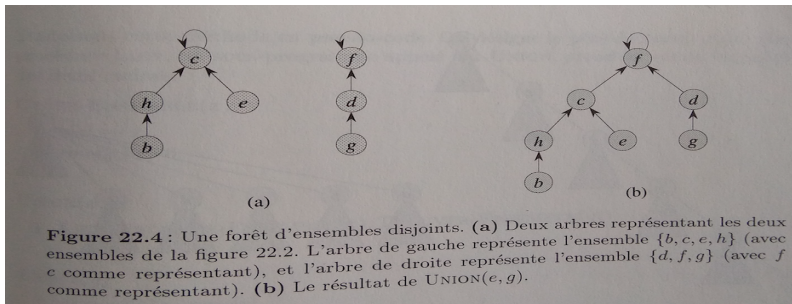


Figure 22.4 : Une forêt d'ensembles disjoints. (a) Deux arbres représentant les deux ensembles de la figure 22.2. L'arbre de gauche représente l'ensemble  $\{b, c, e, h\}$  (avec **c** comme représentant), et l'arbre de droite représente l'ensemble  $\{d, f, g\}$  (avec **f** comme représentant). (b) Le résultat de  $\text{UNION}(e, g)$ .

Figure 92 – Résultat de l'opération **union** sur des ensembles disjoints

### Propriétés des structures de données pour les ensembles disjoints :

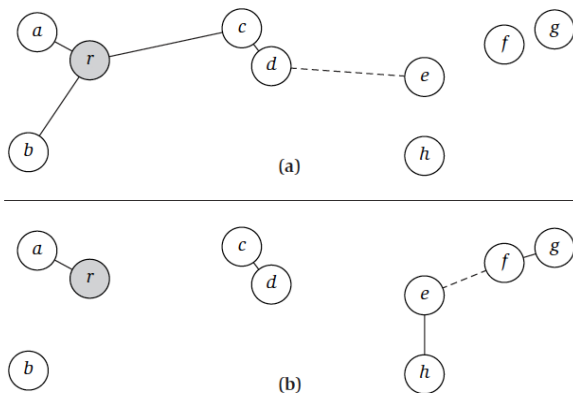
- $\forall x \neq \text{la racine}, \text{rank}(x) < \text{rank}(\pi(x))$
- Chaque racine de rang  $k$  a au moins  $2^k$  noeuds dans son arbre.
- Chaque ensemble de  $n$  éléments possède au plus  $\frac{n}{2^k}$  noeuds de rang  $k$ .
- La hauteur de l'arbre représentant un ensemble de  $n$  éléments ne dépasse pas  $\log n$ .

**Corolaire 1** La complexité des opérations find et union est en  $O(\log |V|)$ .

**Corolaire 2** La complexité de l'algorithme de Kruskal est en  $O(|E|(\log |E| + \log |V|)) = O(|E| \log |V|)$  puisque  $\log |E| \approx \log |V|$  (la même que la complexité de l'algorithme de Prim).

## Algorithme de Prim versus l'algorithme de Kruskal

Une instance géométrique du problème de l'arbre couvrant de poids minimal (ACM) : les poids des arêtes sont proportionnels à la distance géométrique entre les sommets.



**Figure 93** – (a) visualise les quatre premières arêtes ajoutées par l'algorithme de Prim à partir du sommet *r* ; (b) visualise les quatre premières arêtes ajoutés par l'algorithme de Kruskal. Pour les deux cas la ligne indiquée en pointillés est la suivante à être ajoutée.



# Calcul des Plus Courts Chemins (PCC) dans les graphes

Le cas des valeurs positives des arcs/arêtes

## Parcours en largeur d'abord et calcul des PCC de $s$ aux autres sommets : le cas où la longueur de chaque arête vaut 1

- $G = (V, E) \mid \forall (u, v) \in E, l(u, v) = 1$  (la longueur de chaque arête vaut 1).
- La distance du sommet  $s$  à tout sommet  $v \in V$  accessible depuis  $s$ , est notée  $v.dist$ . Elle représente la longueur d'un plus court chemin  $PCC(s \rightsquigarrow v)$ , en nombre d'arêtes entre  $s$  et  $v$ .
- Ces distances sont calculées par la version suivante de l'algorithme BFS( $G, s$ ) :

---

**Algorithm 19** BFS( $G, s$ ) (2<sup>e</sup> version du parcours en largeur d'abord).

---

**Require:**  $G=(V,E), \forall (u, v) \in E \ l(u,v)=1$ , start vertex  $s$ .

**Ensure:** For all vertices  $u$  reachable from  $s$ ,  $u.dist$  is set to the distance from  $s$  to  $u$ .

```
1: Initialisation :  $\forall v \in V : v.visited \leftarrow false$  and  $v.dist = \infty$ ;  $F \leftarrow [s]$ ;  $s.dist \leftarrow 0$ ;  
    $s.visited \leftarrow true$ ;  
2: while ( $F \neq \emptyset$ ) do  
3:    $u \leftarrow \text{ExtractMin}(F)$  : {  $u$  reçoit le sommet de la file  $F$ , le dernier est supprimé  
   de  $F$  }  
4:   while ( $\exists (u, v) \in E \mid v.visited = false$ ) do  
5:      $v.visited \leftarrow true$   
6:      $v.dist \leftarrow u.dist + 1$   
7:     inject( $F, v$ ) { le sommet  $v$  est inséré dans  $F$  }  
8:   end while  
9: end while
```

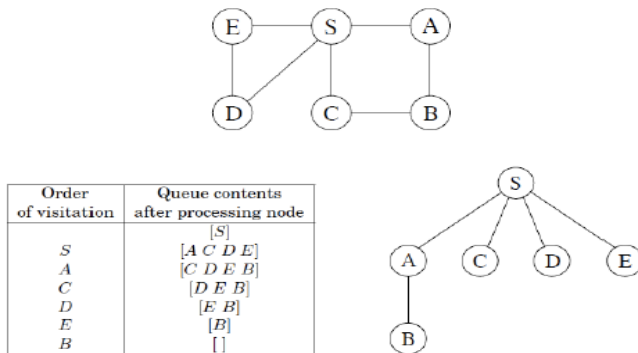


Figure 94 – Fonctionnement de l’algorithme 19 et la file F (FIFO) pour le graphe ci-dessus. On obtient que  $S.dist = 0, A.dist = C.dist = D.dist = E.dist = 1$  et  $B.dist = 2$ .

Remarque : tous les chemins partant de la racine  $s$  sont des PCC (c.-à-d. que ceci est un arbre des PCC de  $s$  à tous les autres sommets).

- **Proposition** : BFS( $G,s$ ) visite chaque sommet accessible à partir de  $s$ .
- *Preuve* : Par induction à la base des propriétés suivantes :
- Lors de l'exécution de BFS( $G,s$ ), pour chaque valeur des distances  $d = 0, 1, 2, \dots, \text{max\_dist}$  il existe un moment tel que
  - pour chaque  $v$  éloigné de  $s$  à une distance  $\leq d$ , la valeur  $v.\text{dist}$  a été correctement calculée.
  - pour tous les autres sommets  $u$ ,  $u.\text{dist} = \infty$ .
  - la file  $F$  contient uniquement les sommets  $u$  éloigné de  $s$  à une distance égale à  $d$ .
- Analyse de complexité (similaire à l'analyse de DFS( $G,s$ )) :
  - chaque sommet est inséré une fois dans  $F$  lors de sa première visite, et est éjecté de  $F$  lorsque tous ses voisins ont été visités ( en total  $2 \times |V|$  opérations insertions/éjections) .
  - Chaque arête  $(u,v) \in E$  est examinée exactement deux fois (en cas d'un graphe non orienté) ; chaque arc est examiné une seule fois (en cas d'un graphe orienté). Donc,  $\Theta(|E|)$  opérations sur les arêtes/arcs.
  - en total  $\Theta(|V| + |E|)$ .

---

**Algorithm 20** Dijkstra\_a ( $G, l, s$ ).

---

**Require:** Graph  $G=(V,E)$  with positive edge weights  $l_e > 0 : \forall e \in E$ , vertex  $s \in V$

**Ensure:**  $\forall u \in V$ ,  $dist(u)$  is set to the shortest distance from  $s$  to  $u$ ;  $prev(u)$  points to the previous of  $u$  on this shortest path

```
1: Initialisation :  $\forall u \in V : dist(u) \leftarrow \infty$  and  $prev(u) \leftarrow nil$ ;  $dist(s) \leftarrow 0$ ;  
2:  $P \leftarrow \emptyset$ , ( $P$  contient les sommets  $v$ , tq  $dist(v)$  est calculée définitivement)  
3:  $T \leftarrow V$  ( $T$  contient des sommets  $v$ , tq  $dist(v)$  n'est qu'une borne supérieure)  
4: while  $T$  is not empty do  
5:    $u \leftarrow \underset{v \in (T)}{\operatorname{argmin}} dist(v)$   
6:    $P \leftarrow P \cup \{u\}$   
7:    $T \leftarrow T \setminus \{u\}$   
8:   for all edges  $(u, v) \in E$  do  
9:      $dist\_update(u, v)$   
10:  end for  
11: end while
```

---

■ où

---

**Algorithm 21**  $\text{dist\_update}(u,v)$ .

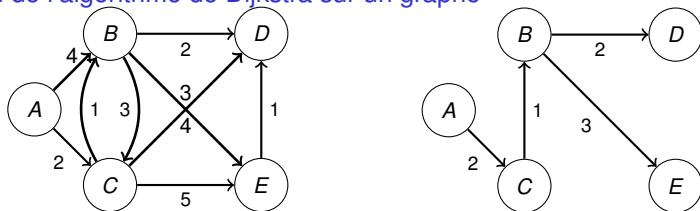
---

```
1: if  $\text{dist}(v) > \text{dist}(u) + l(u, v)$  then  
2:    $\text{dist}(v) = \text{dist}(u) + l(u, v)$   
3:    $\text{prev}(v) \leftarrow u$   
4: end if
```

---

Exemple :

## Application de l'algorithme de Dijkstra sur un graphe



| itér | $P$                 | $T$      |               |               |               |               |
|------|---------------------|----------|---------------|---------------|---------------|---------------|
|      |                     | $A$      | $B$           | $C$           | $D$           | $E$           |
| 0    | $\emptyset$         | <b>0</b> | $\infty, nil$ | $\infty, nil$ | $\infty, nil$ | $\infty, nil$ |
| 1    | $\{A\}$             | —        | 4, $A$        | <b>2, A</b>   | $\infty, nil$ | $\infty, nil$ |
| 2    | $\{A; C\}$          | —        | <b>3, C</b>   | —             | 6, $C$        | 7, $C$        |
| 3    | $\{A; C; B\}$       | —        | —             | —             | <b>5, B</b>   | 6, $B$        |
| 4    | $\{A; C; B; D\}$    | —        | —             | —             | —             | <b>6, B</b>   |
| 5    | $\{A; C; B; D; E\}$ | —        | —             | —             | —             | —             |

**Figure 95** – A est le sommet de départ. Chaque colonne  $u$  contient le couple  $(dist(u), prev(u))$  où  $prev(u)$  est le sommet qui précède  $u$  sur le plus court chemin de A à  $u$ . A chaque itération l'opération  $u \leftarrow \operatorname{argmin}_{v \in (T)} dist(v)$  est effectuée. Les valeurs  $dist(v)$  sont mises à jour suivant la

procédure  $dist\_update(u, v)$ . La mémorisation des valeurs  $prev(v)$  permet de construire l'arbre des plus courts chemins de A vers chaque sommet  $v$  du graphe (ci-dessus à droite).

## Justification de l'alg. de Dijkstra

- Soit le graphe  $G = (V, E, l_e > 0 : e \in E)$ . On cherche les  $\text{PCC}(s \rightsquigarrow u)$  : les plus courts chemins de  $s$  aux autres sommets  $u \in V \setminus \{s\}$ . Notons  $\text{dist}^*(u)$  la longueur du  $\text{PCC}(s \rightsquigarrow u)$ .
- L'algorithme procédera en  $|V| - 1$  itérations. Au début de chacune des itérations  $V$  est partitionné en deux sous-ensembles  $P$  et  $T$ , avec  $s \in P$ .
- Chaque sommet  $u \in V$  est muni d'une étiquette  $\text{dist}(u)$  qui vérifie les propriétés suivantes ;
  - si  $u \in P : \text{dist}(u) = \text{dist}^*(u)$
  - si  $u \in T : \text{dist}(u) = \min_{v \in P, v \in \Gamma^{-1}(u)} \text{dist}(v) + l(v, u)$

La valeur  $\text{dist}(u)$ ,  $u \in T$  donne la longueur du  $\text{PCC}(s \rightsquigarrow u)$  à condition que tous les sommets excepté  $u$  sont dans  $P$ . Ainsi, à chaque itération  $\text{dist}(u)$  fournit une borne supérieure de la valeur cherchée  $\text{dist}^*(u)$ . A début  $\text{dist}(u) = \infty$ , à la fin du calcul,  $\text{dist}^*(u) = \text{dist}(u)$ .



## Justification de l'alg. de Dijkstra

- L'algorithme procédera en  $|V| - 1$  itérations. Au début de chacune des itérations  $V$  est partitionné en deux sous-ensembles  $P$  et  $T$ , avec  $s \in P$ .
- Chaque sommet  $u \in V$  est muni d'une étiquette  $dist(u)$  qui vérifie :
  - si  $u \in P : dist(u) = dist^*(u)$
  - si  $u \in T :$

$$dist(u) = \min_{v \in P, v \in \Gamma^{-1}(u)} dist(v) + l(v, u) \quad (13)$$

- **Lemme :** Soit  $u \in T$  tq

$$dist(u) = \min_{v \in T} dist(v) \quad (14)$$

Alors  $dist^*(u) = dist(u)$ .

- **Preuve :** Il  $\exists$  évidemment un chemin  $\lambda : s \rightsquigarrow u$  de longueur  $dist(u)$ .  
Considérons un autre chemin  $\mu : s \rightsquigarrow u$ , et divisons le en deux parties  $\mu_1 : s \rightsquigarrow h$  et  $\mu_2 : h \rightsquigarrow u$ , où  $h$  est le premier sommet de  $T$  rencontré, et  $\mu_2$  le reste du chemin. Alors  $length(\mu_1) \geq dist(h)$  d'après (13); et  $dist(h) \geq dist(u)$  d'après (14). Puisque  $length(\mu_2) \geq 0$ , on obtient  $length(\mu) = length(\mu_1) + length(\mu_2) \geq length(\mu_1) \geq dist(h) \geq dist(u)$  et donc  $\lambda$  est le plus court chemin  $s \rightsquigarrow u$ .

---

**Algorithm 22** Dijkstra\_a ( $G, l, s$ ).

---

**Require:** Graph  $G=(V,E)$  with positive edge weights  $\{l_e > 0 : e \in E\}$ , vertex  $s \in V$

**Ensure:**  $\forall u \in V$ ,  $dist(u)$  is set to the shortest distance from  $s$  to  $u$ ;  $prev(u)$  points to the previous of  $u$  on this shortest path

```
1: Initialisation :  $\forall u \in V : dist(u) \leftarrow \infty$  and  $prev(u) \leftarrow nil$ ;  $dist(s) \leftarrow 0$ ;  
2:  $P \leftarrow \emptyset$ , ( $P$  contient les sommets  $v$ , tq  $dist(v)$  est calculée définitivement)  
3:  $T \leftarrow V$  ( $T$  contient des sommets  $v$ , tq  $dist(v)$  n'est qu'une borne supérieure)  
4: while  $T$  is not empty do  
5:    $u \leftarrow \underset{v \in (T)}{\operatorname{argmin}} dist(v)$   
6:    $P \leftarrow P \cup \{u\}$   
7:    $T \leftarrow T \setminus \{u\}$   
8:   for all edges  $(u, v) \in E$  do  
9:      $dist\_update(u, v)$   
10:  end for  
11: end while
```

---

- Alg 22 nécessite  $|V|$  opérations **argmin** plus  $|E|$  opérations **update**. La coût de ces opérations dépend du choix de la structure des données.
- Si on utilise **Array** : on obtient  $\Theta(|V|^2 + |E|)$ .

## Arbres parfaits partiellement ordonnés et tas binaire (**Binary heap**)

- Le tas binaire (**Binary heap**) est une file de priorité importante fréquemment utilisée pour améliorer la complexité des algorithmes comme le tri par tas, l'alg. de Dijkstra, l'alg. de Prim ....
- **Définition** : Soit  $\pi(j)$  la clé associée avec l'élément  $j$  de la file  $F$ .  $F$  est un tas binaire si la condition suivante est satisfaite (relation entre le père et ses deux fils) :

$$\pi(j) \geq \pi(\lfloor \frac{j}{2} \rfloor) \quad \forall j \quad (15)$$

$$\iff \pi(j) \leq \pi(2j) \text{ and } \pi(j) \leq \pi(2j+1) \quad \forall j$$

- On représente le tas binaire par un **arbre parfait partiellement ordonné** :
  - $\forall$  niveau est rempli de gauche à droite ;
  - $\forall$  niveau est complètement rempli avant de commencer le niveau suivant ;
  - la condition (15) est satisfaite (l'élément contenu dans tout nœud est inférieur ou égal aux éléments contenus dans les fils de ce nœud).
- Le tas est facile à implémenter en tableau.
- Les propriétés essentielles d'un tas binaire de  $k$  éléments :
  - la racine contient le plus petit élément. L'opération « accès à la racine » se fait en  $O(1)$  ;
  - l'opération « insertion » se fait en  $O(\log_2 k)$  ;
  - l'opération « suppression du minimum » se fait en  $O(\log_2 k)$ .

---

**Algorithm 23** Dijkstra\_bis( $G, l, s$ ).

---

**Require:** Graphe  $G=(V,E)$  directed or undirected, with positive edge weights  $\{l_e : e \in E\}$ ; vertex  $s \in V$

**Ensure:** For all vertices  $u \in V$ ,  $dist(u)$  is set to the shortest distance from  $s$  to  $u$ ;  
 $prev(u)$  points to the previous of  $u$  on this shortest path

- 1:  $\forall u \in V$  : initialize  $dist(u) \leftarrow \infty$  and  $prev(u) \leftarrow nil$
  - 2:  $dist(s) \leftarrow 0$
  - 3:  $P \leftarrow \{s\}$  { $P$  contient les sommets  $v$  pour lesquels la valeur  $dist(v)$  est calculée définitivement}
  - 4:  $T \leftarrow \text{makequeue}(V)$  {créer une **file de priorité T** avec  $dist(v), v \in (T)$  comme clés}
  - 5: **while**  $T$  is not empty **do**
  - 6:    $u \leftarrow \text{ExtractMin}(T)$
  - 7:    $P \leftarrow P \cup \{u\}$
  - 8:    $T \leftarrow T \setminus \{u\}$
  - 9:   **for all** edges  $(u, v) \in E$  **do**
  - 10:      $dist\_update\_bis(T, (u, v))$
  - 11:   **end for**
  - 12: **end while**
-

---

**Algorithm 24**  $\text{dist\_update\_bis}(T, (u, v))$ .

---

**Require:** **Priority queue**  $T$  using  $\text{dist}$  as keys; an edge  $(u, v) \in E$

**Ensure:**  $T$  is updated if the value of  $\text{dist}(v)$  has been modified

**if**  $\text{dist}(v) > \text{dist}(u) + l(u, v)$  **then**

$\text{dist}(v) \leftarrow \text{dist}(u) + l(u, v)$

$\text{prev}(v) \leftarrow u$

**ChangeKey**( $T, v$ ) { $T$  is updated according to the new value of  $\text{dist}(v)$ }

**end if**

---

Analyse de complexité de la version tas binaire de l'alg. de Dijkstra

- **makequeue**( $V$ ) nécessite  $O(|V| \log |V|)$  opérations.
- la boucle **WHILE** (line 6) nécessite  $O(|V|)$  **ExtractMin** plus  $O(|E|)$  **ChangeKey (updates)** :  $O((|V| + |E|) \log |V|)$  opérations.
- En total :  $O((|V| + |E|) \log |V|)$ .

Calcul des chemins les plus courts dans les graphes

Présence de poids négatifs

## Présence de poids négatifs

Les valeurs  $dist(.)$  dans l'alg. de Dijkstra sont soit des valeurs exactes, soit des bornes supérieurs. La valeur  $dist(u)$  équivaut  $dist^*(u)$  à condition que tous les sommets intermédiaires du chemin  $s \rightsquigarrow u$  sont dans  $P$ . Les valeurs  $dist(.)$  peuvent être vues comme une suite des mises à jours :

---

**Algorithm 25**  $update((u, v) \in E)$ .

---

$$dist(v) = \min\{dist(v), dist(u) + l(u, v)\}$$

---

Propriétés :

- $dist(v) = dist^*(v)$  si  $dist(u) = dist^*(u)$  et si  $u$  est l'avant-dernier sommet ( $u = prev(v)$ ) sur le PCC( $s \rightsquigarrow v$ ) ;
- la procédure  $update((u, v))$  ne génère que des bornes supérieures à la valeur  $dist^*(v)$ .

Utilisation pour chercher le PCC( $s \rightsquigarrow v$ ) :

- (a)  $|PCC(s \rightsquigarrow v)| \leq |V| - 1$  ;
- (b) si  $update(.,.)$  est appliqué aux arêtes du PCC( $s \rightsquigarrow v$ )  $(s, u_1), (u_1, u_2), (u_2, u_3), \dots (u, v)$  dans cet ordre, alors  $dist(v) = dist^*(v)$ .

---

**Algorithm 26** Bellman-Ford ( $G, l, s$ ).

---

**Require:** Graphe  $G=(V,E)$  directed, edge weights  $\{l_e : e \in E\}$  with no negative cycles ;  
vertex  $s \in V$

**Ensure:** For all vertices  $u \in V$  reachable from  $s$ ,  $dist(u)$  is set to the shortest distance  
from  $s$  to  $u$  ;

```
1: for all  $u \in V$  do  
2:    $dist(u) \leftarrow \infty$   
3: end for  
4:  $dist(s) \leftarrow 0$   
5:  $k \leftarrow 1$   
6: while  $(k \leq |V| - 1)$  do  
7:   for all edges  $(u, v) \in E$  do  
8:      $dist(v) \leftarrow \min\{dist(v), dist(u) + l(u, v)\}$   
9:   end for  
10:   $k \leftarrow k + 1$   
11: end while
```

---

■ Complexité de l'algorithme (26) :  $O(|V||E|)$



## Algorithme de Bellman-Ford (variant)

Une autre approche est d'utiliser la fonction multivoque  $\Gamma^{-1}$  et l'observation que le (PCC( $s \rightsquigarrow v$ )) passe par un des prédécesseurs  $u \in \Gamma^{-1}(v)$ . On doit donc avoir

$$\text{dist}(v) = \min_{u \in \Gamma^{-1}(v)} \{ \text{dist}(u) + l(u, v) \} = \min \{ \text{dist}(v), \min_{u \in \Gamma^{-1}(v)} \{ \text{dist}(u) + l(u, v) \} \} \quad (16)$$

On obtient la variante suivante.

---

### Algorithm 27 Bellman-Ford\_bis (G,l,s).

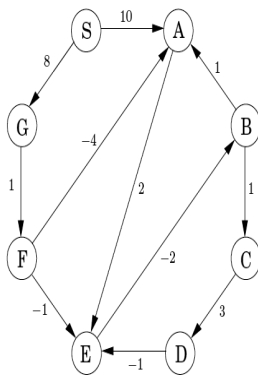
---

**Require:** Graphe  $G=(V,E)$  directed, edge weights  $\{l_e : e \in E\}$ ; vertex  $s \in V$

**Ensure:**  $\text{dist}^k(u)$  is set to the shortest distance from  $s$  to  $u$  over all paths with at most  $k$  arcs. It also detects the presence of negative cycle.

```
1: Initialisation :  $\forall u \in V : \text{dist}^0(u) \leftarrow \infty$ ;  $\text{dist}^0(s) \leftarrow 0$ ;  $\text{stable} \leftarrow \text{false}$ ;  $k \leftarrow 1$ ;
2: while (( $k \leq |V|$ ) and (nonstable)) do
3:    $\text{dist}^k(s) \leftarrow 0$ 
4:   for all vertices  $v \in V \setminus \{s\}$  do
5:      $\text{dist}^k(v) = \min \{ \text{dist}^{k-1}(v), \min_{u \in \Gamma^{-1}(v)} \text{dist}^{k-1}(u) + l(u, v) \}$ 
6:   end for
7:    $\text{stable} \leftarrow (\text{dist}^k(v) = \text{dist}^{k-1}(v), \text{ for all } v \in V)$ ;  $k \leftarrow k + 1$ ;
8: end while
9: if ( $k = |V| + 1$ ) then  $\exists$  negative cycle
10: end if
```

## Illustration pour l'algorithme de Bellman-Ford



|      | Iteration |          |          |          |          |    |    |   |
|------|-----------|----------|----------|----------|----------|----|----|---|
| Node | 0         | 1        | 2        | 3        | 4        | 5  | 6  | 7 |
| S    | 0         | 0        | 0        | 0        | 0        | 0  | 0  | 0 |
| A    | $\infty$  | 10       | 10       | 5        | 5        | 5  | 5  | 5 |
| B    | $\infty$  | $\infty$ | $\infty$ | 10       | 6        | 5  | 5  | 5 |
| C    | $\infty$  | $\infty$ | $\infty$ | $\infty$ | 11       | 7  | 6  | 6 |
| D    | $\infty$  | $\infty$ | $\infty$ | $\infty$ | $\infty$ | 14 | 10 | 9 |
| E    | $\infty$  | $\infty$ | 12       | 8        | 7        | 7  | 7  | 7 |
| F    | $\infty$  | $\infty$ | 9        | 9        | 9        | 9  | 9  | 9 |
| G    | $\infty$  | 8        | 8        | 8        | 8        | 8  | 8  | 8 |

Figure 96 –

---

**Algorithm 28** Procédure  $\text{update}((u, v) \in E)$ .

---

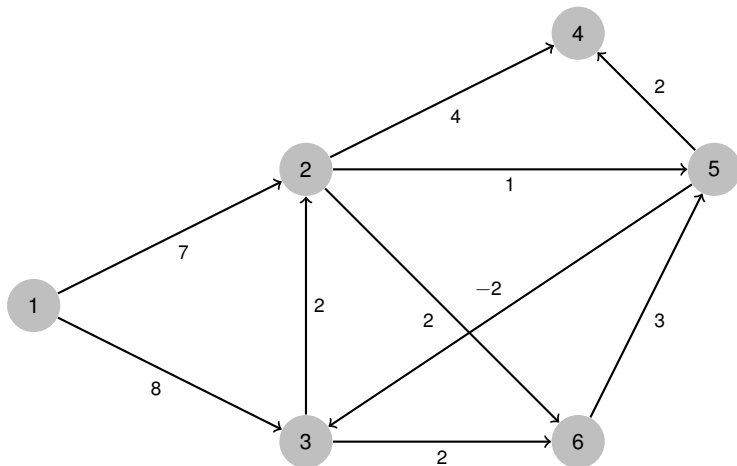
$$\text{dist}(v) = \min\{\text{dist}(v), \text{dist}(u) + l(u, v)\}$$


---

## Techniques d'accélération de l'algorithme de Bellman-Ford\_bis

- $dist^k(v)$  représente la valeur du  $PCC(s \rightsquigarrow v)$  qui ne contient pas plus de  $k$  arcs.
- Dans l'algorithme (27) on a *le choix de l'ordre* à la ligne 4 et on a intérêt à définir un ordre astucieux sur les sommets.
- Par exemple si les  $l_e$  sont positifs et l'ordre est défini par l'alg de Dijkstra, alors l'alg. (27) converge en une seule étape.
- Si peu d'arcs ont une valeur négative, on a aussi intérêt à effectuer d'abord l'alg. de Dijkstra qui donnera une bonne initialisation des valeurs  $dist(v)$  et on prendra alors comme ordre celui des  $dist(v)$  croissants.

## Accélération possible pour l'algorithme de Bellman-Ford\_bis (exemple)



# Calcul des chemins les plus courts/longs (PCC/PLC) dans les graphes

L'approche programmation dynamique

# Les principes de la programmation dynamique

Soit le graphe  $G = (V, E, l_e, \forall e \in E)$ , où  $l_e$  est la longueur de l'arc  $e$ , et soit un sommet donné  $s \in V$ .

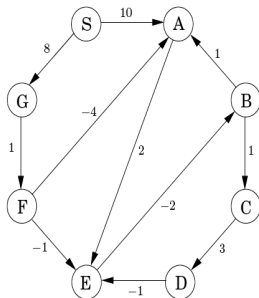
- Problème : Trouver les PCC du sommet  $s \in V$  à tous les autres  $v \in V \setminus \{s\}$ .
- Observation : Pour calculer le  $PCC(s \rightsquigarrow v)$ , dont la valeur est calculée dans la variable  $dist(v)$ , on observe que le  $PCC(s \rightsquigarrow v)$  passe par un des prédécesseurs  $u \in \Gamma^{-1}(v)$ . On doit donc avoir

$$dist(v) = \min_{u \in \Gamma^{-1}(v)} \{dist(u) + l(u, v)\} \quad (17)$$

L'équation (17) permet de calculer la valeur  $dist(v)$  sous l'hypothèse que les valeurs  $dist(u)$ ,  $\forall u \in \Gamma^{-1}(v)$  sont calculées. Cette stratégie permet de résoudre un ensemble de sous-problèmes en commençant par les plus « petits », et en allant vers les plus « grands ». Pour les « plus petits » la solution est connue.

- C'est une méthode de résolution applicable aux problèmes qui satisfont le **principe d'optimalité de Bellman (1955) : chaque sous-chemin d'un chemin optimal est lui-même optimal.**
- C'est un principe générique. La fonction min de la l'équation (17) peut être remplacée par la fonction max et l'opérateur binaire « addition » par « multiplication ».

## L'alg. de Bellman-Ford vu dans le contexte de la programmation dynamique



|      | Iteration |          |          |          |          |    |    |   |
|------|-----------|----------|----------|----------|----------|----|----|---|
| Node | 0         | 1        | 2        | 3        | 4        | 5  | 6  | 7 |
| S    | 0         | 0        | 0        | 0        | 0        | 0  | 0  | 0 |
| A    | $\infty$  | 10       | 10       | 5        | 5        | 5  | 5  | 5 |
| B    | $\infty$  | $\infty$ | $\infty$ | 10       | 6        | 5  | 5  | 5 |
| C    | $\infty$  | $\infty$ | $\infty$ | $\infty$ | 11       | 7  | 6  | 6 |
| D    | $\infty$  | $\infty$ | $\infty$ | $\infty$ | $\infty$ | 14 | 10 | 9 |
| E    | $\infty$  | $\infty$ | 12       | 8        | 7        | 7  | 7  | 7 |
| F    | $\infty$  | $\infty$ | 9        | 9        | 9        | 9  | 9  | 9 |
| G    | $\infty$  | 8        | 8        | 8        | 8        | 8  | 8  | 8 |

Notons  $dist^k(v)$  la valeur du PCC( $s \rightsquigarrow v$ ) qui contient *au plus*  $k$  arcs et réécrivons (17) sous la forme :

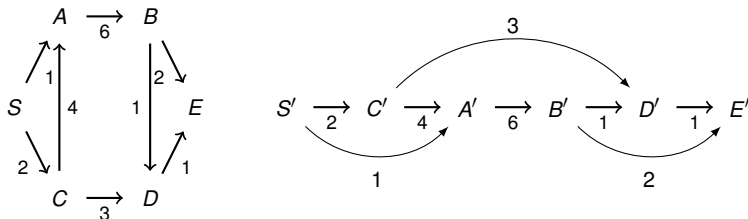
$$dist^k(v) = \min\{dist^{k-1}(v), \min_{u \in \Gamma^{-1}(v)} dist^{k-1}(u) + l(u, v)\} \quad (18)$$

La récurrence (18) permet ainsi de calculer la valeur  $dist^k(v)$  sous l'hypothèse que les valeurs  $dist^{k-1}(u), \forall u \in \Gamma^{-1}(v)$  ont été calculées. Ce qui nécessite de résoudre des sous- problèmes possédant un arc de moins par rapport au problème originel.

# Les principes de la programmation dynamique

- Trouver le PCC est particulièrement simple dans le cas d'un graphe  $G = (V, E)$  de type DAG (graphe orienté, sans circuit).
- Cela est dû à la propriété des DAG d'être linéarisés (les sommets peuvent être numérotés de façon que pour chaque arc  $(u, v) \in E$  on a  $num(u) > num(v)$ ).
- Le calcul des valeurs *dist* dans l'ordre linéaire décroissant garantit que les valeurs nécessaires pour le calcul de  $dist(v)$  sont déjà connues au moment de ce calcul.
- Cela permet de trouver les valeurs *dist* avec un seul parcours des sommets.

**Figure 97 – À gauche :** un graphe  $G$ ; **A droite :** le même graphe après l'avoir trié/ordonné topologiquement/linéairement. Un seul parcours de gauche à droite permet de calculer les valeurs *dist*.





## Algorithmes pour linéariser un DAG (graphe orienté sans circuit) (rappel)

Propriétés importantes des DAG :

- (a) Chaque DAG a au moins un sommet source (sommet sans prédécesseurs) et au moins un sommet puits (sommet sans successeurs)
- (b) Les sommets du graphe peuvent être numérotés dans l'ordre topologique (ils peuvent être placés sur une ligne de façon que tous les arcs soient orientés de gauche à droite).

---

**Algorithm 29** Premier algorithme pour linéariser un DAG.

---

**Require:**  $G=(V,E)$ , DAG

**Ensure:** List vertices in linear order.

- 1: Perform a DFS search and obtain the number  $\text{post}(v)$  for any vertex  $v \in V$ .
  - 2: List vertices in order of decreasing  $\text{post}(v)$  values.
- 

---

**Algorithm 30** Deuxième algorithme pour linéariser un DAG.

---

- 1: Find a source, output it, and delete it from the graph.
  - 2: Repeat until the graph is empty.
-

## Linéarisation d'un DAG par l'algorithme 29

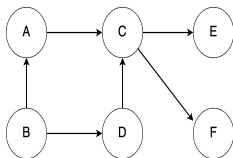


Figure 98 – Un graphe  $G = (V, E)$ .

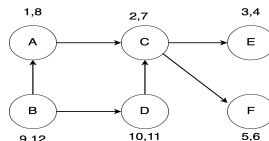


Figure 99 – La numérotation  $[pre, post]$  obtenue avec un parcours  $DFS(G, A)$ .

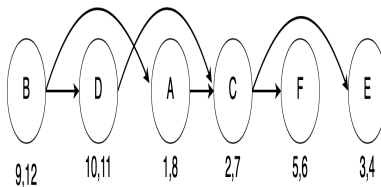


Figure 100 – Le graphe  $G$  est trié topologiquement (linéarisé) par l'ordonnancement des sommets  $v \in V$  dans l'ordre décroissant de leur numérotation  $post(v)$  calculée lors d'un parcours en profondeur  $DFS(G)$ .

## Application de l'ordre topologique/linéaire pour la détermination des chemins les plus courts/longs dans un DAG

---

**Algorithm 31** Procédure dag-shortest-paths( $G, l, s$ ).

---

**Require:** DAG  $G=(V,E)$ , edge weights  $\{l_e : e \in E\}$ , vertex  $s \in V$

**Ensure:** For all vertices  $u$  reachable from  $s$ ,  $dist(u)$  is set to the distance from  $s$  to  $u$

```
1: for all  $u \in V$  do
2:    $dist(u) \leftarrow \infty$ 
3: end for
4:  $dist(s) \leftarrow 0$ 
5: linearize  $G$ 
6: for all  $u \in V$ , in linearized order do
7:   for all edges  $(u, v) \in E$  do
8:      $dist(v) = \min\{dist(v), dist(u) + l(u, v)\}$ 
9:   end for
10: end for
```

---

- Complexité de l'algorithme (31) :  $\Theta(|V| + |E|)$  ( le coût de la linéarisation à la ligne 5 ( $\Theta(|V| + |E|)$ ) + le coût des boucles (6) et (7) ( $\sum_{u \in V} d_u^+ = \Theta(|V| + |E|)$ ).

## Application de l'ordre topologique/linéaire pour la détermination des chemins plus courts/longs dans un DAG

---

**Algorithm 32** Procédure dag-shortest-paths(BIS)(G,l,s).

---

**Require:** DAG  $G=(V,E)$ , edge weights  $\{l_e : e \in E\}$ , vertex  $s \in V$

**Ensure:** For all vertices  $u$  reachable from  $s$ ,  $dist(u)$  is set to the distance from  $s$  to  $u$

```
1: for all  $u \in V$  do
2:    $dist(u) \leftarrow \infty$ 
3: end for
4:  $dist(s) \leftarrow 0$ 
5: linearize  $G$ 
6: for all  $v \in V \setminus \{s\}$ , in linearized order do
7:   for all edges  $(u, v) \in E$  do
8:      $dist(v) := \min\{dist(v), dist(u) + l(u, v)\}$ 
9:   end for
10: end for
```

---

# Les principes de la programmation dynamique

- La récurrence (18) illustre bien l'approche programmation dynamique qui consiste à résoudre un ensemble de sous-problèmes  $\{dist(u), u \in V\}$  en commençant par les plus « petits » (les prédécesseurs) et en allant progressivement vers les plus « grands » (les successeurs).
- Cependant, le graphe DAG peut ne pas être explicitement donné, mais il est implicite.
- Les sommets représentent dans ce cas les sous-problèmes à résoudre, tandis que les arcs correspondent aux dépendances entre les problèmes ; si la solution du sous-problème B nécessite une réponse du sous-problème A, on met un arc (conceptuel) de A à B.
- A est considéré ainsi comme un sous-problème plus petit que B (un prédécesseur de B).

## La plus longue sous-séquence croissante

**Donné :** Une séquence de nombres  $S = a_1, a_2, \dots, a_n$ .

**Def\_1 :** Une sous-séquence : chaque sous-ensemble des nombres  $a_{i_1}, a_{i_2}, \dots, a_{i_k}$  tel que  $1 \leq i_1 \leq i_2 \leq \dots \leq i_k \leq n$ .

**Def\_2 :** Une sous-séquence croissante : telle que les nombres  $a_i$  croissent.

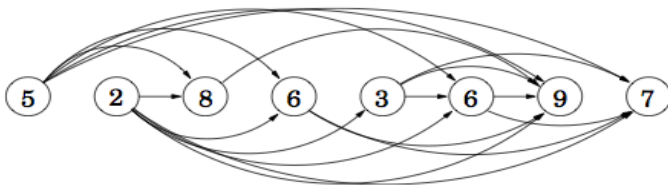
**Problème :** Trouver la plus longue sous-séquence croissante de  $S$  (PLSSC( $S$ )).

**Exemple :** La plus longue sous-séquence croissante de  $S = 5, 2, 8, 6, 3, 6, 9, 7$  est  $2, 3, 6, 9$ .

## La plus longue sous-séquence croissante

- L'espace des solutions sera présenté par un DAG  $G = (V, E)$  où à chaque nombre  $a_i$  on associera un sommet  $i \in V$  et on ajoutera un arc  $(i, j)$  si  $i < j$  et  $a_i < a_j$ .
- On établit ainsi une bijection (« *one-to-one correspondence* ») entre les chemins dans  $G$  et les sous-séquences croissantes dans  $S$ .
- L'objectif est de trouver le plus long chemin (PLC) dans ce graphe.

Exemple : Le graphe DAG associé à la séquence  $S = 5, 2, 8, 6, 3, 6, 9, 7$ .



## Algorithme pour la plus longue sous-séquence croissante

---

**Algorithm 33** Algorithme *longest increasing subsequence*.

---

```
1: for  $j = 1$  to  $n$  do  
2:    $L(j) = 1 + \max\{L(i) \mid (i, j) \in E\}$   
3: end for  
4: return  $\max_j L(j)$   
where  $\max\{\} = 0$ .
```

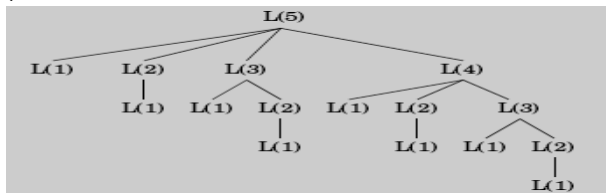
---

- L'idée de base :  $L(j)$  est la longueur de la PLSSC aboutissant au sommet  $j$  (+1) (on cherche le nombre de sommets, pas des arêtes).
- $L(j)$  est la longueur maximale parmi les PLSSC des prédécesseurs du  $j$  (+1).
- On découvre ici l'approche programmation dynamique : l'ordre et les relations entre les problèmes indiquent que pour résoudre un problème donné  $L(j)$ , il faut que ses prédécesseurs (les sous-problèmes qui le précèdent) soient résolus.
- Complexité de l'algorithme (33) : le coût de la boucle **for** de de la ligne 1 à 3 :  $\Theta(|V|)$ ; + le coût de la ligne (2)  $\sum_{j \in V} d^-(j) = \Theta(|E|)$ . En total :  $\Theta(|V| + |E|) = O(n^2)$ .
- L'algorithme (33) ne trouve que la longueur de la PLSSC. Pour connaître la séquence même, il faut sauvegarder les indices  $\operatorname{argmax}$  de la ligne (2).



## Attention aux appels récursifs naïfs!!!!

- La ligne 2 de l'algorithme (33) suggère une alternative – pourquoi ne pas utiliser un algorithme récursif ?
- En fait, c'est une mauvaise idée puisque l'algorithme récursif s'exécute en temps exponentiel.
- Considérons par exemple le cas d'une séquence croissante. Elle contient tous les arcs  $\{(i, j) \mid i < j\}$ . La formule devient  $L(j) = 1 + \max\{L(1), L(2), \dots, L(j-1)\}$ .
- Pour  $L(5)$  on obtient alors :



- Et ce sous-problème sera résolu maintefois!. Pour  $L(n)$  la taille de l'arbre sera exponentielle. Cela s'explique par les dépendances entre les problèmes  $L(j)$  et  $L(j-1)$ ; ils sont presque de la même taille.
- Pour améliorer l'efficacité il faut mémoriser les solutions des sous-problèmes résolus, et les calculer dans le bon ordre (cela évoque déjà la programmation dynamique).

## La fonction rang pour les graphes sans circuit (DAG)

- Propriétés importantes des DAG (rappel) :
  - (a) Il existe au moins un sommet  $s$  tel que  $\Gamma_s^{-1} = \emptyset$  (c.-à-d.  $s$  est sans prédécesseurs).
  - (b) Les sommets du graphe peuvent être numérotés dans l'ordre topologique (ils peuvent être placés sur une ligne de façon que tous les arcs soient orientés de gauche à droite).
- Afin de générer un ordre topologique, en plus les algorithmes 29 et 30, on peut aussi utiliser l'algorithme (34) qui calcule la fonction rang.
- Pour introduire rapidement cette notion, nous supposons que le graphe possède un seul sommet sans prédécesseurs (disons le sommet  $s$ ).
- La fonction rang associée à chaque sommet  $v \in V \setminus \{s\}$  calcule la valeur  $rank(v)$  qui correspond au nombre d'arcs dans un chemin de cardinalité maximum entre  $s$  et  $v$ .

### Obtention d'un diplôme Master d'informatique (sujet d'examen)

Pour obtenir un Master d'informatique, il est nécessaire d'avoir passé un certain nombre de modules. Chaque module demande un certain nombre de prérequis. Un master simplifié pourrait se composer (prérequis entre parenthèses) de :

- Système (Programmation) ;
  - MAG (Programmation, Math Discrètes) ;
  - Sécurité réseau (Système, Initiation réseau) ;
  - Initiation réseau (Programmation, MAG) ;
  - Systèmes répartis (MAG, Système, Initiation réseau).
1. Modéliser les prérequis à l'aide d'un graphe.
  2. Quel est le nombre minimum de semestres pour obtenir ce master ? On suppose bien sur que vous passez tous les examens avec succès, que chaque module dure un semestre et est enseigné chaque semestre. Le nombre de modules suivis par un étudiant pendant un semestre n'est pas limité et il n'y a pas de problème d'emploi du temps. Indiquer l'algorithme utilisé et justifiez votre choix.
  3. Un étudiant décide d'étudier un module dès qu'il a obtenu les modules prérequis. Quel est le nombre maximum de modules qu'il devra suivre simultanément en appliquant cette stratégie ? Quel algorithme permet de le calculer ? Justifiez votre choix.

## 1. Modéliser les prérequis à l'aide d'un graphe.

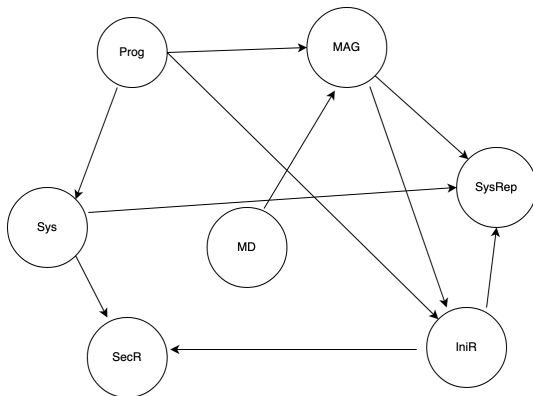


Figure 101 – Modélisation à l'aide d'un graphe : les sommets correspondent aux modules, les arcs - aux prérequis.

## Cas des graphes sans circuit (DAG) : calcul de la fonction rang

Les questions suivantes nécessitent l'utilisation de la fonction rang calculée par :

---

### Algorithm 34 Vertex-rank( $G$ ).

---

**Require:** DAG  $G = (V, E)$ , vertex  $s \in V$  such that  $d^-(s) = 0$ .

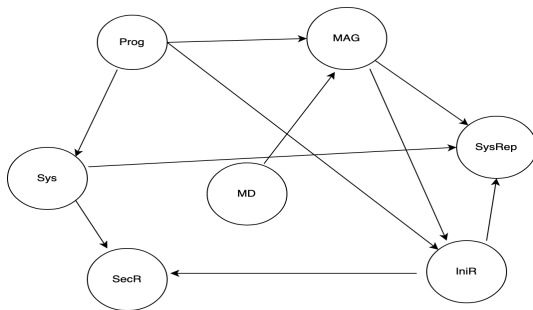
**Ensure:** For all vertices  $u$  reachable from  $s$ ,  $rank(u)$  is set to the number of arcs in the longest (in cardinality) path from  $s$  to  $u$

```
1: Initialisation :  $\forall u \in V : d^-(u) \leftarrow |\Gamma^{-1}(u)| ; k \leftarrow 0 ; S_0 \leftarrow \{s\} ;$   
2: while  $|S_k| > 0$  do ( $S_k$  est l'ensemble des sommets  $v$  tels que  $d^-(v) = 0$  ; )  
3:    $S_{k+1} \leftarrow \emptyset$   
4:   for all  $u \in S_k$  do  
5:      $rank(u) \leftarrow k$   
6:     for all edges  $(u, v) \in E$  do  
7:        $d^-(v) \leftarrow d^-(v) - 1$   
8:       if  $(d^-(v) = 0)$  then  
9:          $S_{k+1} \leftarrow S_{k+1} \cup \{v\}$   
10:      end if  
11:    end for  
12:  end for  
13:   $k \leftarrow k + 1$   
14: end while
```

---

■ Complexité de l'algorithme (34) :  $\Theta(|V| + |E|)$ .

## Calcul de la fonction rang : Illustration de l'algorithme **vertex-rank(G)**



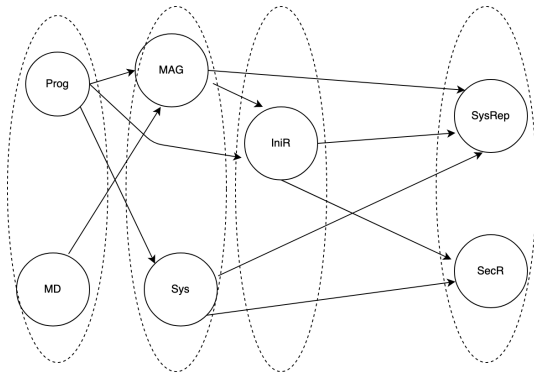
| k | $S_k$               | $d^-(Sys)$ | $d^-(SecR)$ | $d^-(Prog)$ | $d^-(MD)$ | $d^-(MAG)$ | $d^-(SysRep)$ | $d^-(IniR)$ |
|---|---------------------|------------|-------------|-------------|-----------|------------|---------------|-------------|
| 0 | <i>Prog, MD</i>     | 1          | 2           | 0           | 0         | 2          | 3             | 2           |
| 1 | <i>Sys, MAG</i>     | 0          | 2           |             |           | 0          | 3             | 1           |
| 2 | <i>IniR</i>         |            | 1           |             |           |            | 1             | 0           |
| 3 | <i>SysRep, SecR</i> |            | 0           |             |           |            | 0             |             |

**Figure 102** –  $S_k$  est l'ensemble des sommets  $v$  tels que  $d^-(v) = 0$  ; La fonction rang associe à chaque sommet  $v \in V$  le nombre  $rank(v)$  tel que  $rank(v)$  = le nombre d'arcs dans un chemin de cardinalité maximum entre les sommets sans prédécesseurs et  $v$ . On obtient ici  $rank(Prog) = rank(MD) = 0$ ,  $rank(Sys) = rank(MAG) = 1$ ,  $rank(IniR) = 2$  et  $rank(SysRep) = rank(SecR) = 3$ .

## Obtention d'un diplôme Master d'informatique : suite

Les valeurs de la fonction rang calculées précédemment ( $rank(Prog) = rank(MD) = 0$ ,  $rank(Sys) = rank(MAG) = 1$ ,  $rank(IniR) = 2$  et  $rank(SysRep) = rank(SecR) = 3$ ) permettent de visualiser le graphe par niveaux et de répondre aux questions suivantes.

1. Quel est le nombre minimum de semestres pour obtenir ce master ?



**Figure 103** – Les modules sont ordonnancés dans l'ordre croissant de leur numéro *rank*. Le graphe est représenté par niveaux, chaque niveau correspond aux sommets ayant la même valeur *rank*.

## Obtention d'un diplôme Master d'informatique : suite

1. Quel est le nombre minimum de semestres pour obtenir ce master ?

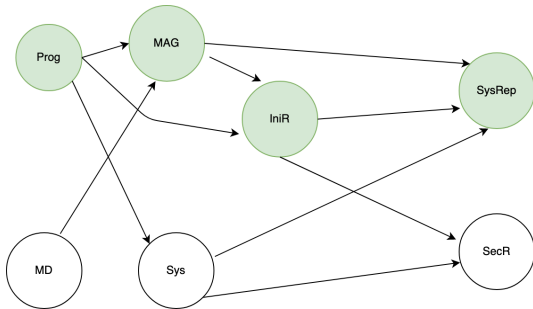


Figure 104 – Au moins 4 semestres sont nécessaires pour obtenir ce master. Cela peut se calculer par l'algorithme **Vertex-rank-longest-paths(G,l,s)** qui permet de calculer le **plus long chemin (PLC)** du premier au dernier niveau dans un graphe par niveaux.



---

**Algorithm 35** Vertex-rank-shortest-paths( $G, l, s$ ).

---

**Require:** DAG  $G = (V, E)$ , edge weights  $\{l_e : e \in E\}$ , vertex  $s \in V$ ,  $G$  is represented by vertex-rank levels

**Ensure:** For all vertices  $u$  reachable from  $s$ ,  $dist(u)$  is set to the shortest distance from  $s$  to  $u$

```
1: for all  $u \in V$  do  
2:    $dist(u) \leftarrow \infty$   
3: end for  
4:  $dist(s) \leftarrow 0$   
5: for all vertices  $v \in V \setminus \{s\}$  in rank order do  
6:    $dist(v) = \min_{u \in \Gamma^{-1}(v)} \{dist(u) + l(u, v)\}$   
7: end for
```

---

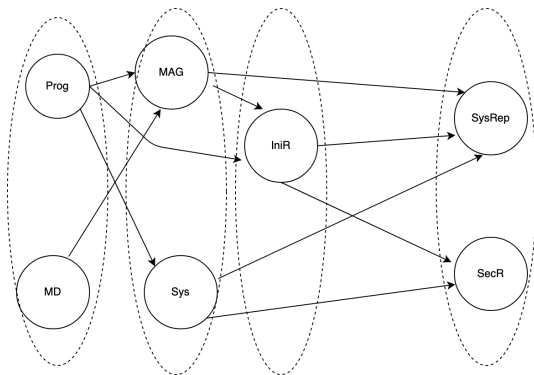
■ Complexité : la boucle des lignes (1-3) :  $\Theta(|V|)$  ; + le coût de la boucle (5-7) :  $\sum_{v \in V} d^-v = \Theta(|V| + |E|)$ . Total pour l'algorithme (35) :  $\Theta(|V| + |E|)$ .

■ l'opérateur *min* à la ligne 6 peut être remplacé par l'opérateur *max* (d'après le principe de la programmation dynamique). On modifie respectivement la ligne 2 en  $dist(u) \leftarrow -\infty$  et on obtient ainsi l'algorithme

**Vertex-rank-longest-paths( $G, l, s$ )** qui permet de calculer les PLC.

## Obtention d'un diplôme Master d'informatique : suite

1. Quel est le nombre maximum de modules que l'étudiant doit suivre simultanément en appliquant cette stratégie ?



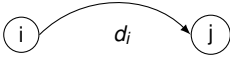
**Figure 105** – Le nombre maximum de modules qu'il devra suivre simultanément vaut 2 : c'est le nombre maximum de sommets dans un niveau (colonne) du graphe représenté par niveaux, où chaque niveau correspond aux sommets ayant la même valeur *rank*.

# Ordonnancement des tâches

**Problème :** Déterminer l'ordre et le calendrier d'exécution des  $N$  tâches, soumises à des contraintes de successions dans le temps, afin de minimiser la durée totale.

### L'approche « graphe potentiels-tâches »

À partir du projet donné, on construit un graphe sans circuit (DAG) de la façon suivante :

- A  $\forall$  tâche  $i$ , on associe un sommet  ;
- On ajoutera l'arc  $(i, j)$  de longueur  $d_i$ , noté par , si la tâche  $i$  de durée  $d_i$  doit précéder la tâche  $j$  ;
- On ajoute deux tâches fictives  $\alpha$  (début) et  $\omega$  (fin) ;
- $\alpha$  est reliée aux sommets sans prédécesseurs ;
- $\omega$  est reliée aux sommets sans successeurs.
- On représente le graphe par niveaux après avoir calculé sa fonction rang ;

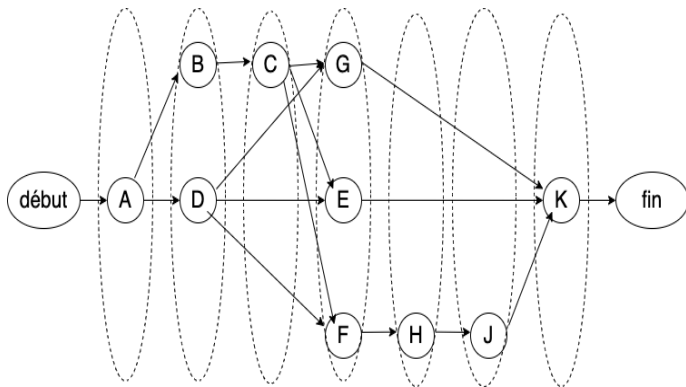
### Exemple : Construction d'un pavillon

La construction d'un pavillon demande la réalisation d'un certain nombre de tâches. La liste des tâches à effectuer, leur durée et les contraintes d'antériorités à respecter sont données dans le table ci-dessus. Le travail commençant à la date 0, on cherche un planning des opérations qui permet de minimiser la durée totale.

| Code tâche | libellé                 | durée<br>(semaines) | antériorité |
|------------|-------------------------|---------------------|-------------|
| A          | Travaux de maçonnerie   | 7                   | -           |
| B          | Charpente de la toiture | 3                   | A           |
| C          | Toiture                 | 1                   | B           |
| D          | Installation électrique | 8                   | A           |
| E          | Façade                  | 2                   | D,C         |
| F          | Fenêtres                | 1                   | D,C         |
| G          | Aménagement du jardin   | 1                   | D,C         |
| H          | Travaux de plafonnage   | 2                   | F           |
| J          | Mise en peinture        | 2                   | H           |
| K          | Emménagement            | 1                   | E,G,J       |

Figure 106 – Liste des tâches, durée et contraintes.

## Les tâches pour la construction d'un pavillon



**Figure 107** – La représentation du graphe des tâches par niveaux après avoir calculé sa fonction rang et l'ajout des deux tâches fictives début et fin.

## Ordonnement des tâches : l'approche « graphe potentiels-tâches »

### Notions principales :

- La tâche fictive  $\alpha$  (début) est reliée aux sommets sans prédécesseurs ;
- La tâche fictive  $\omega$  (fin) est reliée aux sommets sans successeurs ;
- Le graphe est représenté par niveaux après avoir calculé sa fonction rang ;
- La *date au plus tôt*  $t_i$  de début de la tâche  $i$  :  $t_i = \max_{j \in \Gamma_i^{-1}} (t_j + d_j)$

(c.-à-d. la longueur du *plus long chemin* de  $\alpha$  à  $i$  ( $PLC(\alpha \rightarrow i)$ )).

- La *durée minimale*  $t_\omega$  :  $I(PLC(\alpha \rightarrow \omega))$ .
- Si la durée du projet est fixé à  $t_\omega$ , la *date au plus tard*  $T_i$  pour commencer la tâche  $i$  sans influencer  $t_\omega$  est :

$$T_\omega = t_\omega; T_i = \min_{j \in \Gamma_i} (T_j - d_j) \Rightarrow T_i = t_\omega - I(PLC(i \rightarrow \omega))$$

- La *marge totale* ( $MT_i$ ) de la tâche  $i$  :  $MT_i = T_i - t_i$ . Pour une durée de projet fixée à  $t_\omega$  la marge totale d'une tâche correspond au retard maximale que peut prendre son début sans retarder la date  $t_\omega$ .
- Si  $MT_i = 0$ , la tâche  $i$  est dite *tâche critique*.
- La *marge libre* ( $ML_i$ ) la tâche  $i$  :  $ML_i = \min_{(i,j) \in E} (t_j - t_i - d_i)$ . Pour un calendrier

donné, la marge libre d'une tâche correspond au retard maximal que peut prendre son début sans modifier les dates de début des autres tâches.

**Remarque :** Entre deux tâches  $i$  et  $j$ , tq  $j \in \Gamma_i$ , on doit avoir  $t_j \geq t_i + d_i$ . La valeur  $t_i$ , associée à  $\forall i$ , peut être considérée comme un potentiel, d'où la dénomination *graphe potentiels-tâches*.

## Ordonnement des tâches : calcul des dates au plus tôt et au plus tard

Soit le graphe  $G=(V,E)$  orienté, sans circuit, avec les durées des tâches comme des poids sur les arcs  $\{d_e : e \in E\}$ , le sommet  $\alpha$  pour le début des tâches, le sommet  $\omega$  pour la fin. Les dates les plus tôt et les plus tard peuvent être calculées par les algorithmes suivants :

---

**Algorithm 36** Dates\_au\_plus\_tôt ( $G, d, \alpha, \omega$ ).

---

**Ensure:** Les dates au plus tôt  $t_i, \forall i$ ;

$t_\alpha \leftarrow 0; d_\alpha \leftarrow 0;$

**for all**  $i \in V$  **do** dans l'ordre croissant des rangs

$t_i := \max_{(j,i) \in E} (t_j + d_i)$

**end for**

---

---

**Algorithm 37** Dates\_au\_plus\_tard ( $G, d, \alpha, \omega$ ).

---

**Require:** que  $t_\omega$  soit calculé ;

**Ensure:** Les dates au plus tard pour calculer  $T_i, \forall i$  sans impacter  $t_\omega$  ;

$T_\omega \leftarrow t_\omega;$

**for all**  $i \in V$  **do** dans l'ordre décroissant des rangs

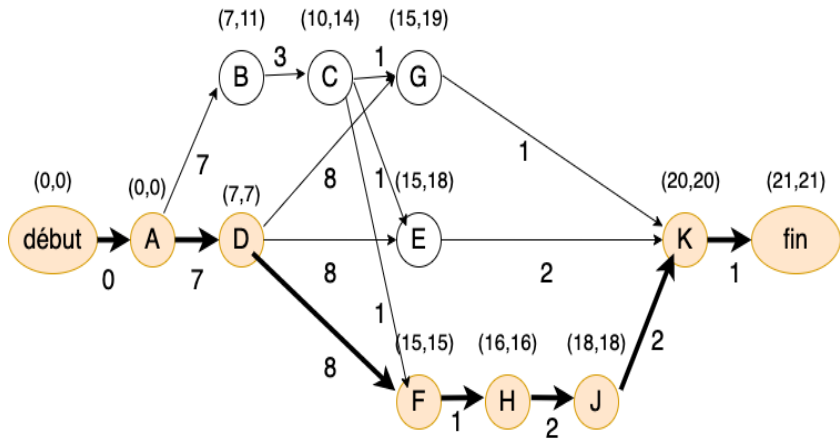
$T_i \leftarrow \min_{(i,j) \in E} (T_j) - d_i$

**end for**

---



## L'ordonnancement des tâches pour la construction d'un pavillon



**Figure 108** – L'étiquette  $(t_i, T_i)$  associé au sommet  $(i)$  représente les dates au plus tôt et au plus tard. Les dates au plus tôt ( $t_i$ ) sont calculées par l'algorithme (36), tandis que les dates au plus tard ( $T_i$ ) sont calculées par l'algorithme (37). Les tâches  $i$  dont les marges  $MT_i = T_i - t_i$  sont nulles sont appelées des tâches critiques. Les tâches critiques et les arcs du chemin critique obtenu pour la construction de ce pavillon sont soulignés. Si un quelconque retard est pris sur une de ces tâches la durée minimale de la construction de la villa sera augmentée d'autant.

## L'ordonnement des tâches pour la construction d'un pavillon

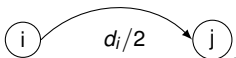
| Code tâche | date au plus tôt $t_i$ | date au plus tard $T_i$ | marge totale $MT_i$ | marge libre $ML_i$ |
|------------|------------------------|-------------------------|---------------------|--------------------|
| A          | 0                      | 0                       | 0                   | 0                  |
| B          | 7                      | 11                      | 4                   | 0                  |
| C          | 10                     | 14                      | 4                   | 4                  |
| D          | 7                      | 7                       | 0                   | 0                  |
| G          | 15                     | 19                      | 4                   | 4                  |
| E          | 15                     | 18                      | 3                   | 3                  |
| F          | 15                     | 15                      | 0                   | 0                  |
| H          | 16                     | 16                      | 0                   | 0                  |
| J          | 18                     | 18                      | 0                   | 0                  |
| K          | 20                     | 20                      | 0                   | 0                  |
| fin        | 21                     | 21                      | 0                   | 0                  |

Figure 109 – Caractéristiques de l'ordonnement des tâches. Les tâches peuvent être classées en trois catégories : a) tâches pour lesquelles le moindre retard de démarrage allonge le délai de réalisation du projet. Ce sont les tâches critiques ( $MT_i = 0$  - ici {A, D, F, H, J, K} ) ; b) tâches pour lesquelles le moindre retard de démarrage perturbe le calendrier ( $ML_i = 0$  - ici {B} ) ; c) tâches pour lesquelles un léger retard ne modifie pas le planning ( $ML_i \neq 0$  - ici {C, G, E} ) ;

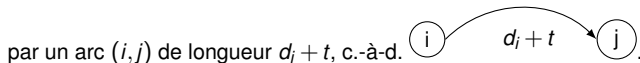
## Compléments pratiques

Les contraintes de type potentiel (la tâche  $j$  doit commencer après la fin de  $i$ , ou bien après la moitié de la réalisation de  $i$ , ou bien un certain temps après la fin de  $i$ , etc.) se modélisent facilement dans l'approche « graphe potentiels-tâches ».

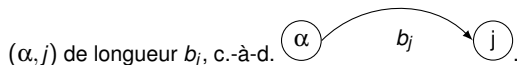
- la contrainte :  $j$  ne doit pas commencer avant la moitié de la réalisation de la tâche  $i$  se représente par un arc supplémentaire  $(i, j)$  de longueur  $\frac{d_i}{2}$ , c.-à-d.



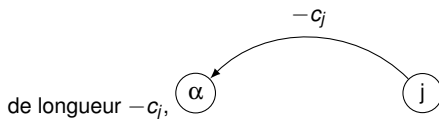
- la contrainte :  $j$  ne peut commencer qu'un temps  $t$  après la fin de  $i$  se représente



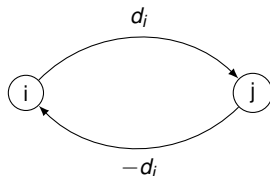
- la contrainte :  $j$  ne peut commencer qu'après la date  $b_i$ , se représente par un arc



- la contrainte :  $j$  doit commencer avant la date  $c_j$ , se représente par un arc  $(j, \alpha)$



- la contrainte :  $j$  doit suivre immédiatement  $i$ , s'écrit  $t_i + d_i = t_j$  et se représente par un arc  $(i, j)$  de longueur  $d_i$  et un arc  $(j, i)$  de longueur  $-d_i$



**Remarque :** Les deux dernières contraintes introduisent des circuits et des arcs de longueurs négatives. La condition d'existence d'un ordonnancement deviendra alors *il n'existe pas de circuit de longueur strictement positive* (au lieu de *il n'existe pas de circuit*). On devra utiliser un des algorithmes vus en cours plus adaptés à de telles situations que la programmation dynamique. On veillera aussi à remplacer chaque fois l'opérateur min par max (puisque l'on cherche des plus longs chemins).

# Chemins, flots et programmes linéaires dans les graphes

## Programmes Linéaires (PL) versus PL en Nombres Entiers (PLNE)

**Exemple de PL :** Un artisan chocolatier décide de confectionner des œufs en chocolat. En allant inspecter ses réserves, il constate qu'il lui reste 18 kg de cacao, 8 kg de noisettes et 14 kg de lait. Il a deux spécialités : l'œuf Extra et l'œuf Sublime. Un œuf Extra nécessite 1 kg de cacao, 1 kg de noisettes et 2 kg de lait. Un œuf Sublime nécessite 3 kg de cacao, 1 kg de noisettes et 1 kg de lait. Il fera un profit de 20 € en vendant un œuf Extra, et de 30 € en vendant un œuf Sublime. Combien d'œufs Extra et Sublime doit-il fabriquer pour faire le plus grand bénéfice possible ?

**Modélisation :** Consiste en trois éléments :

- Définir les variables du problème et leur type informatique ;
- Donner la fonction objectif (fonction économique) ;
- Donner les contraintes du problème.

|      |             |   |         |           |                                         |
|------|-------------|---|---------|-----------|-----------------------------------------|
| max  | $z = 20x_1$ | + | $30x_2$ |           | Objectif                                |
| c1 : | $x_1$       | + | $3x_2$  | $\leq 18$ | c1 : disponibilité du cacao             |
| c2 : | $x_1$       | + | $x_2$   | $\leq 8$  | c2 : disponibilité des noisettes        |
| c3 : | $2x_1$      | + | $x_2$   | $\leq 14$ | c3 : disponibilité du lait              |
| c4 : | $x_1$       |   |         | $\geq 0$  | $x_1$ : la quantité des œufs Extra ;    |
| c5 : |             |   | $x_2$   | $\geq 0$  | $x_2$ : la quantité des œufs Sublimes ; |

**Solution :** 3 œufs Extra, 5 œufs Sublime, bénéfice 210 €.

Si on avait 18,5 kg de cacao à la place de 18 kg on aurait obtenu la solution 2,75 Extra et 5,25 Sublime avec un bénéfice=212,5 €. Ce qui n'est pas admissible !. Pour rendre le problème admissible on ajoute la contrainte :  $x_1, x_2$ , **entier** ; ( **un PLNE** ).

L'intérêt de la Programmation Linéaire (PL) réside dans ce qui suit :

- Les programmes linéaires modélisent convenablement un grand nombre de situations réelles.
- L'existence des solveurs efficaces pour la résolution (**IBM ILOG CPLEX**, **Gurobi**, **COIN**, **MINOS**, **GLPK** etc.).
- L'existence des langages de modélisation comme **AMPL** (A Mathematical Programming Language) et **Pyomo** (Python-based, open-source optimization modeling language), **JuMP** (Julia for Mathematical Optimization) ; **PuLP** : A Linear Programming Toolkit for Python.

La programmation linéaire n'est pas le seul moyen d'aborder un problème d'optimisation. Il y a deux autres grands paradigmes, la **programmation par contraintes (PC)** et les modèles **SAT**. Chaque méthode est différente et possède ses forces et faiblesses.

## Un PL sous forme matricielle

$$\begin{array}{ll}\text{Maximiser} & \sum_{j=1}^n c_j x_j \\ \text{s. c.} & \sum_{j=1}^n a_{ij} x_j \leq b_i, \quad i = 1, \dots, m \\ & x_j \geq 0 \quad j = 1, \dots, n\end{array}$$

ou

$$\max\{cx \mid Ax \leq b, x \geq 0\}$$

où

$$A = \begin{pmatrix} a_{11} & \dots & a_{1n} \\ \vdots & & \vdots \\ a_{m1} & \dots & a_{mn} \end{pmatrix}, \quad x = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix}, \quad b = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix},$$

$$c = (c_1 \dots c_n)$$

$c$  désigne le vecteur **objectif**,

$b$  – le vecteur **membre droit**,

et  $A^{m \times n}$  – la **matrice des contraintes**.



La programmation linéaire est dans les fondements de la **recherche opérationnelle** (RO) ou **aide à la décision** : propose des modèles conceptuels pour analyser des situations complexes et permet aux décideurs de faire les **choix les plus efficaces**.

|         |                                                 |                                                                                                     |
|---------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| 1940    | P. Blackett<br>prix Nobel de<br>physique (1948) | dirige la 1 <sup>e</sup> équipe de RO :<br>implantation optimale<br>de radars de surveillance       |
| 1939-45 | L. Kantorovich                                  | programmation linéaire                                                                              |
| 1947    | G. Dantzig<br>(le fondateur)                    | algorithme du simplexe<br>" one of <b>the top 10 algorithms</b> of the<br>century", CSE, 2 :1, 2000 |
| 2005    | décès de Dantzig                                |                                                                                                     |

Aujourd'hui : développement considérable grâce aux solveurs très performants (Gurobi, CPLEX) et langages de modélisation AMPL, JuMP, Pyomo, PuLP.

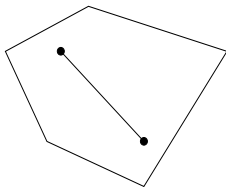
En France : Société Française de Recherche Opérationnelle et d'Aide à la Décision (ROADEF) (<http://www.roadef.org/>).

## Espace des solutions admissibles

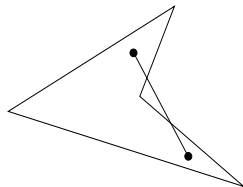
Soit un PL :  $\max\{cx \mid Ax \leq b, x \geq 0\}$

L'espace des solutions *admissibles/réalisables/faisables* défini par l'intersection d'un nombre fini de contraintes linéaires est un polyèdre convexe  $P = \{x \mid Ax \leq b, x \geq 0\} \subseteq \mathbb{R}^n$ .

( $P$  est convexe si  $x \in P, y \in P$  et  $0 \leq \lambda \leq 1$  alors  $\lambda x + (1 - \lambda)y \in P$ ).



convexe



non convexe

Un vecteur  $x \in S \subseteq \mathbb{R}^n$  est un point *extrême* (*sommet de  $S$* ) si on ne peut pas l'exprimer comme une combinaison convexe de deux autres points de  $S$   
( $\nexists (y \in S, z \in S, \alpha, 0 < \alpha < 1)$  tq  $x = \alpha y + (1 - \alpha)z$ ).

**Résultat principal : L'optimum, s'il existe, est atteint en au moins un sommet du polyèdre  $P$ .**

## Points extrêmes du $P \subset \mathbb{R}^n$

Il y a un nombre fini de façons de choisir  $n$  parmi les  $m + n$  contraintes.

$$\binom{n+m}{n} = \frac{(n+m)!}{m!n!}$$

Ceci donne le nombre des points extrêmes de  $P$  qui est une borne supérieure du nombre de solutions réalisables.

Exemple du problème du chocolatier :

$$\max z = 20x_1 + 30x_2$$

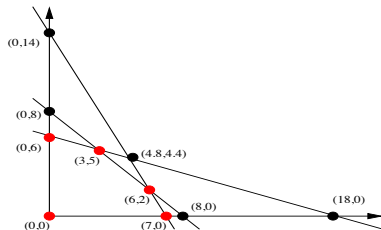
$$x_1 + 3x_2 \leq 18$$

$$x_1 + x_2 \leq 8$$

$$2x_1 + x_2 \leq 14$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$



*Approche naïve* : On est sûr de trouver la solution optimale dans un sommet. Donc il suffit de parcourir tous les sommets et de prendre le meilleur.

*Amélioration dans l'algorithme du simplexe* : Parcourir les sommets de  $P$  de façon plus intelligente (on verra cela ultérieurement).

## L'approche graphique pour les programmes linéaires

**Rappel :** Le gradient d'une fonction  $f(x_1, x_2, \dots, x_n)$  est donnée par :

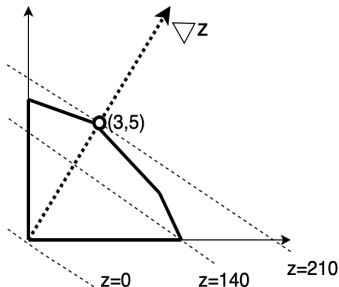
$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}. \text{ Si } f \text{ est linéaire } f(x) = \sum_{i=1}^n c_i x_i : \nabla f(x) = \begin{pmatrix} c_1 \\ \vdots \\ c_n \end{pmatrix}$$

Le gradient donne la direction d'acroissance de la fonction  $f$ . L'approche graphique :

- On se déplace selon la direction d'acroissance (de montée) ;
- L'optimum est atteint en un point extrême du domaine.

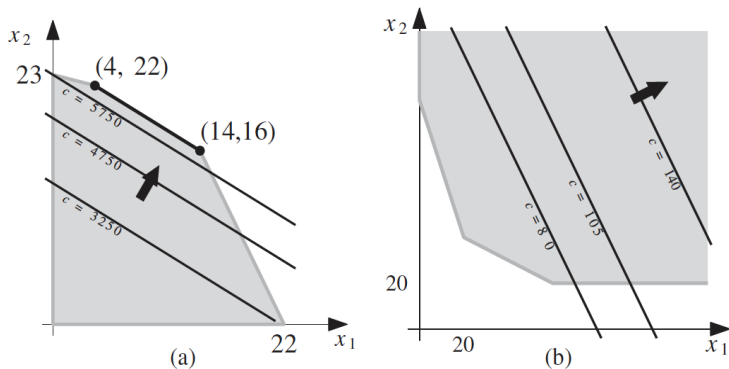
$$\max z = 20x_1 + 30x_2$$

$$\begin{array}{rclcl} x_1 & + & 3x_2 & \leq & 18 \\ x_1 & + & x_2 & \leq & 8 \\ 2x_1 & + & x_2 & \leq & 14 \\ x_1 & & & \geq & 0 \\ & & x_2 & \geq & 0 \end{array} \quad \nabla z = \begin{pmatrix} 20 \\ 30 \end{pmatrix}$$



**Figure 110** – Les lignes de niveau de la fonction objectif sont visualisées par des lignes droites perpendiculaires au vecteur de gradient  $\nabla z$  (indiquées sur la figure en pointillés). La solution pour ce problème est :  $x^* = (x_1^*, x_2^*) = (3, 5)$  et  $z^* = z(x_1^*, x_2^*) = z(3, 5) = 210$ .

# L'approche graphique pour les programmes linéaires



**Figure 111** – Un programme linéaire peut avoir : (a) plusieurs points d'optimum ou ; (b) ne pas avoir d'optimum. Dans ce deuxième cas la fonction objectif n'est pas bornée.

## L'approche graphique pour les programmes linéaires

Résoudre les programmes linéaires ci-dessous par l'approche graphique.

Exercice 1 :

$$\begin{array}{llllll} \text{Maximiser } z = & 2x_1 & + & x_2 & & \\ \text{s. c.} & 4x_1 & + & 2x_2 & \leq & 8 \\ & & & x_2 & \leq & 2 \\ & & & x_1, x_2 & \geq & 0 \end{array}$$

Exercice 2 :

$$\begin{array}{llllll} \text{Maximiser } z = & x_1 & + & 2x_2 & & \\ \text{s. c.} & -x_1 & + & x_2 & \leq & 2 \\ & & & x_2 & \leq & 3 \\ & & & x_1, x_2 & \geq & 0 \end{array}$$

Exercice 3 :

$$\begin{array}{llllll} \text{Maximiser } z = & 5x_1 & + & 8x_2 & & \\ \text{s. c.} & x_1 & + & x_2 & \leq & 2 \\ & x_1 & - & 2x_2 & \leq & 0 \\ & -x_1 & + & 4x_2 & \leq & 1 \\ & & & x_1, x_2 & \geq & 0 \end{array}$$

## L'algorithme du simplexe : idée géométrique

*Idée* : Parcourir les sommets du polyèdre  $P$  (l'espace des solutions admissibles) de façon intelligente.

Passer itérativement d'un sommet à un sommet adjacent de façon à augmenter la valeur de la fonction à optimiser jusqu'à trouver un sommet où le maximum est atteint.

- *Initialisation* : Choisir un sommet  $x^0 \in P$ ,  $t = 1$
- *Itération  $t$*  : Soit  $y^1, \dots, y^k$  tous les sommets voisins de  $x^t$  (les sommets reliés avec  $x^t$  par une arête).
  - Si  $cx^t > cy^s$ ,  $s = 1, \dots, k$ , stop. La solution optimale est  $x^t$ .
  - Sinon, choisir un voisin  $y^s$ , tel que  $cy^s \geq cx^t$ . Poser  $x^{t+1} = y^s$  et passer à l'itération  $t + 1$ .

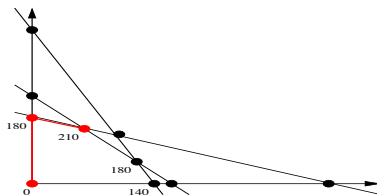
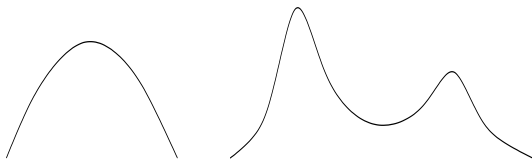


Figure 112 – Deux itérations suffisent dans ce cas pour trouver l'optimum.

## Simplexe – pourquoi ça marche ?

Grâce à la convexité du polyèdre des contraintes et à la linéarité de la fonction objectif, il n'y a pas de *maxima locaux*.



montagne convexe

montagne non-convexe

On se déplace d'un sommet à un autre en augmentant la valeur de la fonction  $z$ . Le nombre de sommets est fini (au plus  $\binom{m+n}{n}$ ).

- Si la fonction objectif est bornée, on va finir par trouver le maximum
- Si elle ne est pas bornée, on va le détecter.

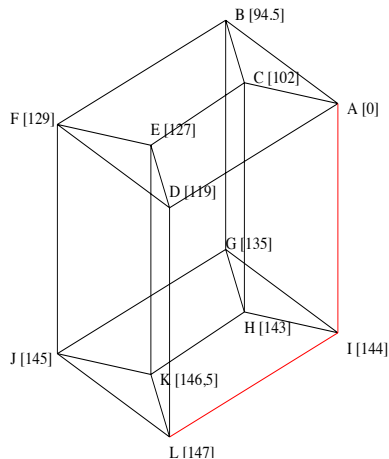


## Exemple

|             | product1 | product 2 | product 3 | product 4 | stock |
|-------------|----------|-----------|-----------|-----------|-------|
| ressource A | 2        | 4         | 5         | 7         | 42    |
| ressource B | 1        | 1         | 2         | 2         | 17    |
| ressource C | 1        | 2         | 3         | 3         | 24    |
| benefice    | 7        | 9         | 18        | 17        |       |

$$\begin{aligned}
 \text{Maximize } z = & 7x_1 + 9x_2 + 18x_3 + 17x_4 \\
 \text{s. c.} & 2x_1 + 4x_2 + 5x_3 + 7x_4 \leq 42 \\
 & x_1 + x_2 + 2x_3 + 2x_4 \leq 17 \\
 & x_1 + 2x_2 + 3x_3 + 3x_4 \leq 24 \\
 & x_1, x_2, x_3, x_4 \geq 0
 \end{aligned}$$

## Interprétation géométrique du parcours effectué par l'algorithme de simplexe



| vertex | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $z$   |
|--------|-------|-------|-------|-------|-------|
| A      | 0     | 0     | 0     | 0     | 0     |
| B      | 0     | 10.5  | 0     | 0     | 94.5  |
| C      | 0     | 0     | 0     | 6     | 102   |
| D      | 17    | 0     | 0     | 0     | 119   |
| E      | 11.67 | 0     | 0     | 2.67  | 127   |
| F      | 13    | 4     | 0     | 0     | 129   |
| G      | 0     | 3     | 6     | 0     | 135   |
| H      | 0     | 0     | 7     | 1     | 143   |
| I      | 0     | 0     | 8     | 0     | 144   |
| J      | 4     | 1     | 6     | 0     | 145   |
| K      | 3     | 0     | 6.5   | 0.5   | 146.5 |
| L      | 3     | 0     | 7     | 0     | 147   |

**Figure 113** – Les points extrêmes du domaine (il y en a douze) et les valeurs de la fonction objectif correspondantes. Le parcours de l'alg de simplexe est visualisé en rouge. Deux itérations suffisent pour trouver l'optimum.

## Convergence et complexité de l'algorithme de simplexe

On se déplace d'un sommet du polyèdre à un autre en améliorant à chaque déplacement la valeur de la fonction objectif  $z$ . Le nombre de sommets est majoré par  $\binom{n}{m}$ .

- Si la fonction objectif est bornée, l'optimum sera atteint.
- Sinon, l'algorithme détecte que la fonction objectif n'est pas bornée.

Complexité :

- En 1972 Klee et Minty ont démontré que la complexité est exponentielle dans le pire cas.
- Cependant, il a été montré que dans le cas moeyn l'algorithme se comporte polynomialement ( $O(\min\{(m-n)^2, n^2\})$ ).
- En 1979 le mathématicien Soviétique Leonid Khachiyan a proposé un algorithme polynomial pour la porgrammation linéaire (la méthode de l'ellipsoïde). Cependant, il s'est avéré trop lent pour des applications réelles.
- En 1984 Narendra Karmarkar a proposé un autre algorithme polynomial de la programmation inéaire (la méthode du point intérieur). Cet algorithme est considéré compétitif à l'algorithme de simplexe.

## Application 1 : Le problème du plus court chemin (PCC) dans un graphe vu comme un PL

---

### Algorithm 38 Bellman-Ford ( $G, l, s$ ) (Rappel)

---

**Require:**  $G=(V,E)$  directed, weights  $\{l_e : e \in E\}$  with no negative cycles ; vertex  $s \in V$

**Ensure:**  $\forall u \in V$  reachable from  $s$ ,  $\text{dist}(u)$  is set to the shortest distance from  $s$  to  $u$  ;

- 1:  $\text{dist}(s) \leftarrow 0, \quad u \in V \setminus \{s\} : \text{dist}(u) \leftarrow \infty$
  - 2: **for all** edges  $(u, v) \in E$  **do**
  - 3:    $\text{dist}(v) \leftarrow \min\{\text{dist}(v), \text{dist}(u) + l(u, v)\}$
  - 4: **end for**
- 

**Approche PL :** Soit le graphe  $G = (V, E, l)$  et un sommet  $s \in V$ . On cherche le PCC du sommet  $s$  au sommet  $t$ . Pour chaque sommet  $v \in V$  on introduit la variable  $\text{dist}_v \geq 0$  : la distance la plus petite de  $s$  à  $v$ .

Une première formulation du PCC est donnée par le PL suivant :

$$\min \text{dist}_t \tag{19}$$

$$\text{dist}_s = 0 \tag{20}$$

$$\text{dist}_v - \text{dist}_u \leq l(u, v), \quad \forall (u, v) \in E \tag{21}$$

La relation précédente est une expression du critère d'arrêt de l'algorithme de Bellman-Ford. Cette relation nous invite à considérer le vecteur  $\text{dist}$  comme un *potentiel*.

## Le problème du PCC dans un graphe vu comme un PL : formulation bis

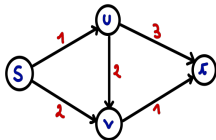


Figure 114 –  $G = (V, E, l)$ , les valeurs sur les arcs sont les longueurs  $l_e, e \in E$ .

$$A = \begin{array}{c|ccccc} & e_{su} & e_{sv} & e_{uv} & e_{ut} & e_{vt} \\ \hline s & +1 & +1 & 0 & 0 & 0 \\ t & 0 & 0 & 0 & -1 & -1 \\ u & -1 & 0 & +1 & +1 & 0 \\ v & 0 & -1 & -1 & 0 & +1 \end{array}$$

Figure 115 – La matrice d'incidence sommet-arc du  $G$ .

Chaque chemin de  $s$  à  $t$  peut être représenté par un vecteur  $\mathbf{x}$  tel que  $x_e = 1$  si l'arc  $e$  appartient au chemin,  $x_e = 0$  sinon ; et les conditions suivantes :

- le sommet  $s$  est le début du chemin ;

$$x_{su} + x_{sv} = 1 \quad (22)$$

- le sommet  $t$  est la fin du chemin ;

$$x_{ut} + x_{vt} = 1 \Leftrightarrow -x_{ut} - x_{vt} = -1 \quad (23)$$

- la loi de conservation pour le sommet intermédiaire  $u$  :

$$x_{su} = x_{uv} + x_{ut} \Leftrightarrow -x_{su} + x_{uv} + x_{ut} = 0 \quad (24)$$

- la loi de conservation pour le sommet intermédiaire  $v$  :

$$x_{sv} + x_{uv} = x_{vt} \Leftrightarrow -x_{sv} - x_{uv} + x_{vt} = 0 \quad (25)$$

## Formulation matricielle du problème PCC

$$\min z = x_{su} + 2x_{sv} + 2x_{uv} + 3x_{ut} + x_{vt} \quad (27)$$

$$\begin{array}{lcl}
 s: & x_{su} & +x_{sv} & & & = 1 \\
 t: & & & -x_{ut} & -x_{vt} & = -1 \\
 u: & -x_{su} & & +x_{uv} & +x_{ut} & = 0 \\
 v: & & -x_{sv} & -x_{uv} & & +x_{vt} = 0
 \end{array}
 \quad A = \begin{array}{c|ccccc}
 & x_{su} & x_{sv} & x_{uv} & x_{ut} & x_{vt} \\
 \hline
 s & +1 & +1 & 0 & 0 & 0 \\
 t & 0 & 0 & 0 & -1 & -1 \\
 u & -1 & 0 & +1 & +1 & 0 \\
 v & 0 & -1 & -1 & 0 & +1
 \end{array}$$

- Soit  $a_i$  la ligne  $i$  de la matrice  $A$ . La loi de conservation pour chaque sommet intermédiaire  $i$  :

$$a_i \mathbf{x} = 0 \quad (28)$$

- La formulation matricielle du programme linéaire du PCC :

notons  $d \in \mathbb{R}^n$  le vecteur

$$d_i = \begin{cases} +1 & i = s \\ -1 & i = t \\ 0 & \text{otherwise} \end{cases}$$

$$\min \sum_{e \in E} x_e w_e \quad (29)$$

$$A\mathbf{x} = d \quad (30)$$

$$\forall e \in E, \quad x_e \in \{0, 1\} \quad (31)$$

## Application 2 : Le problème du flot maximum comme un PL

- Soit  $R = (V, E, C)$  un graphe connexe orienté (réseau). A  $\forall$  arc  $u$  on affecte une valeur (*capacité*)  $c_u$  qui est une borne supérieure du flux sur l'arc.
- Soit deux sommets particuliers  $s \in V$  (source) et  $t \in V$  (puits). Considérons le graphe  $G^0 = (V, E^0)$  où  $E^0 = E \cup \{t, s\}$ . L'arc  $(t, s)$  est appelé l'*arc de retour* du flot (numéroté 0). Notons  $m = |E|$ .
- On dit que  $[\phi_1, \phi_2, \dots, \phi_m]^T$  est un *flot de  $s$  à  $t$*  ssi la loi de conservation du flot est vraie en tout  $\forall u \in V \setminus \{s, t\}$ , i.e.

$$\sum_{(u,v) \in E} \phi_{(u,v)} = \sum_{(v,u) \in E} \phi_{(v,u)} \quad (32)$$

- La valeur du flot est notée par  $\phi_0$ . Elle est définie par

$$\sum_{(s,v) \in E} \phi_{(s,v)} = \sum_{(u,t) \in E} \phi_{(u,t)} = \phi_0 \text{ (valeur du flot)} \quad (33)$$

- **But :** Trouver dans  $G^0$  un *flot compatible*  $\phi' = [\phi_0, \phi_1, \phi_2, \dots, \phi_m]$  (c.-à-d.

$$0 \leq \phi_u \leq c_u, \forall u \in E \quad (34)$$

et tel que  $\phi_0$  soit *maximale*.

## Le problème du flot maximum comme un PL

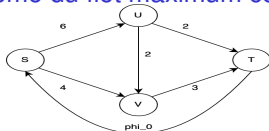


Figure 116 – Un graphe  $G = (V, E, c)$  avec des capacités  $c_e, e \in E$ .

$$A = \begin{array}{c|ccccc} & e_{su} & e_{sv} & e_{uv} & e_{ut} & e_{vt} \\ \hline s & +1 & +1 & 0 & 0 & 0 \\ t & 0 & 0 & 0 & -1 & -1 \\ u & -1 & 0 & +1 & +1 & 0 \\ v & 0 & -1 & -1 & 0 & +1 \end{array}$$

Figure 117 – La matrice d'incidence sommet-arc du  $G$ .

Le flot est représenté par un vecteur  $[\phi_1, \phi_2, \dots, \phi_m]^T$  tel que  $0 \leq \phi_u \leq c_u, \forall u \in E$  et qui satisfait les conditions suivantes :

- un flot de volume  $\phi_0$  sort du sommet  $s$ ;

$$\phi_{su} + \phi_{sv} = \phi_0 \quad (35)$$

- un flot de volume  $\phi_0$  entre dans sommet  $t$ ;

$$\phi_{ut} + \phi_{vt} = \phi_0 \Leftrightarrow -\phi_{ut} - \phi_{vt} = -\phi_0 \quad (36)$$

- la loi de conservation pour le sommet intermédiaire  $u$  :

$$\phi_{su} = \phi_{uv} + \phi_{ut} \Leftrightarrow -\phi_{su} + \phi_{uv} + \phi_{ut} = 0 \quad (37)$$

- la loi de conservation pour le sommet intermédiaire  $v$  :

$$\phi_{sv} + \phi_{uv} = \phi_{vt} \Leftrightarrow -\phi_{sv} - \phi_{uv} + \phi_{vt} = 0 \quad (38)$$

- le but est de maximiser le flot total  $\phi_0$  :

$$\max \phi_0 \quad (39)$$



## La forme matricielle du problème du flot maximum (MF)

$$\max \phi_0 \quad (40)$$

$$\begin{array}{lcl}
 s: & \phi_{su} & + \phi_{sv} & = 1 \\
 t: & & & -\phi_{ut} & -\phi_{vt} & = -1 \\
 u: & -\phi_{su} & & +\phi_{uv} & +\phi_{ut} & = 0 \\
 v: & & -\phi_{sv} & -\phi_{uv} & & +\phi_{vt} & = 0
 \end{array}
 \quad A = \begin{array}{c|ccccc}
 & e_{su} & e_{sv} & e_{uv} & e_{ut} & e_{vt} \\
 \hline
 s & +1 & +1 & 0 & 0 & 0 \\
 t & 0 & 0 & 0 & -1 & -1 \\
 u & -1 & 0 & +1 & +1 & 0 \\
 v & 0 & -1 & -1 & 0 & +1
 \end{array}$$

- Soit  $a_i$  la ligne  $i$  de la matrice  $A$ . La loi de conservation pour chaque sommet intermédiaire  $i$  :

$$a_i \phi = 0 \quad (41)$$

- La forme matricielle du problème du flot maximum (MF) :

notons  $d \in R^n$  le vecteur

$$d_i = \begin{cases} -1 & i = s \\ +1 & i = t \\ 0 & \text{otherwise} \end{cases}$$

$$\max \phi_0 \quad (42)$$

$$d\phi_0 + A\phi = 0 \quad (43)$$

$$\phi \leq c \quad (44)$$

$$\phi \geq 0 \quad (45)$$

$$\phi_0 \geq 0 \quad (46)$$

## Application 3 : Le problème de mariage (couplage)

Soit  $G = (V, E)$  un graphe. Définitions :

- Un *couplage*  $M$  est un ensemble d'arêtes deux à deux non adjacentes.
- Une *couverture*  $C$  est un ensemble de sommets tel que chaque arête est incidente avec au moins un sommet de  $C$ .
- Un graphe est dit *biparti* si son ensemble de sommets peut être divisé en deux sous-ensembles disjoints  $U$  et  $V$  tels que chaque arête ait une extrémité dans  $U$  et l'autre dans  $V$ .

Exemple :

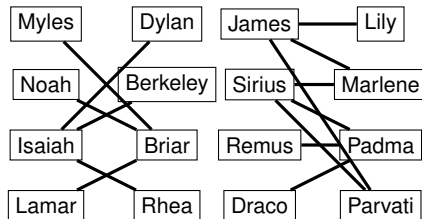


Figure 118 – Groupe 1    Figure 119 – Groupe 2

**Figure 120** – Les affinités entre les membres de deux groupes peuvent être visualisées par des graphes bipartis. Tentons à marier ces personnes en satisfaisant les contraintes des couples compatibles.

- Donner le programme linéaire pour le problème de couplage
- Montrer que ce problème se réduit au problème du flot maximum.

## Le problème de couplage maximum : modèle linéaire direct

Soit un graphe biparti  $G(U \cup V, E)$  et soit  $n = |V|$ ,  $k = |U|$  et  $m = |E|$ . Le couplage sera représenté par des variables binaires  $x_e$ ,  $\forall e \in E$ , telles que  $x_e = 1$  si l'arête  $e$  fait partie du couplage,  $x_e = 0$  sinon, et sous les conditions suivantes :

- ensemble d'arêtes deux à deux non adjacentes :

$$\forall s \in U \quad \sum_{(s,v) \in E} x_{sv} \leq 1 \quad (47)$$

$$\forall t \in V \quad \sum_{(u,t) \in E} x_{ut} \leq 1 \quad (48)$$

- l'objectif est de maximiser le cardinal du couplage :

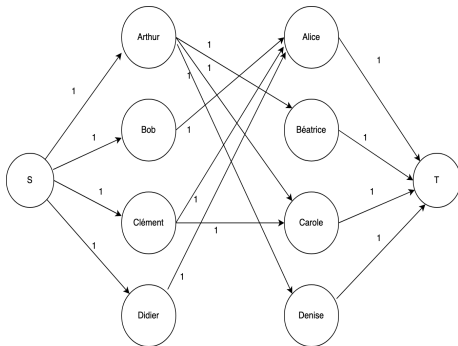
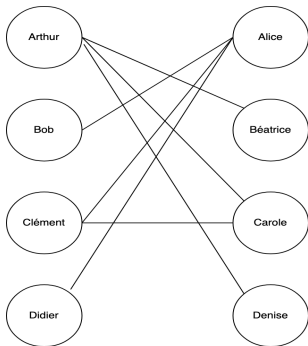
$$\max z = \sum_{(u,v) \in E} x_{uv} \quad (49)$$

Les équations 47, 48 et 49 représentent un programme linéaire (PL).

## Le problème de couplage maximum résolu comme un problème de flot maximum

Soit un graphe biparti  $G(U \cup V, E)$ . L'ensemble des arêtes représente les couples compatibles. Il s'agit de trouver le *couplage maximum* (un couplage contenant le plus grand nombre possible d'arêtes). On peut montrer que ce problème se réduit au problème du flot maximum.

- Ci-dessous le problème de mariage modélisé comme un problème du flot maximum. Les capacités sont fixées à 1 partout.



## Le problème du flot maximum : solution

- Dans le graphe ci-dessous à gauche on cherche le nombre maximum de chemins sommet-disjoints (donc ils ne se croisent pas) de  $s$  à  $t$ .
- La solution est donnée à droite. On trouve trois chemins sommet-disjoints de  $s$  à  $t$ . Cette valeur correspond à la valeur du couplage maximum.

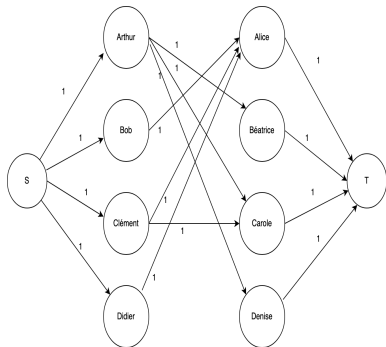


Figure 121 – On peut vérifier par énumération qu'on ne peut marier que trois couples.

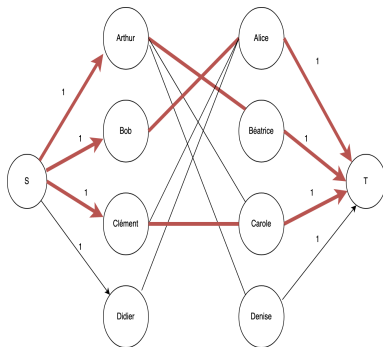


Figure 122 – La valeur du flot maximum vaut aussi 3.

### Problème des cases admissibles

Soit le tableau  $T$  de taille  $m \times n$ , dont certaines cases sont dites “admissibles” (les autres sont “non admissibles”. On se donne également  $m + n$  entiers positifs ou nuls  $l_1, \dots, l_m, c_1, \dots, c_n$ . Il s’agit d’affecter des nombres entiers aux cases admissibles (et à celles-ci seulement) de telle sorte que :

- la somme des nombres affectés aux cases d’une ligne  $i$  soit inférieure ou égale à  $l_i$  ( $i = 1, 2, \dots, m$ ).
  - la somme des nombres affectés aux cases d’une colonne  $j$  soit inférieure ou égale à  $c_j$  ( $j = 1, 2, \dots, n$ ).
  - la somme des nombres affectés aux cases du tableau soit maximum.
1. Formuler le problème des cases admissibles comme un problème de flot maximum en décrivant le graphe approprié.
  2. Formuler le même problème comme un programme linéaire.

## Problème des cases admissibles : continuation

Considérons l'instance suivante :

| C1 | C2 | C3 | C4 | C5 |    |
|----|----|----|----|----|----|
|    |    |    |    |    | L1 |
|    |    |    |    |    | L2 |
|    |    |    |    |    | L3 |
|    |    |    |    |    | L4 |

Figure 123 – Exemple du problème de cases admissibles. Les cases interdites sont en gris.

- Taille du tableau :  $m = 4$  et  $n = 5$ .
- Valeurs des lignes :  $l_1 = 9, l_2 = 10, l_3 = 15, l_4 = 2$ .
- Valeurs des colonnes :  $c_1 = 7, c_2 = 5, c_3 = 9, c_4 = 4, c_5 = 8$ .
- Cases interdites :  $t_{13}, t_{14}, t_{15}, t_{22}, t_{25}, t_{31}, t_{33}, t_{34}, t_{41}, t_{42}, t_{44}$ .

## Problème des cases admissibles vu comme un problème de flot maximum

On utilise un graphe biparti  $G = (U \cup V, E)$  où  $U = \{C1, C2, C3, C4, C5\}$  et  $V = \{L1, L2, L3, L4\}$  et l'ensemble  $E$  contient les cases admissibles. Les capacités des arcs sortants de  $S$  sont données par les bornes sup. associées aux colonnes. Les capacités des arcs entrants en  $T$  sont données par les bornes sup. associées aux lignes. On considère que les capacités des arcs intermédiaires ne peuvent pas bloquer la solution (ici 15 par exemple). Le flot maximum trouvé dans ce graphe donnera la solution du problème des cases admissibles.

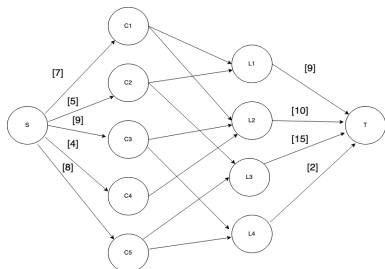


Figure 124 – Les capacités des arcs sont écrites entre crochets. Par construction, chaque flot dans ce graphe est acceptable pour le problème des cases admissibles.

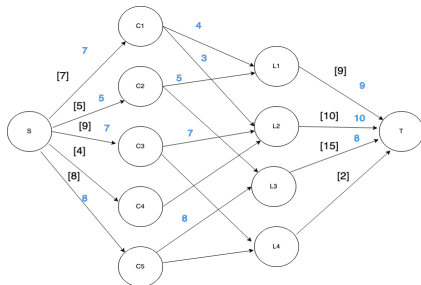
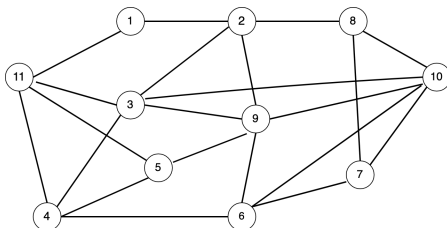


Figure 125 – Un flot de valeur 32 est visualisé en bleu. On peut démontrer que c'est le flot maximum.



## Application 5 : Fiabilité d'un réseau

On considère le réseau de télécommunication militaire donné figure ci dessous. Il est composé de onze sites (noeuds) reliés par des lignes bidirectionnelles de transmission de données.

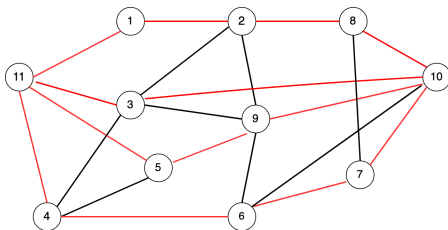


Pour des raisons de fiabilité en cas de conflit, le cahier des charges exige que les deux sites 10 et 11 de ce réseau puissent continuer à communiquer malgré la destruction de trois autres sites quelconques.

Ce réseau respecte-t-il cette exigence ?

## Fiabilité d'un réseau : modélisation

- Ce problème peut être astucieusement converti en problème de flot maximal (si les sites 10 et 11 peuvent communiquer, il suffit qu'il existe un chemin entre eux).
- On cherche, en effet, que des chemins "disjoints par sommets". Pourquoi ?
- Imaginez que les arcs aient une capacité maximale égale à 1. Le flot maximal donne alors le nombre maximum de chemins "disjoints par sommets". Que donne la coupe minimale ?
- Que peut on dire de tout nombre de sites détruits si il est inférieur à la valeur de la coupe minimale ?
- On constate que ce réseau satisfait les exigences de fiabilité puisqu'il y a quatre chemins "disjoints par sommets" de 11 à 10.



- Ecrivez un programme linéaire pour résoudre ce problème dans le cas général.

On oriente d'abord le graphe : avec  $\forall$  arête  $(i, j)$  on associe les arcs  $(i, j)$  et  $(j, i)$ . On denote par  $s$  le sommet 11 et par  $t$  le sommet 10. On cherche à maximiser le nombre de chemins de  $s$  à  $t$ .

- Les variables :  $x_{ij} = 1$  si l'arc  $(i, j)$  est utilisé par un chemin, 0 sinon ;
- On maximise le nombre de chemins sortant du sommet  $s$  :

$$\max \sum_{(s,j) \in E} x_{sj} \quad (50)$$

- La loi de conservation pour chaque sommet intermédiaire  $\forall u \in V \setminus \{s, t\}$  :

$$\sum_{(u,v) \in E} x_{uv} = \sum_{(v,u) \in E} x_{vu} \quad (51)$$

- On interdit le flot entrant à  $s$ .

$$\sum_{(u,s) \in E} x_{us} = 0 \quad (52)$$

- On cherche que des chemins "disjoints par sommets", c.a.d.

$$\sum_{(u,v) \in E} x_{uv} \leq 1 \quad \forall u \in V \setminus \{s, t\}. \quad (53)$$